

פרויקט סייבר



שם בית הספר: תיכון אלון רמת השרון

שם העבודה: easy order.

שם התלמיד: רועי אביב.

ת.ז התלמיד: 330840695

שם המנחה: עידן פלדברג, רום רפסון, עודי רז ויעקב שוצמן.

שם החלופה: הגנת סייבר

תאריך ההגשה: 21.5.25

תוכן עניינים

5	מבוא :
5	ייזום :
5	תיאור ראשוני של המערכת :
5	הגדרת הלקוח :
5	הגדרת יעדים/מטרות :
6	בעיות, תועלות וחשכונות :
6	סקירת פתרונות קיימים :
6	סקירת טכנולוגית הפרויקט :
7	תיחום הפרויקט :
8	תיאור המערכת (אפיון) :
8	תיאור המערכת :
9	פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת :
10	פירוט הבדיקות (קופסה שחורה) :
12	תכנון וניהול לויז' לפיתוח המערכת :
13	ניהול הסיכונים בפרויקט וההתמודדות איתם :
14	תיאור תחום הידע - פרק מילולי- :
14	פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר (ניתוח) :
14	פירוט היכולות בצד לקוח :
21	פירוט היכולות בצד שרת :
28	מבנה / ארכיטקטורה של הפרויקט :
28	תיאור הארכיטקטורה של המערכת- :
28	תיאור החומרה והרכיבים :
29	תיאור הטכנולוגיות הרלוונטיות :
29	פרוטוקולי תקשורת ואבטחה :
29	תחומי עניין מרכזיים בפרויקט :
30	תיאור זרימת המידע במערכת- :
30	הרשמה והתחברות למערכת :
31	תהליך שליחת קוד אימות :
31	תהליך מסך הבית של המשתמש- נפתח לאחר שהתחבר לאתר :
32	תהליך הזמנת מקום במסעדה :
33	צפייה בהזמנות קיימות :
33	מסך בית אדמין :
34	פעולות אדמין :
36	תהליך הצפנת הסיסמה :
36	תהליך שליחת מייל :

37	תיאור האלגוריתמים מרכזיים בפרויקט :
37	שם האלגוריתם : הצפנה ואבטחת סיסמאות משתמש
40	שם האלגוריתם : שליחת קוד אימות במייל (אימות דו-שלבי)
41	שם האלגוריתם : חסימת גישה לאחר ניסיונות התחברות שגויים
42	שם האלגוריתם : שליחת מיילים מהירה ולא חוסמת
42	שם האלגוריתם : העברת מידע בטוחה בין השרת ללקוח
44	שם האלגוריתם : איתור מסעדות פנויות בהתחשב בפרטי ההזמנה
45	תיאור סביבת הפיתוח
45	כלי הפיתוח הדרושים לפיתוח :
46	כלים וסביבה הדרושים לבדיקות :
47	תיאור פרוטוקול התקשורת
47	תיאור מילולי :
48	פירוט כלל ההודעות הזורמות במערכת :
50	תיאור מסכי המערכת :
50	מסך פתיחה (Welcome Screen) :
51	מסך התחברות (Log In) :
51	מסך הרשמה (Register) :
52	מסכי משתמש :
56	מסכי אדמין :
62	תרשים מסכים :
63	תיאור מבני הנתונים -
63	פירוט כללי :
63	טבלאות :
66	סקירת חולשות ואיומים
66	שכבת האפליקציה :
68	שכבת התעבורה :
69	חולשות וטיפולן -
72	מימוש הפרויקט
72	חלק א'- סקירת כל המודולים / מחלקות המרכיבים את המערכת וקשרי הגומלין ביניהם :
72	מודולים / מחלקות מיובאים :
73	מודולים / מחלקות שפיתחתי :
86	חלק ב'- קטעי קוד ייחודי ואלגוריתמים מרכזיים בפרויקט :
86	נושא : שליחת מיילים בצורה מהירה ולא חוסמת (Threading)
87	נושא : הצפנת סיסמאות כפולה (RSA → bcrypt)
87	נושא : קבלת הודעות שלמות בתקשורת Socket
88	נושא : חסימת IP לאחר ניסיונות התחברות כושלים

89	נושא : בדיקת זמינות שולחן לפי מסעדה, גודל, קטגוריה וזמן
89	נושא : שליחת הודעת אישור הזמנה יפה במייל
90	חלק ג'- מסך בדיקות מלא :
90	הבדיקות שתוכננו בשלב האפיון :
91	בדיקות נוספות :
93	מדריך למשתמש
93	פירוט כלל קבצי המערכת- עץ קבצים
93	התקנת המערכת :
93	סביבה נדרשת :
93	כלים נדרשים :
94	מיקומי קבצים :
94	נתונים התחלתיים :
94	דרישות רשת :
94	ארכיטקטורה מינימלית נדרשת :
94	משתמשי המערכת :
94	משתמש קצה (משתמש רגיל)
102	מנהל מערכת (אדמין) :
107	סיכום אישי :
107	תהליך העבודה :
107	תהליך הלמידה :
107	כלים שנלקחו להמשך :
107	תובנות :
108	בראייה לאחר :
108	שדרוגים עם משאבים נוספים :
108	שאלות חקר עצמי :
108	תודות :
109	ביבליוגרפיה
110	נספחים :
110	קוד הפרויקט :
110	Server:
119	Database:
128	Communication:
130	Rsa:
132	Mail:
135	client_module:
136	Gui:

מבוא:**ייזום:****תיאור ראשוני של המערכת:**

המערכת המוצעת היא אפליקציה להזמנת מקומות במסעדות, שמספק פתרון חכם על ידי שימוש בפילטרים שונים. המערכת מאפשרת למשתמשים להזמין מקום במסעדות על בסיס קריטריונים כמו סוג אוכל, מיקום, תאריך, שעה מספר סועדים, כשרות, דירוג ומחיר ממוצע לסועד.

המערכת תספק חווית משתמש נוחה ואינטואיטיבית, ובמקום לעבור בין מסעדות שונות ולבדוק זמינות באופן ידני, המערכת תציג את כל המסעדות שעומדות בקריטריונים שהוזנו. מדובר במערכת דמה שעובדת על מסעדות מומצאות, אשר נוצרו במאגר נתונים דמוי-מציאותי. המאגר כולל נתונים על סוגי אוכל, מיקומים גיאוגרפיים, זמינות שולחנות ושעות פעילות של המסעדות, ומטרתו לדמות את תהליך ההזמנה בצורה ריאליסטית, ללא חיבור למסעדות אמיתיות.

בחרתי בפרויקט זה בגלל שנתקלתי בבעיה אישית בהזמנת מקומות במסעדות דרך האתרים הקיימים, בהם לא היה ניתן לבצע חיפוש מותאם אישית שמספק פתרון מהיר ויעיל.

האתגרים הצפויים בפרויקט הם :

1. **יצירת חווית משתמש נוחה** – יצירת מערכת שתהיה ידידותית ונגישה עבור כל סוגי המשתמשים, גם אלו שלא בהכרח טכנולוגיים או מנוסים בשימוש באתרים.
2. **עבודה עם מאגר נתונים רחב ומאובטח** – מאגר הנתונים יהיה חשוב מאוד, כי הוא יכיל את כל המידע אודות המסעדות, זמינותן, פרטי המשתמשים וכו'. שהמאגר יהיה מדויק, עדכני ויעיל, וכן צריך להבטיח שהוא מאובטח כדי למנוע חשש לדליפות מידע אישי או פרטי הזמנות.
3. **יכולת לעבד מידע בזמן אמת בצורה מהירה ואפקטיבית** – חלק חשוב במערכת יהיה היכולת לעבד ולהציג נתונים בזמן אמת. לדוגמה, אם הוזמן או בוטל מקום במסעדה מסוימת, המערכת צריכה לעדכן את המשתמשים במהירות, אחרת הם עלולים לפספס מסעדות שיש להן מקום פנוי בסופו של דבר או להזמין מקום שלא זמין.
4. **שימוש במאגר נתונים דמוי-מציאותי** – יש צורך להקים ולהתחזק מאגר נתונים מומצאותי, שיכלול את כל המידע הדרוש לגבי המסעדות המומצאות, תוך שמירה על עדכניות הנתונים והסימולציה של חווית הזמנה אמיתית.

הגדרת הלקוח:

המערכת מיועדת לכל אדם שמעוניין להזמין מקום במסעדה בצורה נוחה ויעילה, במיוחד אלו אשר מתמודדים עם בעיית זמן ואי נוחות בתהליך החיפוש וההזמנה של מקום פנוי במסעדות. הקהל כולל אנשים בכל הגילאים, בעיקר אלו שלא רוצים לבזבז זמן בתהליך חיפוש ממושך.

הגדרת יעדים/מטרות:

המערכת מטרתה לאפשר :

1. הזמנה מהירה וממוקדת : חיפוש מסעדות לפי פילטרים מדויקים (סוג אוכל, מיקום, שעה, תאריך).
2. שיפור חווית המשתמש : יצירת ממשק נוח וידידותי לכל סוגי המשתמשים.
3. יעול תהליך הזמנת המקום : פתרון מהיר וממוקד לזמינות מסעדות בצורה חכמה.
4. שמירה על פרטיות ואבטחת מידע : הגנה על נתוני המשתמשים תוך שמירה על אבטחה גבוהה.

בעיות, תועלות וחסכוניות:

בעיה: כיום תהליך הזמנת מקום במסעדה הוא מסורבל, דורש חיפוש ידניים ובזבוז זמן.

תועלות: המערכת תספק פתרון מהיר וממוקד, תיעל את תהליך ההזמנה ותספק חווית משתמש נוחה. חסכוניות:

1. **חיסכון בזמן למשתמשים:** המשתמשים חוסכים זמן יקר בכך שלא צריכים לערוך חיפוש ידני ואיטי למקום פנוי במסעדות השונות ויכולים להזמין מקום בצורה מיידית ונוחה.
2. **סנכרון מידע עם מסעדות בזמן אמת:** כל המסעדות שמשתמשות במערכת ייהנו מתיאום מדויק ויעיל יותר עם לקוחותיהם, דבר שיכול להפחית טעויות ולהגביר את שביעות הרצון של הלקוחות.
3. **הגדלת הכנסות עבור בעלי מסעדות:** באמצעות המערכת, מסעדות יוכלו למקסם את ההזמנות על פי זמינות השולחנות, ולנצל את כל יכולת ההושבה שלהן בצורה אופטימלית.

סקירת פתרונות קיימים:

היום קיימים מספר אתרים ואפליקציות המאפשרים הזמנת מקום במסעדות, כמו *TripAdvisor*, *Rest.co.il* ו-*OpenTable* למרות זאת, אתרים אלו לא תמיד מציעים חיפוש חכם וממוקד שמסנן את המסעדות לפי קריטריונים אישיים מדויקים. לדוגמה, אתרים רבים מציעים אפשרות לחפש מסעדות לפי סוג אוכל או מיקום, אך לא תמיד יש אפשרות לסנן את התוצאות על פי פרמטרים נוספים כמו מקום פנוי, שעה, תאריך ומספר סועדים.

כמו כן, אתרים רבים אינם מציעים חוויית משתמש נוחה ואינטואיטיבית. חוויית המשתמש באתרי הזמנות קיימים לעיתים קרובות אינה מספיק פשוטה או נוחה, במיוחד למי שאינו טכנולוגי או מנוסה בשימוש באתרים כאלה.

סקירת טכנולוגית הפרויקט:

הפרויקט מבוסס על טכנולוגיות מוכרות מעולם התכנות והסייבר, אך עבורי כתלמיד מדובר בשילוב חדש, מורכב ובלתי מוכר בתחילת הדרך. הטכנולוגיות שנבחרו דרשו הבנה עמוקה, חקר עצמאי והתנסות מעשית שלא הייתה לי קודם.

טכנולוגיות חדשות עבורי:

- **RSA** – מימוש ידני של הצפנה אסימטרית במערכת קצה-לקוח, כולל ניהול מפתחות.
- **bcrypt** – הצפנה חד-כיוונית מתקדמת, כולל שימוש ב־salt לאבטחת סיסמאות.
- **SMTP עם TLS (starttls)** – שליחת מיילים בצורה מאובטחת דרך חיבור מוצפן.
- **CustomTkinter** ספריית GUI מתקדמת שלא הכרתי לפני כן, שנדרשה לבניית ממשק משתמש מודרני.
- **socket** – כולל שליחת מידע בפורמט JSON והגנה מ־DDoS

סייגים והגבלות בפרויקט:

במהלך העבודה התגלו מספר מגבלות ואתגרים טכנולוגיים, ביניהם:

- אבטחת תקשורת מלאה (TLS/HTTPS) לא יושמה בכל המערכת – בשל מגבלות תשתית, לא שולב פרוטוקול HTTPS מלא לתקשורת כללית, אלא רק TLS לשליחת מיילים. אם היה לי שרת אינטרנטי אמיתי – הייתי משלב HTTPS
- שליחת מיילים תלויה בגישה לחשבון Gmail עם App Password – פתרון זה אינו אידיאלי לפרודקשן ודורש ניהול זהיר של פרטי התחברות.

- מסד הנתונים מבוסס – SQLite מתאים לפרויקט בודד, אך לא למערכת רשת עם עומס גבוה או משתמשים מרובים ברזמנית.
- הצפנת RSA מבוצעת ידנית בקוד – לא נעשה שימוש בספריית קריפטוגרפיה מוכרת, מה שעלול להיות פחות בטוח לעומת מימושים סטנדרטיים.
- ממשק GUI מיועד לדסקטופ בלבד – אין גרסת ווב או מובייל, דבר שמגביל את נגישות המשתמשים.

סיכום:

למרות שהשתמשתי בטכנולוגיות קיימות, הדרך בה שולבו – ובעיקר האופן שבו יישמתי אותן בעצמי מאפס – הפך את העבודה למורכבת מאוד ומעשירה במיוחד. התמודדות עם המגבלות והסייגים תרמה לי רבות מבחינת הבנה טכנולוגית, פתרון בעיות וראייה מערכתית שלמה.

תיחום הפרויקט:

המערכת עוסקת בפיתוח פתרון טכנולוגי לניהול הזמנות מקוונות למסעדות, תוך שילוב מרכיבי רשת, ממשק משתמש, אבטחת מידע ועבודה עם מאגרי נתונים. המערכת מתבססת על מודל **Client-Server** ופועלת בתקשורת **Socket**, המאפשרת שליחת וקבלת נתונים בזמן אמת בין הלקוח לשרת. בנוסף, המערכת עושה שימוש ב-**Multithreading** לניהול תהליכים מקבילים, כגון עיבוד בקשות ברזמנית ושליחת מיילים – תוך ניצול תשתיות מערכת ההפעלה.

המערכת פותחה בשפת Python ומשתמשת ב-**CustomTkinter** ליצירת GUI אינטראקטיבי, וב-**SQLite** כמאגר נתונים מקומי לאחסון מידע על מסעדות, שולחנות, משתמשים והזמנות. היא כוללת גם מנגנוני **אבטחת מידע** כגון הצפנת RSA, Hashing ואימות דו-שלבי (MFA) כחלק מתכנון כוללני להגנה על פרטיות המשתמשים.

. כל המידע המוזן והמעובד במערכת מתעדכן בזמן אמת ומאפשר חוויית הזמנה מדויקת וממוקדת.

תחומים בהם המערכת עוסקת:

1. **מערכת ניהול הזמנות:** המערכת מתמקדת בניהול הזמנות מקוונות למסעדות, מאפשרת חיפוש מותאם אישית ומבצע עדכון מיידי של זמינות שולחנות.
2. **מודל מנהל:** המערכת כוללת מודל מנהל למנהל האפליקציה אשר מציע לאדמין פיצ'רים שונים כגון: חיפוש הזמנות קיימות לפי תאריך או שם משתמש, הוספת או מחיקת מסעדות ועוד.
3. **מערכת חיפוש חכמה:** המערכת מציעה חיפוש חכם המבוסס על פילטרים מותאמים אישית (סוג אוכל, מיקום, שעה, תאריך ומספר סועדים).
4. **מאגר נתונים דמוי-מציאותי:** המערכת פועלת עם מאגר נתונים דמוי-מציאותי המכיל נתונים על מסעדות מומצאות, אשר משקפים את הזמינות של השולחנות במסעדות.
5. **אבטחת מידע:** המערכת משתמשת בטכנולוגיות הצפנה ואבטחת מידע (RSA, Hashing) להבטחת פרטיות המשתמשים והגנה על נתוני הזמנות.
6. **מערכת Client-Server:** המערכת פועלת לפי מודל Client-Server בו הלקוח (המשתמש) מתקשר עם השרת המנהל את כל פעולתו של האתר, כולל חיפושים והזמנות.

תחומים בהם המערכת לא מטפלת:

1. **תשלום דרך האתר:** המערכת לא מטפלת בניהול תשלומים או בתשלום על הזמנות. כל ההזמנות יתבצעו ללא קשר לפעולות פיננסיות או מבצעים נוספים.
2. **הזמנות למסעדות אמיתיות:** המערכת אינה מחוברת למסעדות אמיתיות, אלא משתמשת במאגר נתונים דמוי-מציאותי המייצג את חוויית ההזמנה. הנתונים אינם אקטואליים או משקפים את המידע של מסעדות פיזיות.

3. **שירותי משלוחים או מכירות:** המערכת אינה עוסקת בניהול משלוחים או מכירות של מזון, אלא מתמקדת אך ורק בהזמנת מקומות במסעדות.
4. **שירות לקוחות או תמיכה ישירה:** המערכת אינה כוללת מערכת לתמיכה ישירה או שירות לקוחות. כל ההתנהלות מתבצעת דרך הממשק של האתר והמאגר הדמוי-מציאותי.

לסיכום, המערכת מספקת פתרון חכם לניהול הזמנות במסעדות ומיועדת למשתמשים המעוניינים להזמין מקום בצורה יעילה וממוקדת. היא אינה עוסקת בתחומים כמו תשלומים, משלוחים או תמיכה ישירה, אלא מתמקדת ביצירת חווית משתמש אינטואיטיבית וניהול הזמנות במסעדות בעזרת נתונים דמוי-מציאותיים.

תיאור המערכת (אפיון)

תיאור המערכת:

המערכת היא אתר להזמנת מקומות במסעדות, המבוסס על מאגר נתונים דמוי-מציאות, שמדמה את תהליך ההזמנה האמיתי. המשתמשים יכולים לבצע חיפוש חכם ומותאם אישית על פי קריטריונים כמו מיקום גיאוגרפי, סוג אוכל, תאריך, שעה ומספר סועדים, ולבצע הזמנות בקלות דרך הממשק.

המערכת שפותחה מדמה תהליך הזמנת מקומות למסעדות בצורה ריאליסטית, תוך שילוב בין ממשק משתמש מתקדם, מסד נתונים מפורט, ושכבות אבטחת מידע. המערכת פועלת במבנה מודולרי, המשלב GUI, תקשורת TCP, ואבטחת מידע מקצה לקצה.

המערכת פועלת במודל Client-Server, בשילוב ממשק גרפי נוח (GUI) המפותח ב־ Custom Tkinter, לוגיקה עסקית ב־ Python ומאגר נתונים מנוהל על ידי SQLite. היא כוללת גם מנגנוני אבטחת מידע מתקדמים (RSA, Hashing) ושכבת אימות כפול (MFA) לשמירה על פרטיות המשתמשים.

מאגר נתונים דמוי-מציאות:

א. מבנה הנתונים

מאגר הנתונים בנוי כך שידמה מערכת הפועלת בעולם האמיתי. הוא כולל את הטבלאות והמידע הבאים:

- **מסעדות:** שם, סוג אוכל, שעת פתיחה, שעת סגירה, משך זמן לשמירת שולחן, עיר, מחוז, מחיר ממוצע לסועד, כשרות, ודירוג. (1–5)
- **שולחנות:** המסעדה אליה שייך כל שולחן, קטגוריה (פנים / חוץ), גודל, וכמות שולחנות מאותו סוג.
- **משתמשים:** שם פרטי, שם משפחה, אימייל, סיסמה) מוצפנת ב־ (Hashing) שם משתמש, מספר טלפון, תאריך יצירה.
- **הזמנות:** המשתמש המזמין, מספר שולחן, תאריך ההזמנה, שעת התחלה, שעת סיום.
- **היסטוריית הזמנות + ביקורות:** מאפשרים מעקב ושימוש עתידי בניתוחים מתקדמים.

ב. ניהול ועדכון מידע

המערכת מתעדכנת באופן אוטומטי בהתאם לפעולות המשתמשים:

- הזמנה חדשה - עדכון זמינות שולחנות
- ביטול הזמנה - שחרור שולחן בחזרה למאגר
- פעולות מנהל (כמו הוספת מסעדה או עריכת פרטים) - משפיעות מיידית על טבלאות הנתונים

המאגר מתוחזק בפורמט מודולרי, שמאפשר **תחזוקה שוטפת, הרחבה עתידית, ושימוש חוזר** – לדוגמה עבור חיזוי ביקושים או סינון מתקדם.

ג. השפעה על חוויית המשתמש

השימוש במאגר דמוי-מציאות מאפשר:

- **חיפוש חכם וממוקד** לפי פרמטרים רלוונטיים (אזור, סוג אוכל, כשרות וכו').
- **זמינות בזמן אמת** שמציגה רק אפשרויות אמיתיות לפי שעה, תאריך וגודל שולחן.
- **הימנעות מתסכול** – המשתמש אינו מבזבז זמן על מקומות שלא מתאימים או אינם זמינים.

המערכת שואפת להעניק חוויית משתמש אינטואיטיבית, דינמית וריאליסטית – גם כשהיא אינה מחוברת למסעדות אמיתיות.

הבחירה ב-GUI ולא ב-HTML:

בחרתי לפתח את המערכת כאפליקציית GUI (ממשק גרפי למשתמש) ולא כאתר אינטרנט מבוסס HTML, מכיוון שהמערכת מבוססת על מאגר נתונים **דמוי-מציאות** שאינו מחובר למסעדות אמיתיות או לשירותים חיצוניים ברשת. מאחר ואין צורך בתמיכה בחיבור אינטרנטי למסעדות חיות או בשירותים אונליין, אין יתרון ממשי בפיתוח אתר אינטרנט.

בנוסף, פיתוח ממשק GUI מאפשר שליטה מלאה בחוויית המשתמש, תגובה מיידית ללא תלות בדפדפן או חיבור אינטרנט, ופשטות בפיתוח ואינטגרציה עם הקוד הלוגי והמאגר המקומי. לכן, עבור מערכת מקומית המדמה תהליך הזמנה, הבחירה ב-GUI היא היעילה והמתאימה.

יכולות עיקריות של המערכת:

- **לבצע חיפוש** מסעדות על פי פילטרים מגוונים: סוג אוכל, אזור גיאוגרפי, תאריך, שעה, כמות סועדים ועוד.
- **להציג רשימת מסעדות זמינות** בזמן אמת – בהתבסס על הנתונים במסד.
- **לבצע הזמנה** במספר צעדים פשוטים, תוך שמירת שולחן ועדכון מידי של זמינותו.
- **לאבטח נתונים רגישים** של המשתמשים באמצעות הצפנה כפולה RSA – בעת שליחה ו־bcrypt במסד.
- **לאמת זהות המשתמשים** באמצעות מנגנון אימות דו-שלבי (2FA) עם שליחת קוד למייל.
- **לספק ממשק גרפי אינטואיטיבי (GUI)** בעיצוב נקי, תגובתי ויזווי למשתמש.
- **להבטיח תקשורת יציבה ויעילה** דרך מודל Client–Server המבוסס על פרוטוקול TCP.
- **לאפשר ניהול מערכת מלא למנהלים** – כולל הוספה, עריכה ומחיקה של מסעדות, שולחנות והזמנות.
- **לזהות משתמשים פעילים ומסעדות פופולריות** לפי היסטוריית הזמנות – למעקב סטטיסטי.

פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת:

משתמש רגיל:

- **חיפוש מסעדות מותאם אישית**
- לפי סוג אוכל, אזור, תאריך, שעה, כמות סועדים ועוד.
- **ביצוע הזמנה**
- בחירה במסעדה וזמן ישיבה והזמנה ישירה.

- **ניהול הזמנות אישיות**
צפייה בהיסטוריית הזמנות, ביטול הזמנות עתידיות.

מנהל מערכת: (Admin)

- **חיפוש הזמנות לפי תאריך או שם משתמש**
צפייה בכל ההזמנות לפי קריטריונים.
- **זיהוי מסעדות פופולריות**
צפייה ב-5 המסעדות עם הכי הרבה הזמנות.
- **זיהוי משתמשים פעילים**
צפייה ב-5 המשתמשים עם הכי הרבה הזמנות.
- **ניהול מסעדות**
הוספה, עריכה או מחיקה של מסעדות.
- **ניהול שולחנות**
הוספה או הסרה של שולחנות לכל מסעדה.

פירוט הבדיקות (קופסה שחורה):

בדיקות קופסה שחורה נועדו לבדוק את הפלט של המערכת בהתאם לקלט, מבלי להיכנס למבנה הפנימי של הקוד. להלן פירוט הבדיקות המתוכננות לפרויקט:

1. בדיקת חיפוש מסעדות

- **מטרה:** לוודא שהמערכת מציגה תוצאות התואמות את הקריטריונים שהוזנו על ידי המשתמש.
- **ביצוע:** המשתמש יזין פרמטרים כמו סוג אוכל, מיקום, תאריך, שעה ומספר סועדים.
- **כיצד נבדוק:** נבצע סדרת חיפושים עם שילובים שונים של פרמטרים ונבדוק שהתוצאות אכן מתאימות למאפיינים שהוזנו (כולל בדיקת גבולות – תוצאות ריקות, חיפוש עתידי, חיפוש בעבר וכו').

2. בדיקת יצירת הזמנה

- **מטרה:** לוודא שהמשתמש יכול להזמין מקום בצורה תקינה ושההזמנה מתועדת במערכת.
- **ביצוע:** המשתמש יבחר מסעדה, יזין את פרטי ההזמנה (מספר סועדים, תאריך, שעה) ויבצע את ההזמנה.
- **כיצד נבדוק:**
 - נוודא שההזמנה מתווספת לטבלת ההזמנות במסד הנתונים.
 - נוודא שהמשתמש מקבל אישור.
 - נבצע בדיקות על הזמנות כפולות, שעות לא תקינות, או הזמנה על שולחן שכבר הוזמן.

3. בדיקת עריכת שולחנות (מנהל מערכת)

- **מטרה:** לוודא שהמנהל יכול להוסיף/להסיר שולחנות בצורה נכונה והמידע משתנה בהתאם.
- **ביצוע:** המנהל יעדכן שולחנות במסעדה מסוימת (הוספה/הסרה).
- **כיצד נבדוק:**
 - נבצע שינוי בשולחנות ונבצע חיפוש כדי לוודא שהשינוי מתעדכן.
 - נבדוק שהזמנה במסעדה מעודכנת בהתאם לשולחנות החדשים.
 - נבחן גם מקרה של הסרת שולחן שהוזמן ונראה שהמערכת מונעת זאת.

4. בדיקת אבטחת מידע

- **מטרה:** לוודא שהמערכת מגינה על המידע הרגיש של המשתמשים (סיסמאות, פרטי התחברות, פרטי הזמנות).
- **ביצוע:**
 - נבדוק הצפנה של סיסמאות (לדוגמה: שימוש ב-RSA וב-bcrypt).
 - נבצע בדיקות חדירה בסיסיות – (Penetration Testing) כמו ניסיון SQL Injection, פריצת סשן, גישה למידע של משתמש אחר.
- **כיצד נבדוק:**
 - נבדוק את עמידות המערכת בפני ניסיונות תקיפה.
 - נוודא שהמערכת שומרת על סטנדרטים של אבטחת מידע.

5. בדיקת ביצועים (Performance Test)

- **מטרה:** לבדוק שהמערכת מגיבה היטב תחת עומס של משתמשים ובקשות רבות.
- **ביצוע:** ניצור סימולציה של עומס על ידי הפעלת מספר תהליכים בו זמנית (Multi-threading) או כלים ייעודיים.
- **כיצד נבדוק:**
 - נמדוד את זמן התגובה של פעולות עיקריות (חיפוש, הזמנה, התחברות).
 - נבחן תפקוד המערכת תחת עומס גובר (Stress test) ונתעד את נקודות הכשל.

6. בדיקת שליחת מיילים (לשכבת האימות והתקשורת)

- **מטרה:** לוודא שמשתמשים מקבלים מיילים רלוונטיים בזמנים הנכונים (לצורך אבטחה ומידע).
- **ביצוע:**
 - נבצע פעולות הדורשות שליחת מייל, כגון:
 - אימות דו-שלבי בזמן התחברות
 - אישור על הזמנה חדשה
 - אישור על ביטול הזמנה
- **כיצד נבדוק:**
 - נבדוק את קבלת המיילים בפועל בתיבת הדואר.
 - נוודא שהתוכן נכון ומגיע בזמן.
 - נבצע בדיקות על שליחה למיילים שגויים או שאינם קיימים ונבחן את תגובת המערכת.

תכנון וניהול לו"ז לפיתוח המערכת:**לו"ז ראשוני:**

1. **אפיון דרישות ותכנון בסיסי**
בשלב זה נגדיר את מטרות המערכת, קהל היעד, יכולות המשתמשים, נפרט את טבלאות מסד הנתונים ונתכנן את מסכי ה-GUI הראשיים.
תאריך משוער: 1/3/2025 עד 7/3/2025 (שבוע עבודה)
2. **בניית מסד נתונים**
יצירת טבלאות: משתמשים, מסעדות, שולחנות, הזמנות, היסטוריית הזמנות, ביקורות. הזנת נתוני דוגמה לצורך בדיקות.
תאריך משוער: 8/3/2025 עד 13/3/2025 (6 ימים)
3. **פיתוח מסך התחברות והרשמה**
פיתוח מסך התחברות למשתמש קיים, רישום משתמש חדש, בדיקות תקינות בסיסיות של טפסים והוספת הצפנה בסיסית לסיסמאות.
תאריך משוער: 14/3/2025 עד 20/3/2025 (שבוע)
4. **פיתוח חיפוש מסעדות והצגת תוצאות**
יצירת מסך לחיפוש מסעדות לפי מיקום, סוג אוכל, תאריך, שעה ומספר סועדים. הצגת תוצאות ממוסד הנתונים.
תאריך משוער: 21/3/2025 עד 27/3/2025 (שבוע)
5. **פיתוח מערכת הזמנות**
פיתוח מערכת שמאפשרת למשתמש להזמין שולחן במסעדה בהתאם לזמינות. שמירת ההזמנה במסד ויצירת בדיקות מניעת כפילות.
תאריך משוער: 28/3/2025 עד 5/4/2025 (9 ימים)
6. **פיתוח ממשק למנהל מערכת (Admin)**
הוספת יכולות ניהוליות: צפייה בהזמנות לפי תאריך/משתמש, ניהול מסעדות, ניהול שולחנות, צפייה במשתמשים פעילים ובמסעדות פופולריות.
תאריך משוער: 6/4/2025 עד 13/4/2025 (שבוע)
7. **הוספת שכבת אבטחה מתקדמת**
הוספת הצפנת RSA, Hashing, עם bcrypt והטמעת אימות דו-שלבי (קוד למייל) למשתמשים.
תאריך משוער: 14/4/2025 עד 20/4/2025 (שבוע)
8. **עיצוב הממשק הגרפי (GUI)**
שיפור הנראות של המסכים, עיצוב אחיד ונעים לעין, התאמת חוויית משתמש נוחה וברורה.
תאריך משוער: 21/4/2025 עד 25/4/2025 (5 ימים)
9. **בדיקות מערכת (קופסה שחורה)**
בדיקת כל הפונקציות במערכת: חיפוש, הרשמה, הזמנות, ביטול, מיילים, ניהול ועוד. כולל בדיקות אבטחה וביצועים.
תאריך משוער: 26/4/2025 עד 2/5/2025 (שבוע)
10. **הכנת פרזנטציה ודוח מסכם**
הכנת הדגמה להצגת הפרויקט, כתיבת דוח מסכם מסודר ותיעוד כל החלקים שבוצעו.
תאריך משוער: 3/5/2025 עד 7/5/2025 (5 ימים)

לו"ז בפועל:

1. **אפיון דרישות ותכנון בסיסי** – בוצע בשבוע הראשון של תחילת הפרויקט. כלל הגדרת מטרות, תכנון בסיסי של הפתרון, והגדרת מבנה מסד הנתונים.
2. **בניית מסד נתונים** – בוצע מיד לאחר שלב האפיון. נבנה מסד נתונים הכולל את כל הטבלאות הדרושות: מסעדות, שולחנות, משתמשים, הזמנות, ביקורות ועוד.
3. **יצירת מערכת תקשורת שרת-לקוח בסיסית** – לאחר הקמת המסד, נבנתה תשתית לתקשורת בין צד השרת לצד הלקוח כדי לאפשר שליחה וקבלה של נתונים.
4. **פיתוח מסך התחברות והרשמה** – נבנו ממשקים גרפיים לכניסת משתמשים ולרישום משתמשים חדשים, כולל קישור למסד הנתונים.

5. **יצירת מחלקת GUI ששולפת נתונים מהמאגר** –נבנתה מחלקת GUI שתומכת בתצוגת נתונים בצורה גרפית מתוך המאגר, בהתאם לקלט של המשתמש.
6. **יצירת מסך בסיסי להזנת קריטריונים וחיפוש מסעדות מתאימות** –המשתמש יכול להזין קריטריונים (כמו אזור, סוג אוכל, תאריך, שעה) ולקבל תוצאות מותאמות ממסד הנתונים.
7. **הוספת שכבת אבטחה** –
- א. שילוב הצפנת RSA להעברת סיסמאות בין לקוח לשרת.
 - ב. שימוש ב Hashing-להצפנת סיסמאות המאוחסנות במסד הנתונים.
 - ג. הגנה בסיסית בפני מתקפות DDoS.
 - ד. הוספת אימות דו-שלבי (2FA) באמצעות שליחת קוד למייל המשתמש.
8. **שילוב הודעות מייל מתאימות** - יצירת מחלקת מייל ושליחת הודעות למשתמש של אישור ביטול הזמנה קיימת או אישור הזמנה חדשה.
9. **עיצוב הממשק הגרפי** –שופרו המרכיבים הוויזואליים של המערכת כך שהממשק יהיה ברור, ידידותי ונעים לשימוש.
10. **בדיקות קופסה שחורה** –בוצעו לאורך שלבי הפיתוח לכל פונקציונליות שהתווספה, כדי לוודא תקינות ודיוק של פעולות המערכת.
11. **הכנת ספר פרויקט** –לאחר סיום הפיתוח, הוכן ספר פרויקט מלא הכולל את כל מרכיבי המערכת, תהליך העבודה, תרשימים, קוד ודוחות.
- ניהול הסיכונים בפרויקט וההתמודדות איתם:
1. **סיכון: עומס בזמן והגעה לא מוכנה לדדליין**
 - **תכנון:** ניהול לוח ברור עם חלוקה לשלבים מוגדרים מראש, עבודה רציפה מדי שבוע והגדרת אבני דרך.
 - **בפועל:** עמדת בלוחות הזמנים באמצעות עבודה שוטפת והשלמת כל שלב לפני המעבר לבא. כל משימה פוצלה לתת-משימות שבוצעו בהתאם.
 2. **סיכון: בעיות תקשורת בין שרת ללקוח**
 - **תכנון:** תכנון מוקדם של מבנה ההודעות והפרוטוקול בין השרת ללקוח, כולל טיפול בשגיאות.
 - **בפועל:** נבנתה תשתית תקשורת ברורה עם טיפול בחריגות וניהול תרחישים של חוסר זמינות או שגיאות בתקשורת.
 3. **סיכון: סיכונים אבטחת מידע (גניבת סיסמאות, חדירה למערכת)**
 - **תכנון:** שילוב טכניקות הצפנה מתקדמות (RSA, Hashing) והוספת אימות דו-שלבי.
 - **בפועל:** יושמה הצפנת RSA להעברת סיסמאות, הצפנת bcrypt למסד הנתונים, אימות דו-שלבי במייל, והוספת הגנות בסיסיות מפני מתקפות.
 4. **סיכון: תקלות או חוסר התאמה במסד הנתונים**
 - **תכנון:** תכנון מוקדם ומדויק של סכמת מסד הנתונים בהתאם לצרכים העתידיים.
 - **בפועל:** בוצעה בנייה הדרגתית של המסד תוך בדיקות שוטפות מול הפונקציות ב GUI- ובשרת. תוקנו מקרים בהם נדרשו התאמות בין טבלאות או עמודות.
 5. **סיכון: בעיות שליחת מייל (SMTP) סינון ספאם**
 - **תכנון:** שימוש בשרת SMTP אמין, בדיקות חוזרות ושליחת מיילים רק כאשר הכרחי.
 - **בפועל:** הוקמה מחלקת שליחה יציבה עם טיפולים בשגיאות, ווידוא שמילים מגיעים לתיבת דואר ולא לספאם.
 6. **סיכון: ממשק משתמש מסורבל או לא ברור**
 - **תכנון:** תכנון ממשק פשוט, אחיד ונוח מההתחלה, כולל בדיקות שימושיות.
 - **בפועל:** לאחר שלבי הפיתוח הראשוניים שופרו המרכיבים הגרפיים, נוספה אחידות בעיצוב והמשוב מהמשתמש נלקח בחשבון.
 7. **סיכון: תקיעת תהליך עקב באגים לא צפויים**
 - **תכנון:** ביצוע בדיקות קופסה שחורה לאורך כל הדרך, כולל בדיקות שוליות לתרחישים חריגים.
 - **בפועל:** בוצעו בדיקות בכל שלב, תוקנו באגים מיידית תוך כדי הפיתוח, והמערכת נבדקה היטב לקראת הסיום.

תיאור תחום הידע - פרק מילולי-

פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר (ניתוח):

פירוט היכולות בצד לקוח:

יכולות משתמש רגיל בצד לקוח:

שם היכולת: הרשמה למערכת

- **מהות:** רישום משתמש חדש על ידי הזנת שם פרטי, שם משפחה, שם משתמש, סיסמה ודוא"ל, עם אימות והצפנה.

יכולות נדרשות:

- ממשק משתמש – טופס רישום
- קליטת נתונים מהשדות
- בדיקת תקינות של שדות
- בדיקת חוזק סיסמה
- הצפנת הסיסמה ב-RSA
- שליחת הבקשה לשרת
- קבלת תשובה
- הצגת הודעה מתאימה למשתמש
- **אובייקטים נחוצים:** ממשק משתמש, הצפנה/פענוח, תקשורת, בסיס נתונים.

שם היכולת: יצירת סיסמה חזקה

- **מהות:** במהלך תהליך ההרשמה, המערכת בודקת את חוזק הסיסמה שהוזנה על ידי המשתמש ומונעת המשך תהליך במקרה של סיסמה חלשה.

אוסף יכולות נדרשות:

- קליטת סיסמה מהמשתמש דרך ממשק ההרשמה.
- בדיקת חוזק הסיסמה לפי קריטריונים מוגדרים:
 - אורך של לפחות 8 תווים
 - מכילה אותיות גדולות
 - מכילה אותיות קטנות
 - כוללת תווים מיוחדים
- הצגת רמת החוזק בזמן אמת (חזקה / בינונית / חלשה).
- עצירת התהליך והצגת שגיאה במקרה של סיסמה חלשה.
- המשך ההרשמה רק כאשר הסיסמה עומדת בדרישות.

אובייקטים נחוצים: ממשק משתמש, הצפנה/פענוח, תקשורת.

שם היכולת: אימות דו-שלבי

מהות: לאחר התחברות עם שם משתמש וסיסמה, המשתמש מתבקש להזין קוד אימות חד-פעמי (8 ספרות) שנשלח למייל שלו. רק לאחר הזנת קוד תקין – הוא מקבל גישה למערכת.

אוסף יכולות נדרשות:

- קליטת שם משתמש וסיסמה בטופס ההתחברות.
 - הצפנת הסיסמה לפני שליחתה לשרת.
 - קבלת מייל המשתמש מהשרת במקרה של התחברות תקינה.
 - פתיחת חלון ייעודי להזנת קוד האימות.
 - שליחת קוד אימות רנדומלי למייל של המשתמש (בת'רד נפרד).
 - הצגת טיימר עד פקיעת הקוד (שתי דקות).
 - אפשרות לשליחה חוזרת של הקוד.
 - בדיקה מקומית של קוד שהוזן מול הקוד שנשלח.
 - מעבר למסך הראשי במקרה של הצלחה או הודעת שגיאה במקרה של כישלון.
- אובייקטים נחוצים:** ממשק משתמש, תקשורת, בסיס נתונים, שליחת מייל, תהליך רקע (threading).

שם היכולת: שליחה חוזרת של קוד אימות

מהות: כאשר המשתמש לא מקבל או לא מזין בזמן את קוד האימות, הוא יכול ללחוץ על כפתור לשליחה חוזרת של הקוד למייל.

אוסף יכולות נדרשות:

- כפתור "Send Again" במסך האימות.
 - שליחה מחודשת של קוד למייל המשתמש.
 - איפוס טיימר האימות.
- אובייקטים נחוצים:** ממשק משתמש, שליחת מייל, תהליך רקע (threading)

שם היכולת: התחברות למערכת

- **מהות:** בדיקת פרטי כניסה קיימים ושליחת בקשה לשרת, כולל קוד אימות דו-שלבי.
- **יכולות נדרשות:**

- ממשק משתמש – טופס התחברות
- קליטת שם משתמש וסיסמה

- הצפנה באמצעות RSA
- שליחת הפרטים לשרת
- קבלת תשובה אם המשתמש קיים במערכת
- שליחה וקליטה של קוד אימות שנשלח למייל
- בדיקה אם הקוד שהתקבל תקין והודעת אישור כניסה
- **אובייקטים נחוצים** : ממשק משתמש, תקשורת, הצפנה/פענוח, תקשורת, בסיס נתונים.

שם היכולת: חיפוש מסעדות

- **מהות** : מאפשר למשתמש לבצע חיפוש מסעדות לפי קריטריונים מותאמים אישית.
- **יכולות נדרשות** :
 - ממשק משתמש (GUI) עם שדות להזנת הקריטריונים : מספר אנשים, תאריך, שעה, סוג אוכל, אזור, כשרות, דירוג, טווח מחיר, וקטגוריית ישיבה.
 - בדיקת תקינות קלט (מספרים תקינים, תאריך ושעה חוקיים וכו').
 - שליחת הבקשה בפורמט JSON לשרת דרך ClientConnection.
 - הצגת הודעת שגיאה במקרה של קלט שגוי או חוסר תגובה מהשרת.
 - קבלת רשימת מסעדות מתאימות מהשרת והפניה ליכולת הבאה — *עיון במסעדות*.
- **אובייקטים נחוצים** : ממשק משתמש, תקשורת, בסיס נתונים

שם היכולת: עיון במסעדות העומדות בקריטריונים

- **מהות** : הצגת רשימה של מסעדות זמינות לפי הבחירה שבוצעה בשלב הקודם.
- **יכולות נדרשות** :
 - יצירת חלון חדש (Toplevel) המציג את תוצאות החיפוש.
 - ממשק גלילה עם כפתורי בחירה עבור כל מסעדה, כולל תיאור קצר : שם וזמינות מקומות ישיבה (פנים/חוץ/שניהם/לא זמין).
 - שמירת מיפוי בין מזהה המסעדה לפרטי המסעדה לצורך שליפת מידע בהמשך.
 - כפתור "המשך" שמעביר ליכולת הבאה — *הצגת פרטים על מסעדה נבחרת*.
- **אובייקטים נחוצים** : ממשק משתמש, תקשורת, בסיס נתונים.

שם היכולת: הצגת פרטים על מסעדה נבחרת

- **מהות** : לאחר בחירת מסעדה מהרשימה, מוצגים למשתמש פרטי המסעדה לצורך בחירה סופית.
- **יכולות נדרשות** :

- שליחת בקשה לשרת לקבלת פרטים מורחבים על המסעדה לפי שמה.
- קבלת תשובת עם פרטי המסעדה : עיר, סוג אוכל, דירוג.
- פתיחת חלון חדש המציג את המידע בעיצוב מסודר ואסתטי.
- אפשרות המשך לשלב הבא : בחירת קטגוריית ישיבה (לא מוצג כאן אך נתמך בקוד).
- **אובייקטים נחוצים** : ממשק משתמש, תקשורת, בסיס נתונים

שם היכולת: בחירת קטגוריית ישיבה

מהות :כאשר המשתמש לא ציין מראש העדפה לקטגוריית ישיבה (פנים / חוץ), המערכת מאפשרת לו לבחור את העדפתו לאחר בחירת המסעדה, בהתאם למה שהמסעדה מציעה בפועל.

אוסף יכולות נדרשות:

- בדיקה אם לא התקבלה העדפה לקטגוריית ישיבה, במידה ולא מתבצעת החלטה על המשך התהליך.
- במידה והמסעדה מציעה רק סוג ישיבה אחד (פנים או חוץ), המערכת בוחרת אוטומטית עבור הלקוח.
- אם המסעדה מציעה גם פנים וגם חוץ – נפתח חלון חדש שבו המשתמש בוחר בעצמו.
- עדכון ערך קטגוריית הישיבה לפי בחירת המשתמש.
- מעבר לשלב הבא : הצגת כפתור לביצוע הזמנה.
- **אובייקטים נחוצים** : ממשק משתמש, לוגיקה למימוש היכולת.

שם היכולת: הזמנת שולחן וקבלת אישור במייל

מהות :לאחר בחירת מסעדה, זמן וישיבה – המשתמש יכול לבצע הזמנה מלאה דרך המערכת. ההזמנה נשלחת לשרת, נשמרת במסד הנתונים ובמידה והפעולה מצליחה, נשלח מייל אישור למשתמש.

אוסף יכולות נדרשות:

- פתיחת חלון הזמנה שמציג סיכום של פרטי ההזמנה שנבחרו (מסעדה, תאריך, שעה, מספר סועדים, קטגוריית ישיבה).
- לחיצה על כפתור "Book Table" ששולחת את פרטי ההזמנה לשרת.
- קבלת תגובה מהשרת – הצלחה או שגיאה.
- במקרה של הצלחה :
 - הצגת חלון אישור להזמנה הכולל סיכום מלא.
 - שליחת מייל אישור למשתמש בת'רד נפרד, הכולל את פרטי ההזמנה.
- **אובייקטים נחוצים** : ממשק משתמש, תקשורת, בסיס נתונים, שליחת מייל, תהליך רקע (threading)

שם היכולת: צפייה בכלל ההזמנות שבוצעו באתר

מהות: הצגת רשימת ההזמנות שביצע המשתמש בעבר ובזמן הקרוב, כולל תאריכים, שעות, שמות מסעדות ומיון לפי זמן ההזמנה.

אוסף יכולות נדרשות:

- שליחת בקשה לשרת עם שם המשתמש לקבלת רשימת ההזמנות שבוצעו.
 - קבלת נתונים על כלל ההזמנות של המשתמש.
 - מיון ההזמנות לפי תאריך ושעה באמצעות פונקציית עזר.
 - הצגת ההזמנות בחלון ייעודי עם עיצוב אחיד, כולל מסגרות גלילה ועיצוב כרטיסים.
 - הבחנה בין הזמנות עבר והזמנות עתידיות לצורך המשך טיפול.
- אובייקטים נחוצים:** ממשיק משתמש, תקשורת, תצוגת נתונים, (ScrollableFrame) עיבוד תאריכים ובסיס נתונים.

שם היכולת: ביטול הזמנות עתידיות

מהות: אפשרות למשתמש לבטל הזמנות שעדיין לא הגיע זמןן, באמצעות לחצן ייעודי לצד כל הזמנה עתידית.

אוסף יכולות נדרשות:

- הצגת כפתור "Cancel Reservation" ליד הזמנות עתידיות בלבד.
 - פתיחת חלון אישור לפני ביצוע הביטול בפועל.
 - שליחת בקשה לשרת עם מזהה ההזמנה לביצוע הביטול.
 - קבלת תגובה מהשרת ואימות ביצוע הביטול.
 - שליחת מייל אוטומטי למשתמש המאשר את הביטול.
 - ריענון המסך והצגת רשימת ההזמנות לאחר העדכון.
- אובייקטים נחוצים:** ממשיק משתמש, תקשורת, בסיס נתונים, שליחת מייל, תהליך רקע (threading)

שם היכולת: ניווט בין המסכים

מהות: המערכת מאפשרת למשתמש לעבור בין מסכי המערכת (הרשמה, התחברות, תצוגת הזמנות, מסך ראשי), כולל חזרה אחורה במידת הצורך.

אוסף יכולות נדרשות:

- הצגת מסך התחברות והרשמה בתחילת השימוש.
- מעבר אוטומטי למסך הראשי לאחר התחברות מוצלחת.
- הצגת מסך הזמנות בלחיצה על כפתור צפייה.
- כפתור "Back" במסך ההזמנות לחזרה למסך הראשי.

אובייקטים נחוצים: ממשיק משתמש**יכולות אדמין צד לקוח:****שם היכולת: צפייה בהזמנות לפי תאריך****מהות:** מאפשר לאדמין לצפות בכל ההזמנות שהתבצעו בתאריך מסוים.**אוסף יכולות נדרשות:**

- פתיחת חלון לבחירת תאריך
- שליחת הבקשה לשרת עם התאריך שנבחר
- קבלת ההזמנות הקשורות לתאריך והצגתן
- תצוגה מעוצבת של ההזמנות בחלון נפרד

אובייקטים נחוצים: ממשיק משתמש, תקשורת, תצוגת נתונים, בסיס נתונים**שם היכולת: צפייה בהזמנות לפי משתמש****מהות:** מאפשר לאדמין להכניס שם משתמש ולקבל את כל ההזמנות שביצע.**אוסף יכולות נדרשות:**

- פתיחת חלון להזנת שם משתמש
- שליחת הבקשה לשרת עם שם המשתמש
- קבלת תוצאות הצפייה מהשרת
- הצגת ההזמנות בחלון מעוצב

אובייקטים נחוצים: ממשיק משתמש, תקשורת, תצוגת נתונים

שם היכולת: צפייה במסעדות הפופולריות ביותר

מהות: הצגת המסעדות עם מספר ההזמנות הגבוה ביותר מכלל המשתמשים.

אוסף יכולות נדרשות:

- שליחת בקשת צפייה לשרת
 - קבלת מידע מסודר מהשרת
 - הצגה ויזואלית של המסעדות הפופולריות
- אובייקטים נחוצים:** ממשק משתמש, תקשורת, תצוגת נתונים

שם היכולת: צפייה במשתמשים הפעילים ביותר

מהות: מאפשר לאדמין לראות את 5 המשתמשים שביצעו הכי הרבה הזמנות.

אוסף יכולות נדרשות:

- שליחת בקשה לשרת
 - קבלת רשימת המשתמשים
 - הצגת הרשימה בעיצוב גרפי
- אובייקטים נחוצים:** ממשק משתמש, תקשורת, תצוגת נתונים

שם היכולת: עריכת מסעדות

מהות: מאפשר לאדמין לבצע שינויים בפרטי מסעדות קיימות (שם, מיקום, דירוג וכו').

אוסף יכולות נדרשות:

- ממשק לבחירת מסעדה
- טופס עריכה
- שליחה לשרת וביצוע עדכון

אובייקטים נחוצים: ממשיק משתמש, תקשורת, בסיס נתונים

שם היכולת: ניהול שולחנות

מהות: הוספה או הסרה של שולחנות זמינים במסעדות קיימות לפי קטגוריה וגודל.

אוסף יכולות נדרשות:

- פתיחת חלון לניהול שולחנות
- כפתורים להוספה והסרה
- שליחת הבקשות לשרת
- עדכון בהתאם לתשובת השרת

אובייקטים נחוצים: ממשיק משתמש, תקשורת, בסיס נתונים

פירוט היכולות בצד שרת:

ניהול משתמשים ואבטחה:

שם היכולת: הרשמה לשרת

מהות: קליטת פרטי משתמש, פענוח סיסמה, הצפנה ושמירה במסד הנתונים.
אוסף יכולות נדרשות:

- קבלת בקשה מהלקוח
- פענוח סיסמה באמצעות RSA
- הצפנת סיסמה ב־bcrypt
- בדיקת ייחודיות המשתמש
- שמירה המשתמש במסד נתונים
- החזרת תגובה ללקוח

אובייקטים נחוצים: תקשורת, הצפנה/פענוח, בסיס נתונים

שם היכולת: התחברות לשרת

מהות: אימות משתמש קיים על ידי בדיקת שם משתמש וסיסמה מוצפנת.
אוסף יכולות נדרשות:

- קבלת בקשת התחברות
- פענוח סיסמה
- בדיקת התאמה למסד נתונים
- מעקב אחרי ניסיונות כושלים של IP
- החזרת תגובת הצלחה או כישלון

אובייקטים נחוצים: תקשורת, הצפנה/פענוח, בסיס נתונים, ניהול ניסיונות

שם היכולת: הגנה מפני מתקפות DDoS

מהות: מניעת עומסי תקשורת זדוניים על ידי הגבלת קצב בקשות מכתובת IP מסוימת.

אוסף יכולות נדרשות:

• ניתוח תדירות הבקשות הנכנסות לפי IP

• זיהוי תבניות חריגות

• חסימת זמנית של לקוחות שחרגו מהמגבלה

אובייקטים נחוצים: תקשורת, ניהול זיכרון זמני, ניהול זמן מערכת

שם היכולת: חסימת התחברות לאחר ניסיונות שגויים

מהות: חיזוק אבטחת המערכת על ידי מניעת ניסיונות התחברות חוזרים מכשלים.

אוסף יכולות נדרשות:

• שמירת מונה ניסיונות התחברות לפי IP

• זיהוי חריגה מהמגבלה

• חסימת גישה זמנית והחזרת הודעת שגיאה מתאימה

אובייקטים נחוצים: תקשורת, זיכרון זמני, בסיס נתונים

שם היכולת: טיפול בשגיאות מערכת

מהות: ניהול מקרים בהם מתקבלת בקשה לא תקינה מהלקוח, כולל תשובות עם קוד שגיאה והודעה מפורשת.

אוסף יכולות נדרשות:

- עטיפת כל פעולה בtry/except
- שליחת הודעות שגיאה אחידות בפורמט JSON
- כתיבת שגיאות ללוג פנימי (לשיפור עתידי)

אובייקטים נחוצים: תקשורת, ניהול שגיאות, קונסול

תפעול מערכת הזמנות:**שם היכולת: חיפוש מסעדות**

מהות: שליפת רשימת מסעדות מהמסד לפי קריטריונים שהוזנו.
אוסף יכולות נדרשות:

- קבלת קריטריונים (מיקום, תאריך, שעה, כמות סועדים וכו')
- בניית שאילתת SQL
- שליחת התשובה ללקוח

אובייקטים נחוצים: תקשורת, בסיס נתונים

שם היכולת: שליפת פרטי מסעדה

מהות: שליחת פרטים מורחבים על מסעדה לפי שם.
אוסף יכולות נדרשות:

- קבלת שם מסעדה
- שליפת פרטים מהמסד
- שליחת נתונים ללקוח

אובייקטים נחוצים: תקשורת, בסיס נתונים

שם היכולת: ביצוע הזמנה

מהות: שמירת הזמנה חדשה במערכת כולל פרטי המשתמש, מסעדה, זמן וקטגוריית ישיבה.

אוסף יכולות נדרשות:

- קבלת פרטי הזמנה
- בדיקת זמינות שולחן
- שמירת נתונים במסד
- החזרת אישור ללקוח

אובייקטים נחוצים: תקשורת, בסיס נתונים

שם היכולת: ביטול הזמנה

מהות: מחיקת הזמנה קיימת ממסד הנתונים לפי מזהה.

אוסף יכולות נדרשות:

- קבלת מזהה הזמנה
- מחיקת ההזמנה
- החזרת תגובה

אובייקטים נחוצים: תקשורת, בסיס נתונים

שם היכולת: שליחת מיילים למשתמש

מהות: שליחת הודעת דוא"ל מאובטחת למשתמש – לצורכי אימות, (2FA) אישור הזמנה או הודעת ביטול.

אוסף יכולות נדרשות:

- יצירת גוף הודעה בהתאם לאירוע (אימות, אישור, ביטול)
- שימוש בפרוטוקול SMTP עם התחברות דרך TLS (starttls)
- שימוש בחשבון Gmail ייעודי עם App Password
- שליחה בתהליך רקע (threading) למניעת האטת שרת
- טיפול בשגיאות שליחה במקרה של כשל

אובייקטים נחוצים: מודול, smtp lib, מחלקת MailSender, תקשורת חיצונית, Thread, מבנה טקסטואלי של ההודעה

יכולות ניהול וניתוח עבור האדמין:

שם היכולת: הצגת הזמנות לפי תאריך (מנהל מערכת)

מהות: הצגת כלל ההזמנות שבוצעו בתאריך מסוים, לצורכי בקרה וניתוח.

אוסף יכולות נדרשות:

- קבלת תאריך יעד מהאדמין
- שליפת נתוני ההזמנות מתוך מסד הנתונים בהתאם לתאריך
- החזרת הנתונים המעובדים לאדמין
- אובייקטים נחוצים:** תקשורת, בסיס נתונים

שם היכולת: הצגת הזמנות לפי משתמש

מהות: הצגת היסטוריית ההזמנות של משתמש מסוים, לצורך מעקב.

אוסף יכולות נדרשות:

- קבלת שם משתמש
- שליפת כלל ההזמנות הקשורות למשתמש המבוקש
- החזרת רשימת ההזמנות לאדמין
- אובייקטים נחוצים:** תקשורת, בסיס נתונים

שם היכולת: הצגת מסעדות פופולריות (מנהל מערכת)

מהות: זיהוי המסעדות שקיבלו את מספר ההזמנות הגבוה ביותר לצורך ניתוח מגמות.

אוסף יכולות נדרשות:

- ביצוע שאילתה עם קיבוץ ומיון לפי מסעדה
- ניתוח כמות ההזמנות לכל מסעדה
- החזרת חמש המסעדות המובילות לאדמין
- אובייקטים נחוצים:** תקשורת, בסיס נתונים

שם היכולת: הצגת משתמשים פעילים ביותר (מנהל מערכת)

מהות: זיהוי המשתמשים שביצעו את מספר ההזמנות הגבוה ביותר במערכת.

אוסף יכולות נדרשות:

- הרצת שאילתה סטטיסטית על טבלת ההזמנות
- קיצוץ ומיון לפי מזהה משתמש
- החזרת חמשת המשתמשים הפעילים ביותר
- אובייקטים נחוצים: תקשורת, בסיס נתונים

ניהול תוכן המסעדות:

שם היכולת: הוספת מסעדה חדשה (מנהל מערכת)

מהות: הוספת מסעדה חדשה למאגר הנתונים של המערכת.

אוסף יכולות נדרשות:

- קבלת נתוני המסעדה מהאדמין
- הוספת רשומת מסעדה חדשה במסד הנתונים
- החזרת אישור ביצוע הפעולה
- אובייקטים נחוצים: תקשורת, בסיס נתונים

שם היכולת: מחיקת מסעדה (מנהל מערכת)

מהות: הסרת מסעדה קיימת ממאגר הנתונים, כולל שולחנות המשויכים אליה.

אוסף יכולות נדרשות:

- קבלת מזהה או שם המסעדה
- מחיקת המסעדה ממסד הנתונים
- מחיקת כלל השולחנות המשויכים אליה
- שליחת אישור ביצוע לאדמין
- אובייקטים נחוצים: תקשורת, בסיס נתונים

שם היכולת: הוספת שולחנות למסעדה (מנהל מערכת)

מהות: הוספת שולחנות חדשים למסעדה קיימת בהתאם לקטגוריה וגודל.

אוסף יכולות נדרשות:

- קבלת פרטי המסעדה והגדרת השולחנות החדשים
- הוספת השולחנות לטבלה המתאימה במסד הנתונים
- החזרת תגובה המאשרת את הפעולה
- אובייקטים נחוצים: תקשורת, בסיס נתונים

שם היכולת: הסרת שולחנות ממסעדה (מנהל מערכת)

מהות: הסרת שולחנות מסוימים ממסעדה קיימת במערכת.

אוסף יכולות נדרשות:

- קבלת מזהה השולחן או מזהי הסרה אחרים
- מחיקת הרשומה המתאימה בטבלת השולחנות
- שליחת תגובה ללקוח המאשרת את המחיקה
- אובייקטים נחוצים: תקשורת, בסיס נתונים

מבנה / ארכיטקטורה של הפרויקט פירוט ההחלטות שנלקחו למימוש המערכת (עיצוב):

תיאור הארכיטקטורה של המערכת-

הפרויקט מבוסס על מודל Client-Server הפועל באמצעות תקשורת IP שממומשת בעזרת ספריית socket של פייתון. המערכת נבנתה כך שתוכל לתמוך בריבוי משתמשים בו זמנית, כאשר כל לקוח מקבל שירות דרך שרת מרכזי אחד.

לצורך הפעלה תקינה, המערכת דורשת לפחות שני מחשבים :

- **מחשב שרת** – מריץ את קוד השרת, (server.py) מאזין לבקשות חדשות ומטפל בהן.
- **מחשב לקוח** – מריץ את הממשק הגרפי (gui.py) ומבצע פעולות כמו רישום, התחברות, חיפוש מסעדות והזמנות.

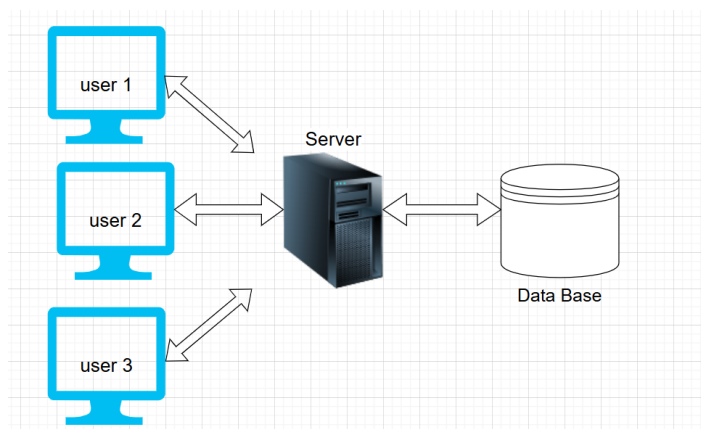
המערכת מבוססת על רשת מקומית, עם לקוח שמריץ אפליקציית GUI על מחשב אישי, (Windows) ושרת שמאזין על פורט קבוע. שני הרכיבים פועלים על אותה מכונה בתצוגה מדומה (localhost), אך יכולים לפעול על שני מחשבים שונים באותה רשת.

תיאור החומרה והרכיבים:

- **שרת מרכזי (Server)** מכיל את לוגיקת השרת ואת מסד הנתונים המקומי (SQLite). נדרש שהשרת יריץ את קוד השרת באופן רציף, ויאפשר גישה לקבצי הנתונים.
- **מסדי נתונים** : קבצי db. הכוללים את כל המידע הנדרש – מסעדות, משתמשים, הזמנות, ביקורות, שולחנות ועוד. מסדי הנתונים מאוחסנים על החומרה של השרת – לדוגמה : דיסק קשיח (HDD) או התקן נייד (Disk-On-Key).
- **מחשבי לקוח (Client)** מריצים את ממשק המשתמש הגרפי, ומבצעים פעולות דרך שליחת בקשות JSON לשרת.
- **שרת מייל חיצוני (SMTP)** משמש לשליחת הודעות אימייל (כגון קוד אימות או אישור הזמנה), דרך פרוטוקול מאובטח TLS.
- **תהליכי רקע** : בשרת מוגדרים Thread ים ייעודיים לשליחת מיילים, כדי למנוע חסימה של פעולות קריטיות נוספות בזמן שליחה

הקשרים בין הרכיבים :

- **תקשורת לקוח-שרת** : דו-כיוונית, מתבצעת באמצעות socket – כל לקוח שולח בקשה בפורמט JSON ומקבל תגובה.
- **תקשורת שרת** : DB – גישה ישירה לקבצי הנתונים – הקריאה והכתיבה מתבצעת דרך מודול sqlite3.
- **שרת** : SMTP – תקשורת חיצונית עם שרת הדוא"ל, מתבצעת דרך (TLS) smtpplib + starttls כדי להצפין את ההודעה ולמנוע יירוט.



שרטוט המציג קשרים אלה :

תיאור הטכנולוגיות הרלוונטיות:

● שפת תכנות Python –

שפת Python נבחרה בזכות הפשטות, הקריאות והנוחות שהיא מציעה למפתחים. היא תומכת בפרדיגמות תכנות שונות (פרוצדורלי, מונחה עצמים ועוד), ומאפשרת מימוש מהיר של לוגיקה מורכבת באמצעות קוד קצר יחסית. במהלך הפיתוח נעשה שימוש במבני נתונים מתקדמים, בניהול קבצים, תקשורת רשת, הצפנה, ממשק משתמש גרפי ועוד – והכול באמצעות ספריות Python סטנדרטיות לצד ספריות צד שלישי.

● מערכת הפעלה Windows –

המערכת נבנתה ופותחה בסביבת Windows 10/11 תוך שימוש ב־Python ובסביבות עבודה כמו PyCharm. עם זאת, היות וכל הספריות בהן נעשה שימוש הן חוצות-פלטפורמה – ניתן להריץ את המערכת גם על Linux או MacOS בכפוף להתקנת הספריות הנדרשות.

● תקשורת Socket – TCP

המערכת מבוססת על ארכיטקטורת Client-Server שבה המשתמש (הלקוח) מפעיל ממשק גרפי ששולח בקשות לשרת המרכזי. התקשורת נעשית על גבי פרוטוקול TCP (אמין ומובנה) בעזרת ספריית socket. העברת המידע מתבצעת בפורמט – JSON המאפשר שליחה של אובייקטים מורכבים בצורה טקסטואלית נוחה.

פרוטוקולי תקשורת ואבטחה:

במערכת פועלים מספר פרוטוקולים ותהליכי הצפנה, אשר תומכים ביכולת שלה **להעביר מידע באופן בטוח, אמין ומובנה** בין רכיביה השונים:

- **TCP/IP**: לתקשורת יציבה ואמינה בין לקוח לשרת.
- **JSON**: לשליחת בקשות ותגובות בפורמט טקסטואלי אחיד וברור.
- **SMTP עם TLS (starttls)**: לשליחת מיילים מוצפנים דרך Gmail.
- **RSA**: להצפנת סיסמאות בצד הלקוח לפני שליחתן – הצפנה אסימטרית.
- **bcrypt**: להצפנה חד־כיוונית של סיסמאות בשרת (אידיאלית לאחסון).
- **2FA**: קוד אימות חד־פעמי (OTP) שנשלח למייל המשתמש כהגנה נוספת.

תחומי עניין מרכזיים בפרויקט:

ממשק משתמש (GUI) מתקדם

הממשק הגרפי נבנה באמצעות הספרייה customtkinter, המאפשרת עיצוב מודרני ותגובה מהירה לפעולות המשתמש. ממשקי ההתחברות, ההרשמה, חיפוש מסעדות וההזמנות – כולם מעוצבים בצורה נוחה, אחידה ונגישה.

אבטחת מידע

המערכת משלבת שכבות הגנה מתקדמות:

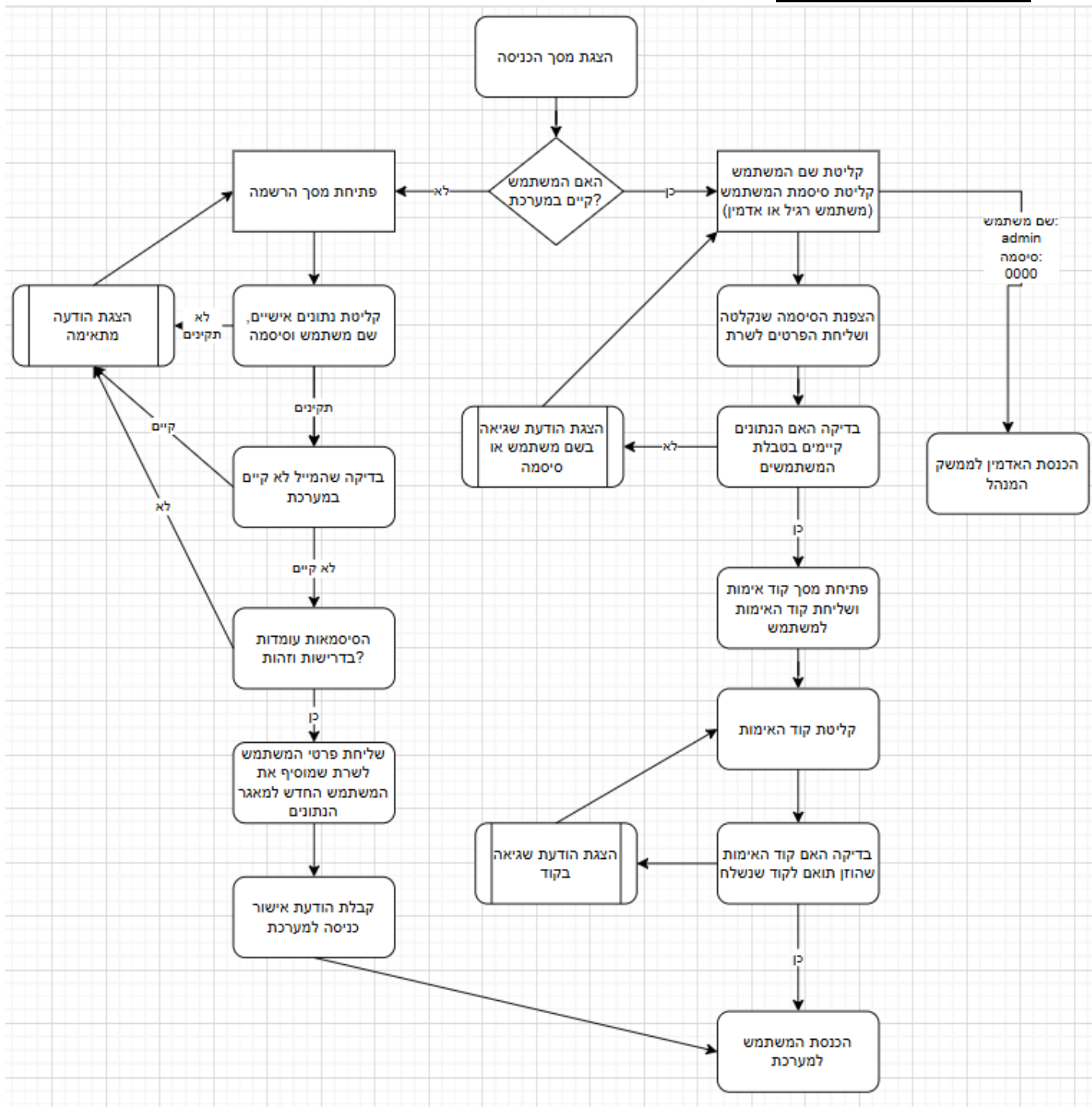
- הצפנת סיסמה ב־RSA לפני שליחה.
- שמירת סיסמאות מוצפנות ב־bcrypt במסד הנתונים.
- אימות דו־שלבי (2FA): שליחת קוד אימות חד־פעמי למייל.
- טיפול בנסיונות התחברות שגויים ומניעת גישת IP זדונית.

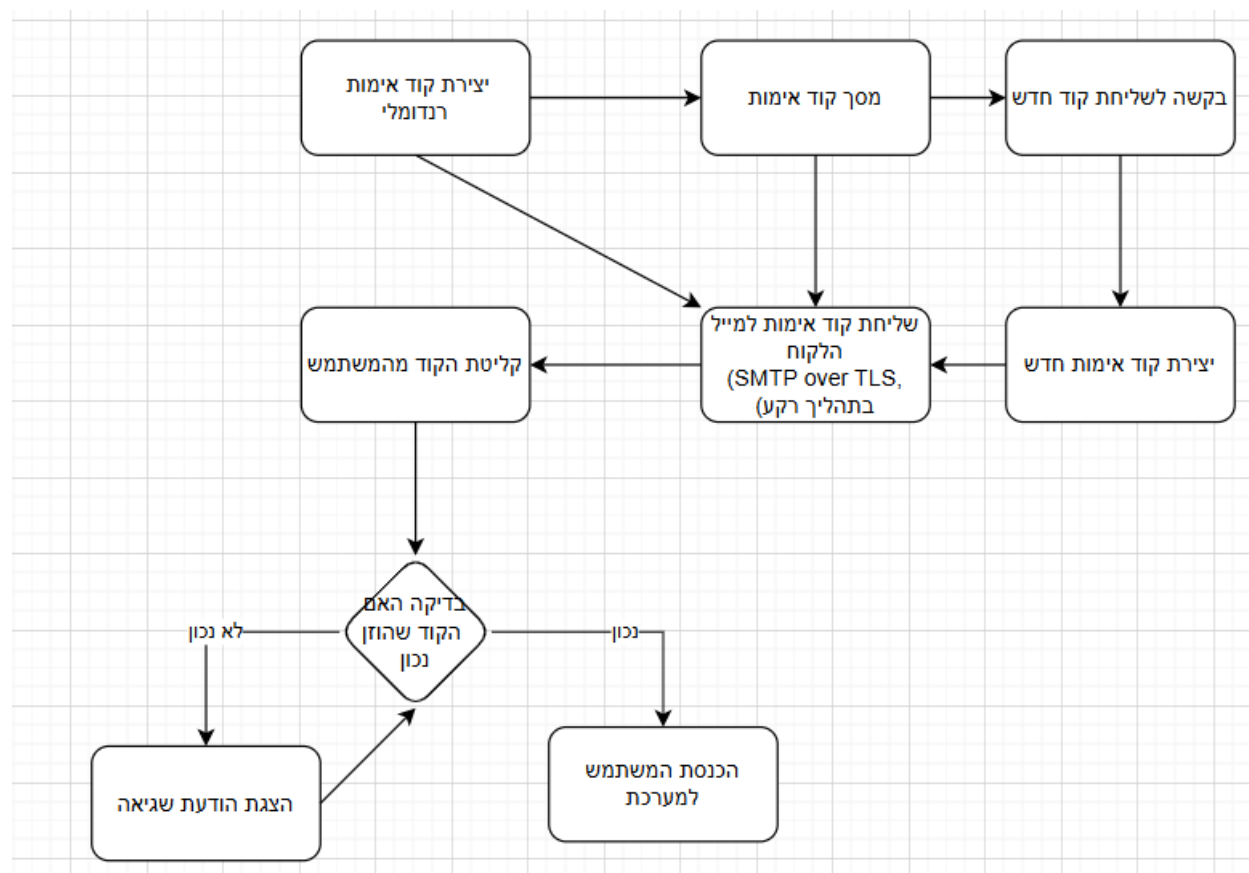
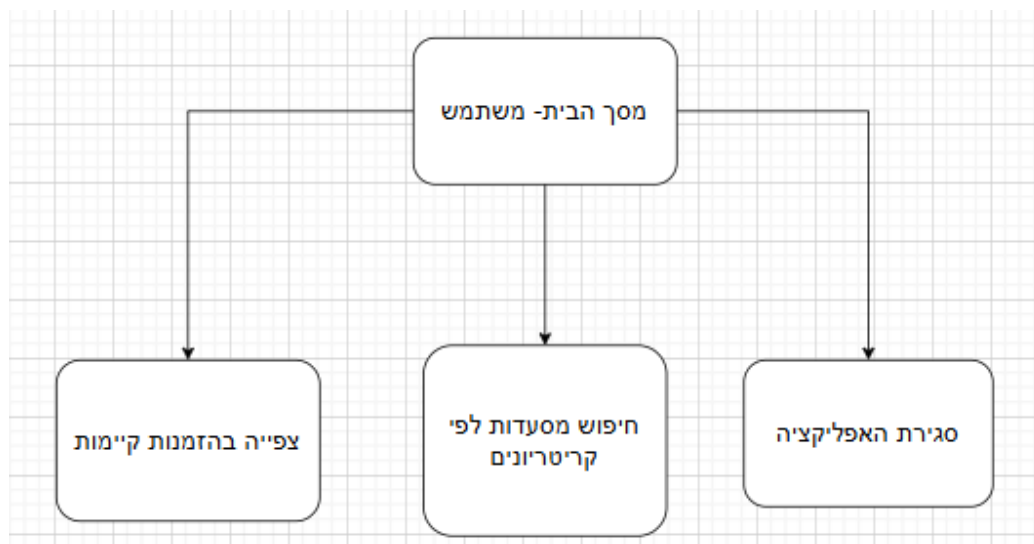
מסד נתונים דמוי-מציאותי

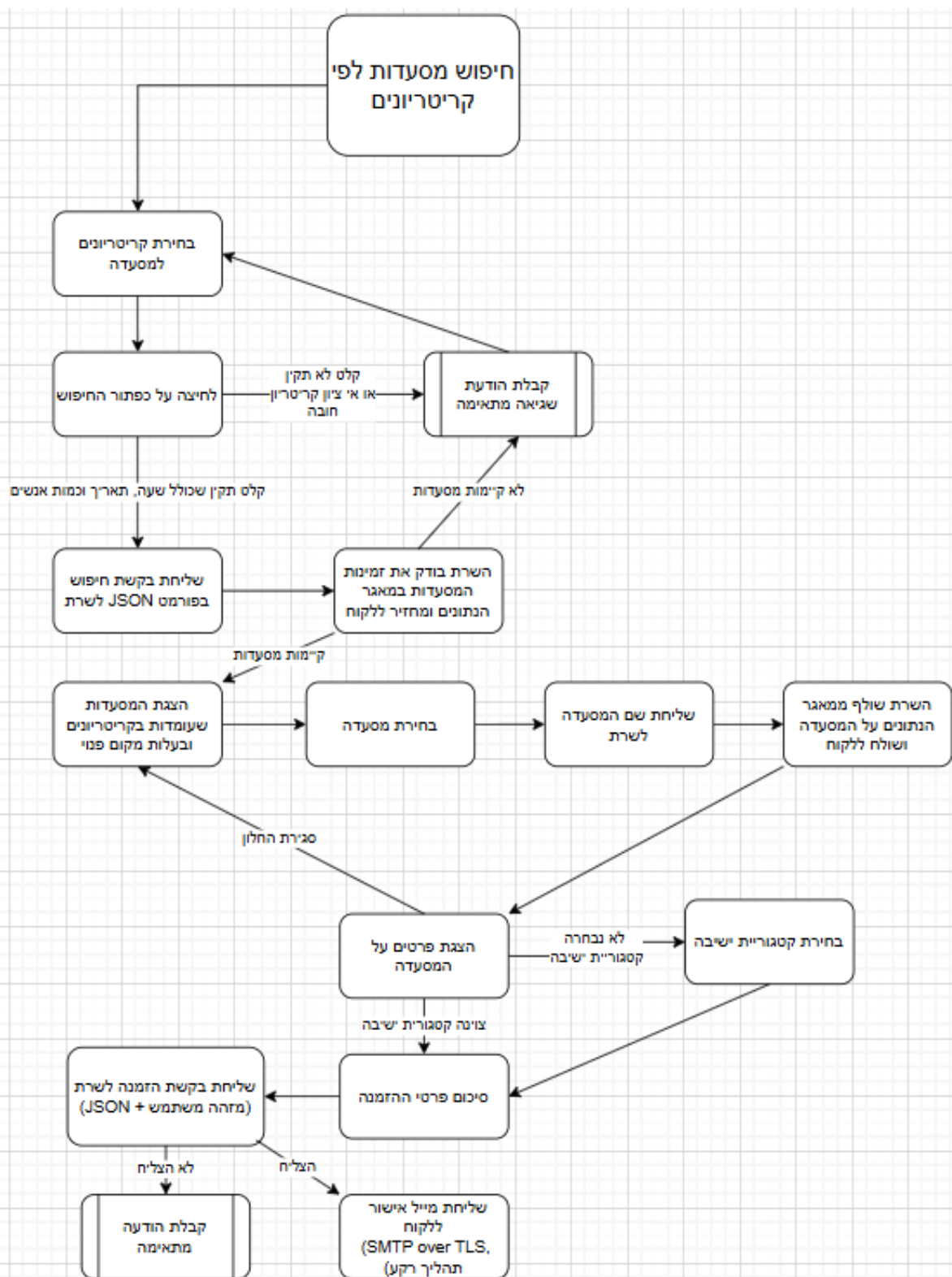
המערכת משתמשת ב־ SQLite מסד נתונים קל ויעיל המאוחסן בקובץ מקומי. במסד נשמרים כל הנתונים: פרטי משתמשים, טבלאות מסעדות, שולחנות, הזמנות, היסטוריית הזמנות, ביקורות ועוד. המבנה כולל קשרים לוגיים בין הטבלאות, מפתחות ראשיים וזרים, ומאפשר שליפות מדויקות ומהירות.

תיאור זרימת המידע במערכת-

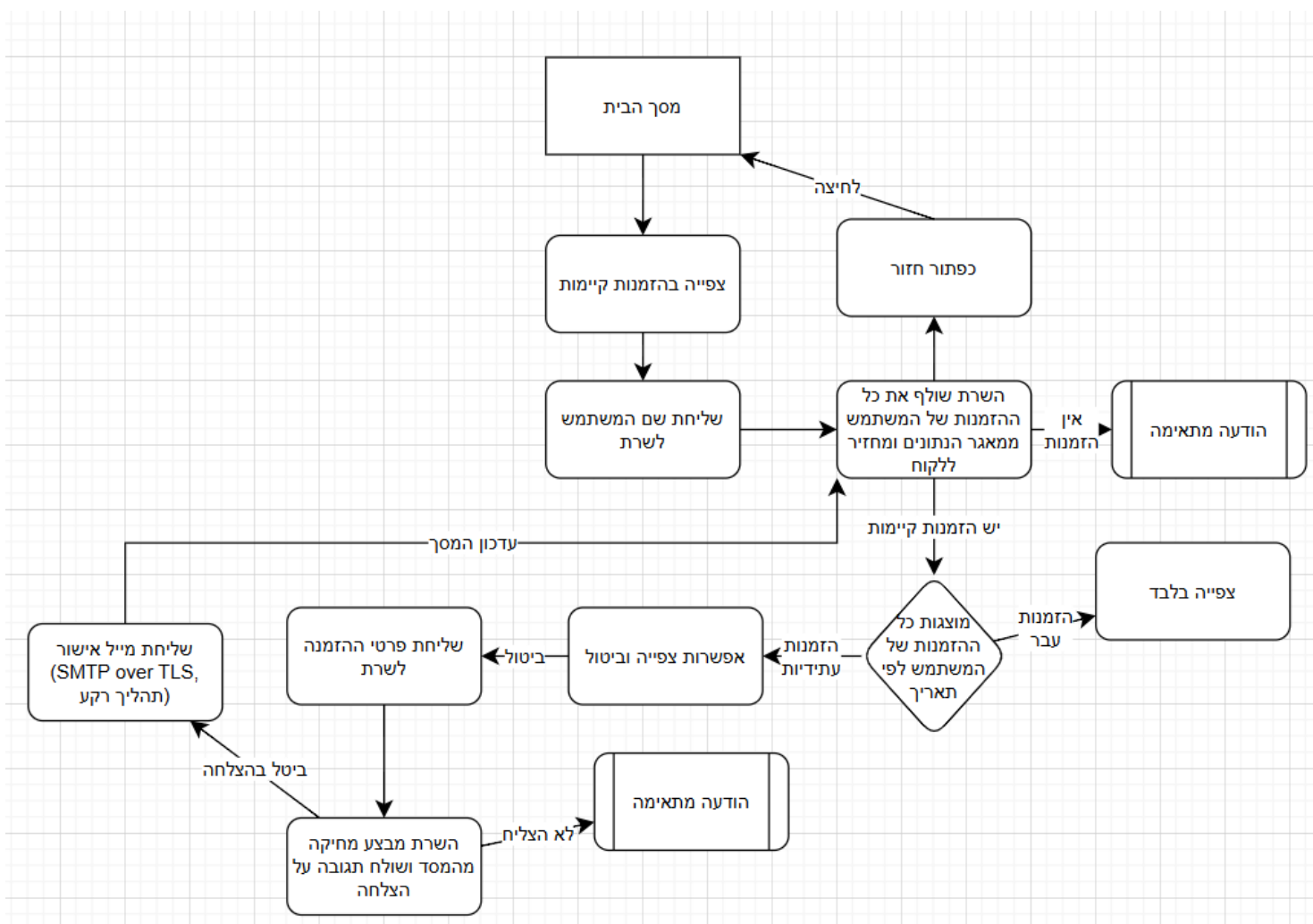
הרשמה והתחברות למערכת:



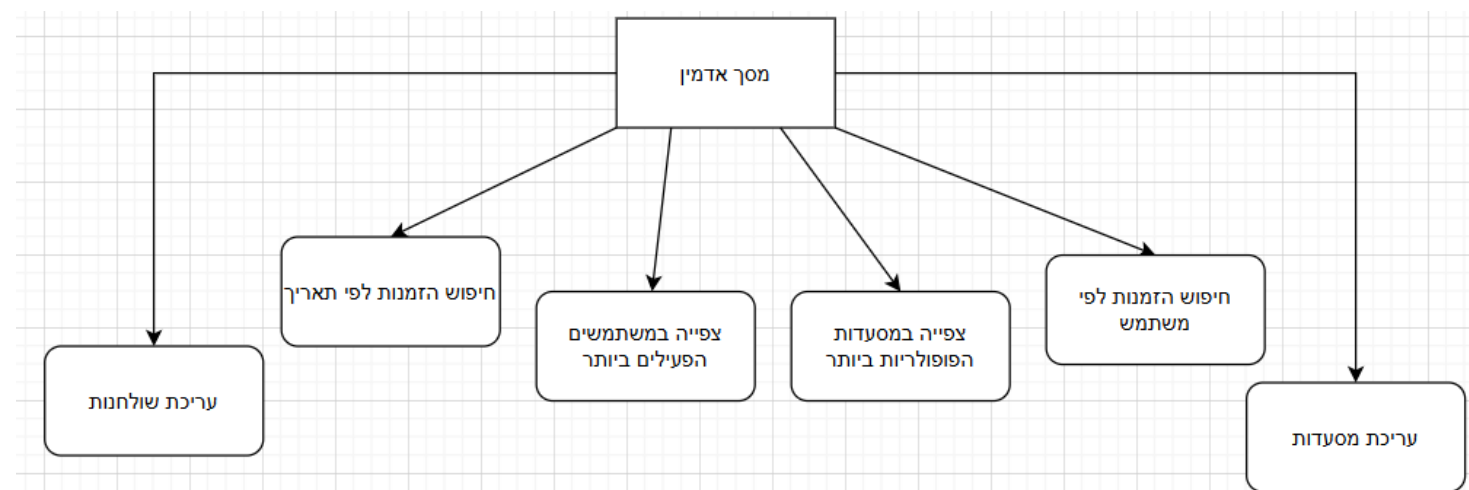
תהליך שליחת קוד אימות:תהליך מסד הבית של המשתמש- נפתח לאחר שהתחבר לאתר

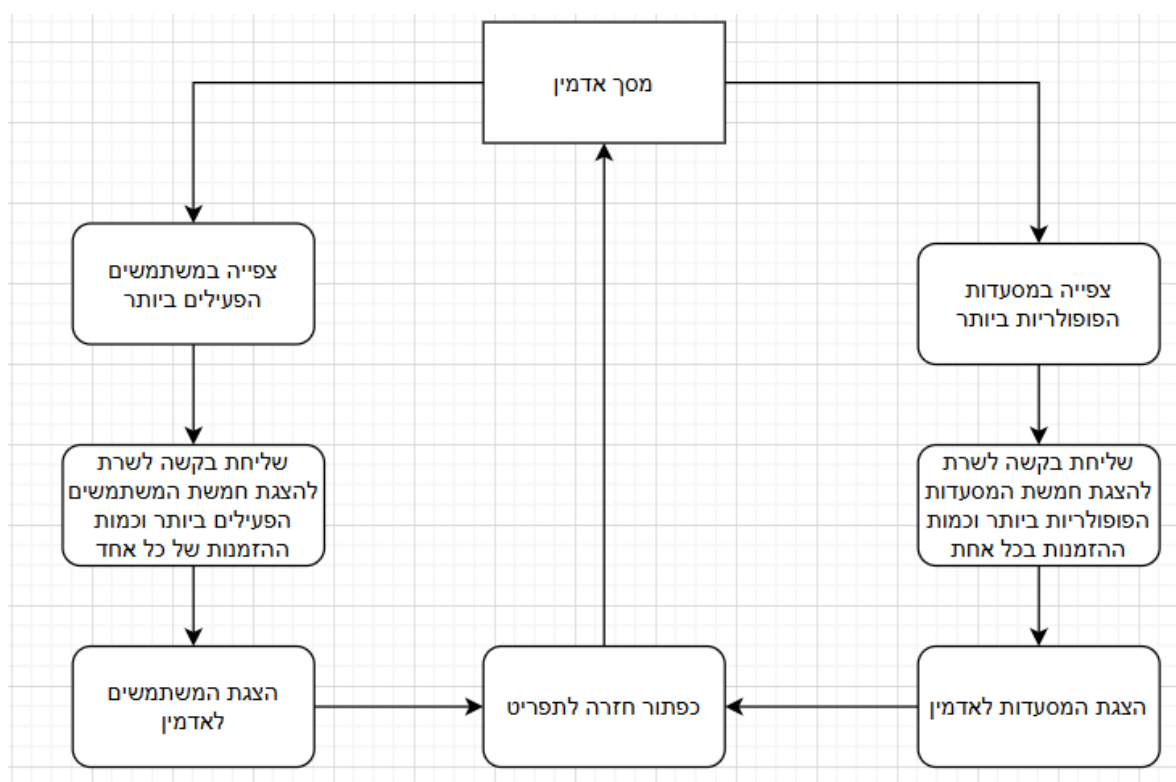
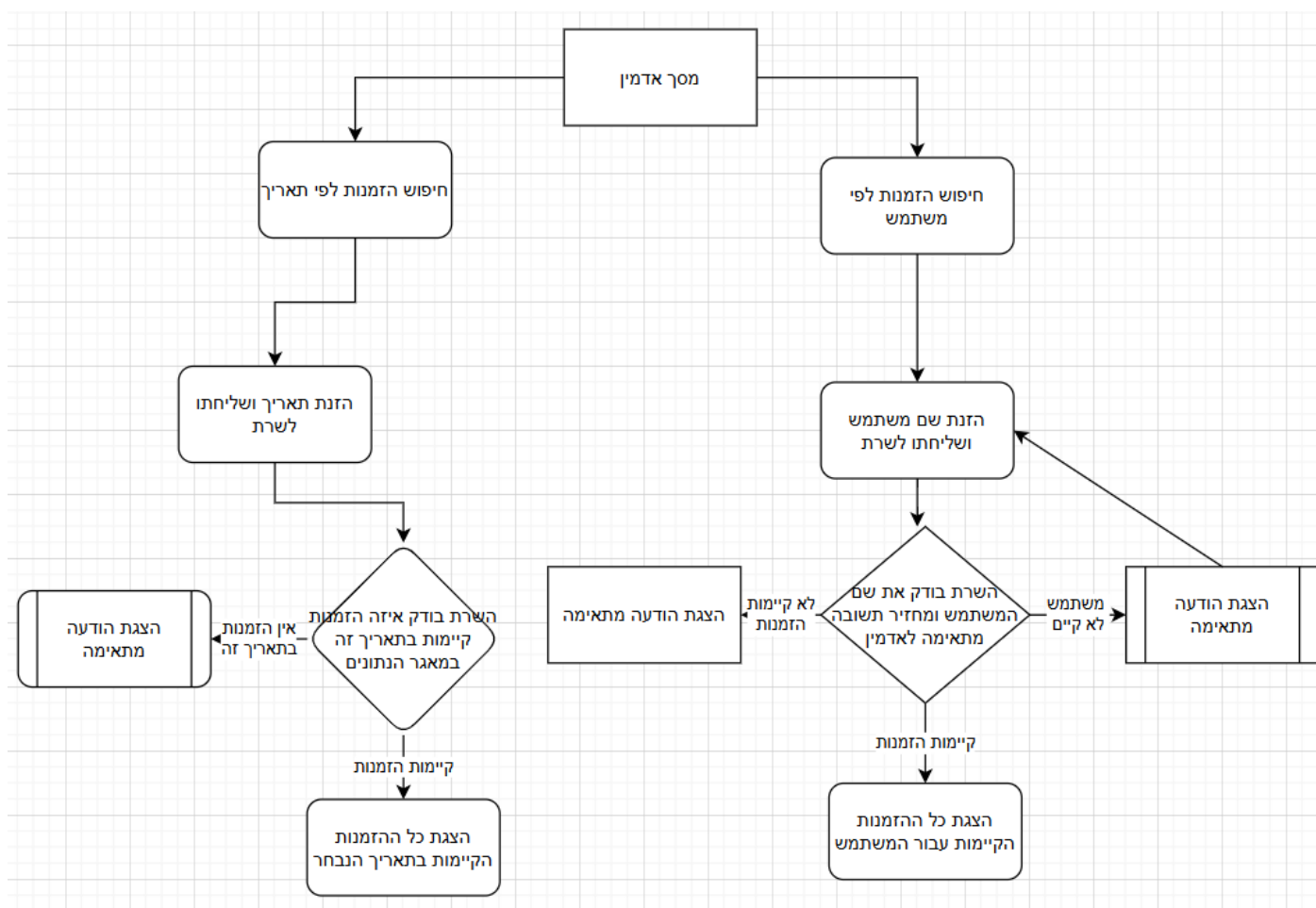
תהליך הזמנת מקום במסעדה:

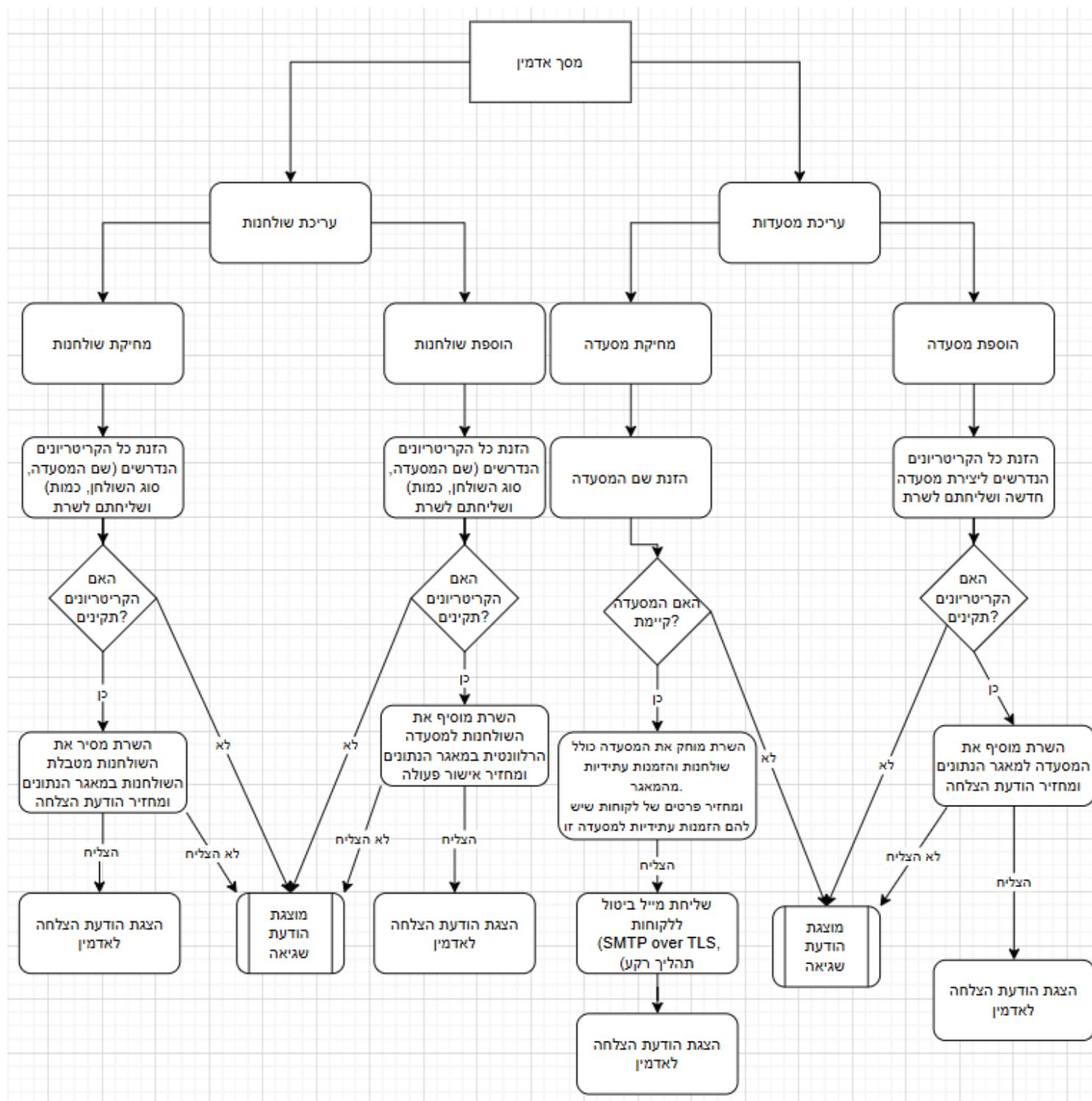
צפייה בהזמנות קיימות:

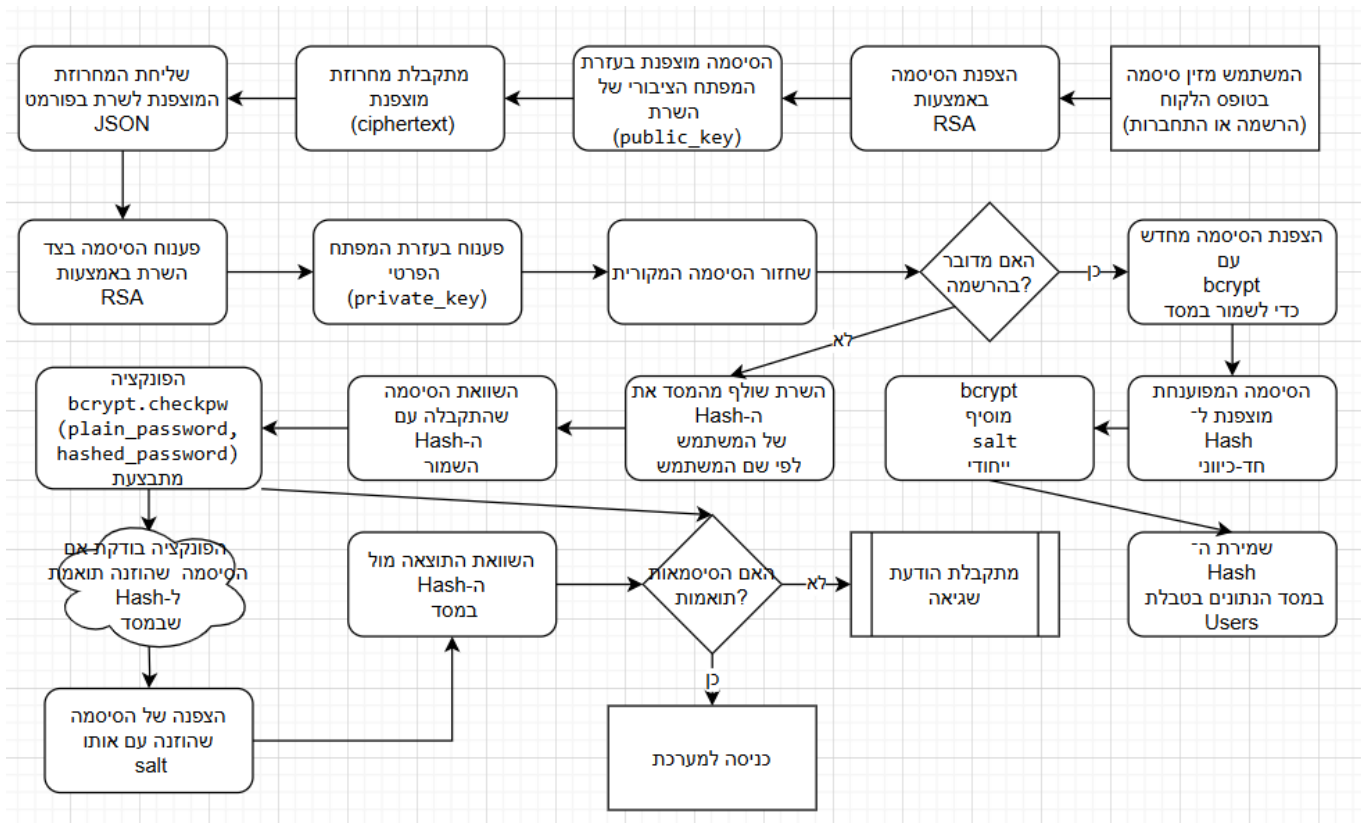
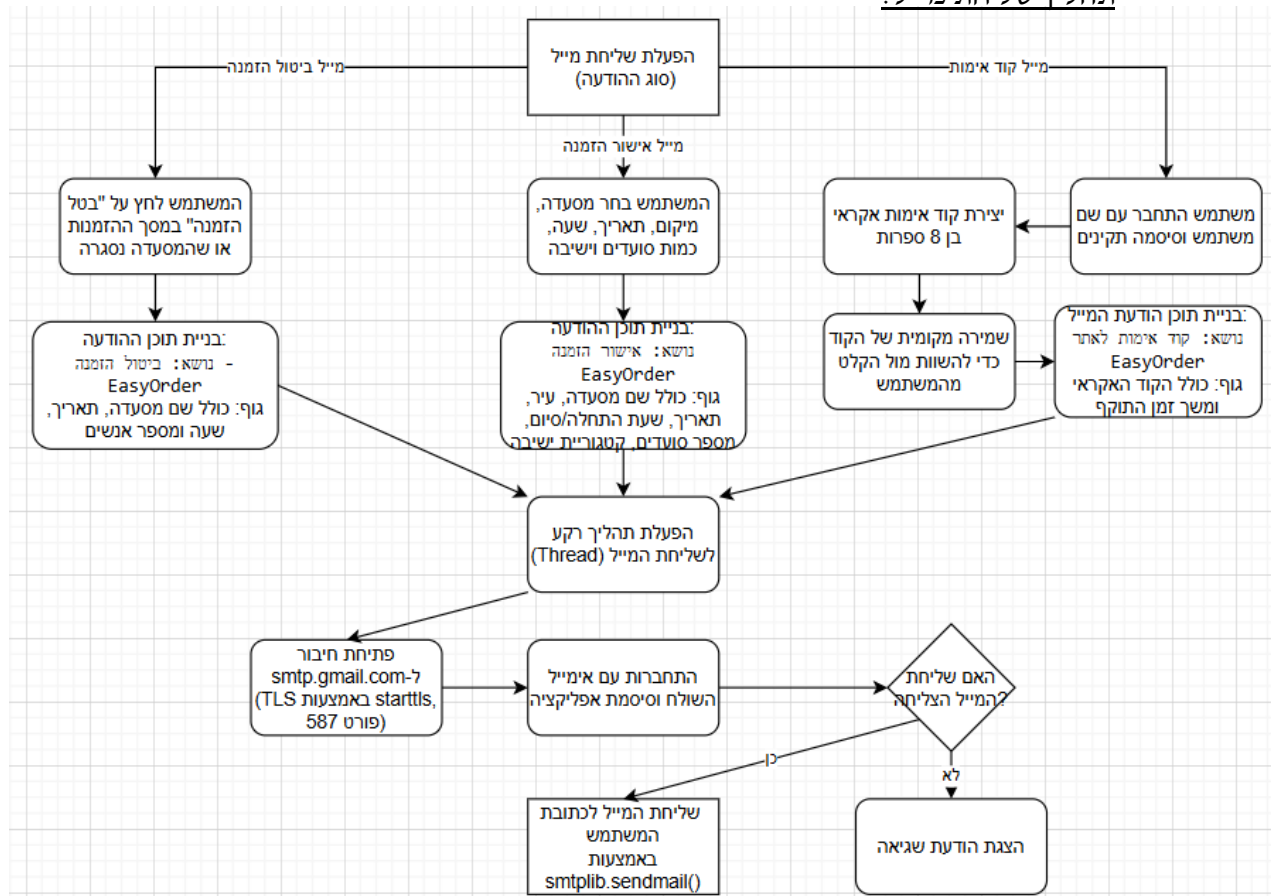


מסך בית אדמין :



פעולות אדמין:



תהליך הצפנת הסיסמה:תהליך שליחת מייל:

תיאור האלגוריתמים מרכזיים בפרויקט:

שם האלגוריתם: הצפנה ואבטחת סיסמאות משתמש

ניסוח וניתוח הבעיה האלגוריתמית:

בעת תהליך ההרשמה למערכת, יש לשמור את סיסמת המשתמש בצורה מאובטחת כך שגם אם מתבצעת פריצה למסד הנתונים, לא ניתן יהיה לשחזר את הסיסמה המקורית. בנוסף, יש להבטיח שהסיסמה תועבר מהלקוח לשרת בצורה בטוחה שאינה ניתנת ליירוט.

אלגוריתמים קיימים לפתרון הבעיה:

- שימוש ב־ RSA להצפנה אסימטרית של הסיסמה בזמן שליחתה
- שימוש ב־ bcrypt להצפנה חד־כיוונית (hash) של הסיסמה לפני שמירה
- שמירה על עקרון Zero-Knowledge: הסיסמה המקורית לעולם לא נשמרת או נשלחת בפורמט קריא

הפנייה למקורות רלוונטיים:

• [https://en.wikipedia.org/wiki/RSA_\(cryptosystem\)](https://en.wikipedia.org/wiki/RSA_(cryptosystem))

• <https://en.wikipedia.org/wiki/Bcrypt>

סקירת הפתרון הנבחר:

במערכת מיושמת **הצפנה כפולה**:

- הסיסמה מוצפנת באמצעות מפתח ציבורי (RSA) בצד הלקוח
- מפוענחת בצד השרת באמצעות מפתח פרטי
- מיד לאחר מכן, הסיסמה מוצפנת באמצעות bcrypt ונשמרת במסד הנתונים שיטת bcrypt נבחרה בגלל עמידותה בפני מתקפות brute-force ושימוש מובנה ב־ salt.

RSA הוא אלגוריתם להצפנה מבוססת מפתחות – כלומר, קיימים **שני מפתחות נפרדים**:

- **מפתח ציבורי (Public Key)** – להצפנה.
- **מפתח פרטי (Private Key)** – לפענוח.

הרעיון המרכזי:

- כל אחד יכול להצפין בעזרת המפתח הציבורי.
- רק בעל המפתח הפרטי יכול לפענח את מה שהוצפן.

שלבי יצירת המפתחות:

1. **בחירת שני מספרים ראשוניים: (p, q)**
שני מספרים ראשוניים נבחרים באקראי מתוך רשימה מוגדרת מראש. אלו יהיו אבני היסוד של המערכת.
2. **חישוב: n**
$$n = p \times q$$
זהו חלק מהמפתח הציבורי. הגודל של n קובע את רמת האבטחה (למשל, 2048 ביט זה מאוד חזק).
3. **חישוב פונקציית אוילר:**
$$\phi(n) = (p-1) \times (q-1)$$
פונקציה זו חשובה למציאת המפתח הפרטי.

4. בחירת מעריך ציבורי: e

בוחרים מספר קטן יחסי (כמו 65537), כזה שאין לו מחלקים משותפים עם $\phi(n)$. הוא חלק מהמפתח הציבורי.

5. חישוב מפתח פרטי: d

זהו המספר שיפענח את ההודעה. נבחר כך ש- $d \times e \equiv 1 \pmod{\phi(n)}$

כלומר d , הוא ה"הופכי המודולרי" של e ביחס ל- $\phi(n)$.

6. פרסום המפתח הציבורי:

זוג הערכים (e, n) נשלח ללקוח –

באמצעותו הוא מצפין את ההודעה (למשל סיסמה).

7. שמירת המפתח הפרטי:

השרת שומר את d בסוד.

בעזרתו הוא יכול לפענח את ההודעה המוצפנת.

8. הצפנה:

הלקוח מבצע $c = m^e \pmod{n}$

(כאשר: m = ההודעה המקורית, c = ההודעה המוצפנת)

9. פענוח:

השרת מבצע $m = c^d \pmod{n}$

וכך הוא מקבל את המידע המקורי – למשל את הסיסמה.

דוגמה מספרית פשוטה:

• נניח:

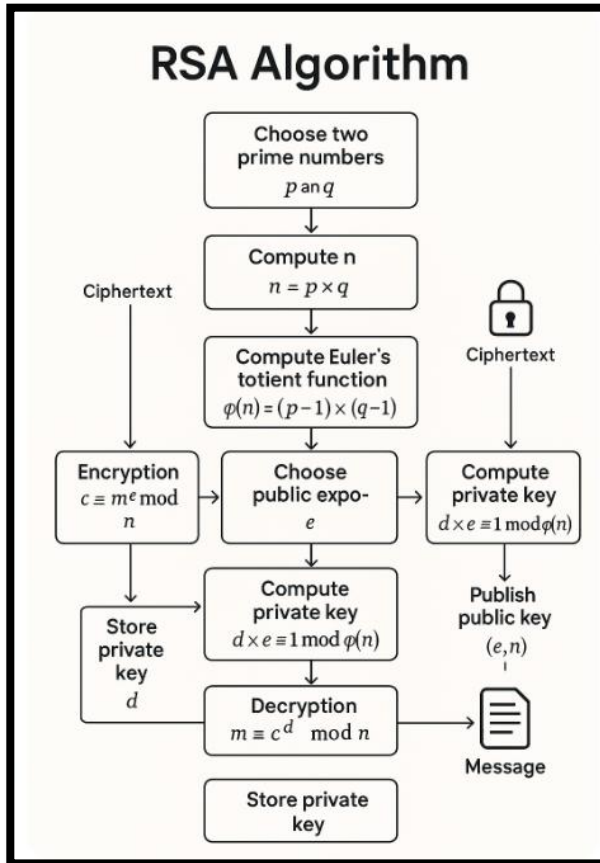
message = 42

$e = 17, n = 3233$

אז:

$\text{cipher} = (42^{17}) \pmod{3233} = 2557$

עכשיו רק עם d ניתן יהיה לפענח את 2557 ולחזור ל-42.



bcrypt הוא אלגוריתם להצפנה חד-כיוונית (Hashing) עם Salt

- בניגוד ל-RSA, bcrypt לא מאפשר פענוח – זו הצפנה חד-כיוונית בלבד.
- משמש לאחסון סיסמאות באופן מאובטח – גם אם יגנבו את מסד הנתונים, אי אפשר לשחזר את הסיסמה.

- מבוסס על אלגוריתם Blowfish שעבר התאמה ל-Hashing.

הרעיון המרכזי:

- כל סיסמה עוברת גיבוב יחד עם salt אקראי.
- התוצאה היא מחרוזת מוצפנת שמכילה גם את ה salt וגם את מספר הסבבים.
- אותה סיסמה עם salt שונה תיתן Hash שונה.
- אי אפשר לשחזר את הסיסמה המקורית מתוך ה Hash.
- בעת התחברות – משתמשים באותו salt ובודקים התאמה של Hash.

שלבי החישוב של bcrypt**1. קלט: סיסמה (Password) ו־Salt**

נניח:

- הסיסמה היא מחרוזת באורך משתנה:
 $p = "1234"$
- Salt הוא מחרוזת באורך 128 ביט (16 בתים):
 $s = "XYZ9g8aRGN2oKsaf"$

2. חישוב פלט bcrypt

bcrypt מבצע:

 $\text{bcrypt}(p, s, c) = \text{Blowfish-ECB-Expand}(s, p, 2c)$

כאשר:

 p – הסיסמה s – ה־salt c – cost factor (מספר הסבבים),לדוגמה: $c = 12$ פירושו 2 בחזקת 12 שווה 4096 סבבים.**3. שלב הפנמה (Key Expansion)**

- bcrypt מתחיל בפונקציית EksBlowfishSetup שמבצעת:

 $\text{State} \leftarrow \text{Initialize}(s, p)$ עם cost גבוה, כלומר ביצוע 2^c פעולות פנימיות:

- שימוש ב־salt ובסיסמה לחישוב מצב פנימי מורכב
- הכפלת זמן המחשב הדרוש לגיבוב

4. Hash סופי

- לאחר ההרחבה, bcrypt מבצע:

 $h = \text{EncryptState}(\text{constant})$

כאשר constant היא מחרוזת קבועה של 192 ביט למשל ("OrpheanBeholderScryDoubt")

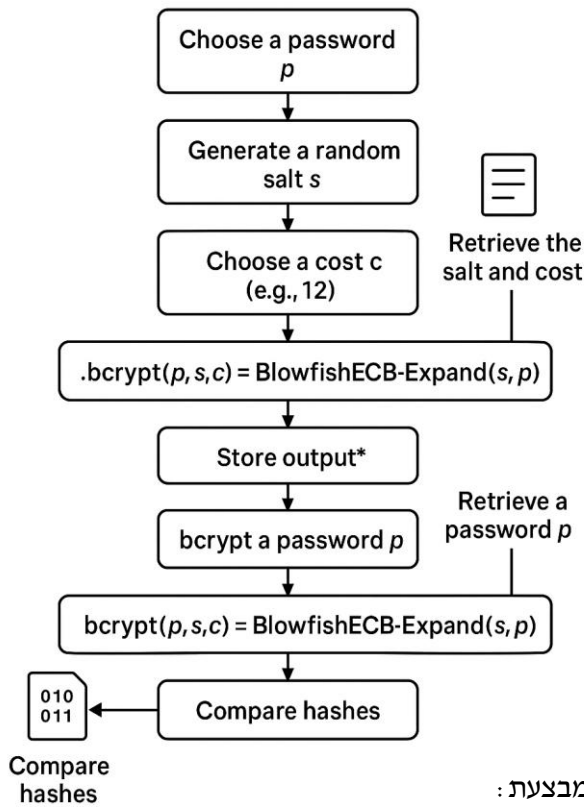
התוצאה היא ערך Hash בגודל 192 ביט (24 בתים)

5. הפלט הסופי כולל: $\text{Output} = \$2b\$c\$salt\$hash$

למשל:

 $b\$12\$XYZ9g8aRGN2oKsaf...xCmS9m1YyOxq\2

- גרסה = 2b
- cost factor = 12

bcrypt Hashing Algorithm

- `XYZ9g8aRGN2oKsaf = salt`
- `xCmS9m1YyOxq = hash`

דוגמה מספרית פשוטה:

- סיסמה "hello":
- Salt: "s0m3S@lt:2345678" (16 תווים).
- 1024 סבבים, כלומר 2 בחזקת 10, cost=10

המערכת תחשב:

`bcrypt("hello","s0m3S@lt:2345678",10)⇒$2b$10$s0m3S@lt...YzAxQk`

(ה' hash בפועל שונה בכל פעם כי הsalt משתנה)

שם האלגוריתם: שליחת קוד אימות במייל (אימות דו-שלבי)

ניסוח וניתוח הבעיה האלגוריתמית:

לאחר שהמשתמש הזין שם וסיסמה תקינים, יש לאמת שזהו אכן האדם שמנסה להיכנס – באמצעות קוד חד-פעמי שנשלח לכתובת הדוא"ל הרשומה. הקוד צריך להיות אקראי, חד-פעמי, לפוג אחרי זמן מה, ובלתי ניתן לניחוש פשוט.

אלגוריתמים קיימים לפתרון הבעיה:

- יצירת קוד רנדומלי (8 ספרות) באמצעות פונקציית random של פייתון
- שמירת הקוד בזיכרון זמני בלבד
- שימוש בטיימר (הפונקציה after ב Tkinter) לקביעת זמן תוקף
- שליחה באמצעות smtp lib ו SMTP

הפנייה למקורות רלוונטיים:

- <https://docs.python.org/3/library/random.html>
- <https://www.geeksforgeeks.org/python-after-method-in-tkinter/>

סקירת הפתרון הנבחר:

המערכת שולחת קוד אימות חד-פעמי בן 8 ספרות שנוצר בצורה אקראית, באמצעות מחלקת MailSender. לאחר יצירת הקוד, הוא נשלח לכתובת הדוא"ל של המשתמש דרך פרוטוקול SMTP. כדי למנוע האטה בממשק הגרפי, פעולת השליחה מתבצעת בתוך **Thread נפרד** (באמצעות המודול threading) כך שהמערכת נותרת זמינה לתפעול בזמן אמת.

עם שליחת הקוד, מוצג למשתמש **חלון ייעודי להזנת הקוד**, כולל טיימר של 120 שניות עד לפקיעת תוקפו. במידה והקוד לא הוזן בזמן – ההתחברות נכשלת ויש לאתחל את התהליך. כמו כן, ניתן לשלוח קוד חדש בלחיצה על כפתור "שלח שוב".

המייל נשלח דרך smtp.gmail.com בפורט 587, תוך שימוש בפרוטוקול TLS המאובטח (starttls). תוכן המייל מוצפן בשכבת ההעברה (Transport Layer), מה שמונע אפשרות של יירוט או שינוי המידע.

(MITM).

הקוד שנשלח אינו נשמר במסד הנתונים, אלא מאוחסן זמנית בזיכרון ומושווה מול הקלט של המשתמש בלבד.

שם האלגוריתם: חסימת גישה לאחר ניסיונות התחברות שגויים

ניסוח וניתוח הבעיה האלגוריתמית:

במערכות המחייבות אימות משתמש, ישנה סכנה ממשית לכך שגורמים זדוניים ינסו לפרוץ לחשבונות באמצעות מתקפות מסוג brute-force ניסוי והטעיה חוזרים של סיסמאות עד להצלחה. מתקפות מסוג זה לא רק מסכנות את פרטיות המשתמשים, אלא גם עלולות להעמיס על השרת וליצור מעין מתקפת מניעת שירות (DDoS). לכן, יש צורך במנגנון שיזהה ניסיונות התחברות חוזרים ונשנים עם סיסמאות שגויות ויחסום זמנית את מקור הבקשות (לרוב כתובת IP) כדי לשבור את הרצף ולמנוע את המשך ההתקפה.

אלגוריתמים קיימים לפתרון הבעיה:

1. **גישה מבוססת מונה ניסיונות לפי IP (Rate Limiting):** שימוש במבנה נתונים (למשל dictionary), לשמירת מונה ניסיונות לכל כתובת IP. לאחר חריגה ממספר ניסיונות (למשל, 5), נחסמת הגישה לפרק זמן מסוים.
2. **גישה מבוססת CAPTCHA לאחר מספר ניסיונות:** הצגת אתגר (למשל, זיהוי תמונה או קוד) לאחר כמות ניסיונות מסוימת כדי לאפשר רק לבני אדם לגשת הלאה.
3. **גישה מבוססת זמן תגובה משתנה (Tarpit):** הארכת זמן התגובה בהדרגה לאחר כל ניסיון שגוי כדי להאט את קצב הניסיונות באופן מלאכותי.
4. **גישה מבוססת חסימת משתמש/חשבון (לא IP):** חסימת המשתמש עצמו (ולא ה-IP) אך פתרון זה רגיש במיוחד במקרה של ניסיונות פריצה לחשבונות קיימים, ועשוי להוות פתח למתקפות השתקה (lockout attacks).
5. **Rate Limiting מבוסס Token Bucket או Leaky Bucket:** גישות מתקדמות מהעולם של תעבורת רשת, שמבקרות את קצב הבקשות לאורך זמן.

הפנייה למקורות רלוונטיים:

- <https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>
- https://en.wikipedia.org/wiki/Brute-force_attack
- <https://www.geeksforgeeks.org/python-dictionary/>
- https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

סקירת הפתרון הנבחר:

בפרויקט זה, בחרתי בגישה פשוטה אך אפקטיבית:

- השרת עוקב אחרי ניסיונות ההתחברות של כל כתובת IP באמצעות מילון (dictionary) בזיכרון.
- עבור כל ניסיון התחברות כושל, המונה עבור ה-IP הספציפי גדל.

- לאחר 5 ניסיונות שגויים, ה-IP נחסם לפרק זמן של 5 דקות. חסימה זו מתבצעת בעזרת שמירת Timestamp של הזמן שבו התחילה החסימה.
- כאשר מגיעה בקשה מכתובת חסומה, השרת בודק אם הזמן עבר, ואם כן – מאפשר מחדש את הגישה.
- במידה והמשתמש מתחבר בהצלחה לפני הגעת הסף – מונה הניסיונות מתאפס.

מדוע פתרון זה נבחר:

- הוא פשוט ליישום ומונע עומס מיותר על מסד הנתונים (המעקב נעשה בזיכרון בלבד).
- החסימה זמנית, ולכן אינה פוגעת בחוויית המשתמש בטווח הארוך.
- מאפשר אבחנה בין ניסיונות התחברות לגיטימיים לכאלו שמקורם בהתקפה, תוך תגובה מהירה.
- תומך היטב במערכות קטנות ובינוניות שאינן פועלות תחת עומס תעבורתי גדול במיוחד.
- הגישה אינה משפיעה על משתמשים לגיטימיים מכיוון שהחסימה זמנית בלבד.

שם האלגוריתם: שליחת מיילים מהירה ולא חוסמת

ניסוח וניתוח הבעיה האלגוריתמית:

בעת שליחת קוד אימות או אישור הזמנה, ייתכן שהפעולה תיקח זמן רב ותאט את המערכת אם לא תתבצע בצורה אסינכרונית.

אלגוריתמים קיימים לפתרון הבעיה:

- שימוש ב- threading לשליחת מיילים ברקע
- טיפול בשגיאות באמצעות try-except
- איפסור שליחה חוזרת בלי קריסת הממשק

סקירת הפתרון הנבחר:

בעת שליחה – נפתח thread נפרד לביצוע השליחה בפועל.
(במידה ויש כשלון בעיה ברשת/שרת SMTP) המערכת תתריע על כך אך לא תקרוס.
השימוש ב- thread מאפשר גם המשך תפקוד מלא של הממשק במקביל.

שם האלגוריתם: העברת מידע בטוחה בין השרת ללקוח

ניסוח וניתוח הבעיה האלגוריתמית:

בעת תקשורת בין לקוח לשרת, קיים סיכון שמידע רגיש (כגון סיסמאות או פרטי הזמנה) ייחשף לגורמים זדוניים או יעבור שיבוש עקב תקלות רשת. על מנת להבטיח שהמערכת תישאר מאובטחת ואמינה, יש לטפל במספר אתגרים עיקריים:

1. **שמירה על שלמות המידע** – מניעת קטיעה או עיוות של הודעות במהלך המעבר.
2. **אבטחת המידע** – הבטחת סודיות המידע באמצעות הצפנה.
3. **גמישות מבנית** – יכולת תמיכה בהודעות מכל הסוגים, כולל מבנים מורכבים או נפחים גדולים.

אלגוריתמים קיימים לפתרון הבעיה:

- שימוש בפרוטוקולי תקשורת (כמו SSL/TLS במערכות Web)
- הצפנת המידע באמצעות RSA או AES.
- שימוש בתבניות JSON או Pickle להמרת אובייקטים ל-bytes.
- שליחת מידע במקטעים (chunks) עם ציון גודל ההודעה מראש.
- שימוש בתו מסיים להודעה (delimiter) כדי לזהות את סיום ההודעה.

הפנייה למקורות רלוונטיים:

- <https://docs.python.org/3/library/socket.html>
- <https://stackoverflow.com/questions/17667903/python-socket-receive-large-amount-of-data>
- <https://en.wikipedia.org/wiki/JSON>
- [https://en.wikipedia.org/wiki/RSA_\(cryptosystem\)](https://en.wikipedia.org/wiki/RSA_(cryptosystem))

סקירת הפתרון הנבחר:

בפרויקט זה נבחר מנגנון תקשורת מותאם אישית, בעל יתרונות של גמישות, אבטחה ותמיכה במידע באורך משתנה. הפתרון כולל את המרכיבים הבאים:

1. הצפנה איסימטרית באמצעות RSA

- הלקוח מצפין את הסיסמה באמצעות מפתח ציבורי הנמצא ברשותו (מופק בצד השרת).
- השרת מפענח את ההודעה באמצעות מפתח פרטי תואם, כך שגורם צד שלישי אינו יכול לקרוא את הנתונים המועברים.
- מנגנון זה מונע אפשרות של **man-in-the-middle** ומבטיח **סודיות** גם בתעבורת רשת רגילה.

2. המרת ההודעות לפורמט JSON

- כל ההודעות מועברות כ- strings בפורמט JSON, המאפשר סדר, קריאות ושימוש בטיפוסים פשוטים להעברה.
- פורמט זה מאפשר העברת מידע מובנה (כמו מילונים ורשימות), כך שכל צד יודע בדיוק איך לפרש את ההודעה.
- נבחר פורמט JSON ולא pickle בשל רמת הקריאות והאבטחה הגבוהה יותר (pickle פגיע להזרקת קוד)

3. סיום הודעה באמצעות מזהה ייחודי (__END__)

- כדי להתמודד עם מגבלת גודל הודעה (1024 בתים בכל קריאה מ- socket), כל הודעה שמועברת מהלקוח לשרת (או להפך) מסתיימת במחרוזת מיוחדת: **__END__**.
- כך, גם אם ההודעה נשלחת במספר מקטעים – הצד המקבל יודע בדיוק מתי ההודעה הסתיימה בפועל.
- השימוש ב- **__END__** הוא פרקטיקה ידועה בתקשורת socket שאינה מבוססת פרוטוקול מסודר.

4. שכבת עיבוד מרוכזת של תקשורת

- בקובץ התקשורת של השרת והלקוח קיימת פונקציית שליחה אחידה (send) אשר מבצעת את כל תהליך ההמרה, ההצפנה והשליחה.
- פונקציית הקבלה (receive) יודעת להרכיב את כל מקטעי ההודעה יחד עד לזיהוי __END__, ואז לפענח את התוכן.

היתרונות של פתרון זה:

- אינו תלוי בפרוטוקול HTTP ולכן מתאים גם לאפליקציות שאינן Web.
- גמיש מאוד – ניתן לשלוח כל הודעה בכל מבנה נתונים.
- מגן על סיסמאות ונתונים רגישים בצורה מוצפנת, ללא צורך בהעברת מפתחות משותפים מראש.
- תומך בהודעות ארוכות מאוד (כמו כל ההזמנות הקיימת של לקוח מסוים או כל ההזמנות בתאריך מסוים), הודות למנגנון קבלת הודעה חלקית.

חשוב לציין:

בפרויקט זה, המידע שמועבר בין השרת ללקוח אינו כולל פרטים רגישים במיוחד כמו מספרי כרטיס אשראי, תעודת זהות או כתובות מגורים. המידע הרגיש היחיד שעובר בתקשורת הוא **הסיסמה של המשתמש** – והיא מוגנת בשתי שכבות הצפנה **RSA**: להעברה ו **bcrypt** לאחסון, כך שהסיכון לחשיפת מידע רגיש נמוך מאוד, והמנגנונים שנבחרו נותנים מענה מצוין לדרישות אבטחה אלה.

שם האלגוריתם: איתור מסעדות פנויות בהתחשב בפרטי ההזמנה

ניסוח וניתוח הבעיה האלגוריתמית:

בעת חיפוש מסעדה, המשתמש מזין קריטריונים שונים כמו תאריך, שעה, מספר סועדים, אזור, סוג אוכל, כשרות ודירוג. לאחר סינון ראשוני של המסעדות שעונות על הקריטריונים, יש לבדוק האם במסעדות אלו קיים שולחן פנוי בתאריך ובשעה המבוקשים, בהתאם לקטגוריית הישיבה המבוקשת. האתגר המרכזי הוא לוודא שהשולחן לא כבר הוזמן לאותו הזמן, מבלי ליצור התנגשויות בין הזמנות קיימות לחדשות.

אלגוריתמים קיימים לפתרון הבעיה:

- סינון מסעדות לפי מאפיינים מתוך טבלת restaurants.
- שליפת שולחנות רלוונטיים לפי קטגוריה וגודל מתוך טבלת tables.
- ביצוע ספירה של הזמנות קיימות בטבלת reservations עבור כל שולחן אפשרי במסעדה בתאריך ובטווח השעות המבוקש.
- השוואת כמות השולחנות הכוללת מול מספר ההזמנות באותו זמן כדי לבדוק זמינות בפועל.

הפנייה למקורות רלוונטיים:

- <https://stackoverflow.com/questions/2854927/sql-find-overlapping-reservations>
- <https://www.sqltutorial.org/sql-aggregate-functions/sql-count/>

סקירת הפתרון הנבחר:

לאחר סינון המסעדות העומדות בקריטריונים הבסיסיים, המערכת ניגשת לבדוק זמינות בפועל על ידי מעבר על כל השולחנות המתאימים (לפי גודל וסוג ישיבה) במסעדות אלו. לכל שולחן, המערכת סופרת את מספר הפעמים שהוא כבר הוזמן לאותו תאריך ושעה. החישוב מתבצע בהתחשב גם במשך ההזמנה (למשל, 90 דקות). רק אם מספר ההזמנות לשולחנות אלו קטן ממספר השולחנות הכולל שמציעה המסעדה – המסעדה תיחשב כזמינה.

גישה זו מאפשרת למערכת לספק מידע מדויק ואמין למשתמש, כך שהתוצאה הסופית כוללת אך ורק מסעדות שיש בהן בפועל שולחנות פנויים בהתאם להגבלות הזמן והמאפיינים שנבחרו.

היתרונות של הפתרון:

- מונע כפילויות והזמנות חופפות
- מתחשב בכמות שולחנות זהים במסעדה
- מבטיח שהמשתמש רואה רק אופציות זמינות אמיתיות

תיאור סביבת הפיתוח**כלי הפיתוח הדרושים לפיתוח:**

- **שפת תכנות Python 3.10 – ומעלה:**
שפת תכנות עילית, פשוטה לקריאה ולתחזוקה, המתאימה במיוחד לפיתוח מערכות מבוססות Client-Server. מאפשרת שימוש נרחב בספריות להצפנה, תקשורת GUI, ועוד.
- **IDE – PyCharm (Community Edition):**
סביבת פיתוח משולבת לפיתוח הכוללת:
 - ניהול קבצים ומודולים
 - השלמה אוטומטית (IntelliSense)
 - ניפוי שגיאות (debugging)
 - הרצה מקומית נוחה של צד לקוח ושרת
- **GUI Framework – CustomTkinter:**
הרחבה של Tkinter לבניית ממשק משתמש גרפי מודרני, רספונסיבי וידידותי. מאפשרת עיצוב חדשני וקל לשימוש.
- **מסד נתונים: SQLite –**
מסד נתונים מקומי בקובץ db, ללא צורך בשרת חיצוני. משתלב בצורה ישירה עם פייתון באמצעות sqlite3. מאפשר אחסון נתוני משתמשים, מסעדות, הזמנות, שולחנות ועוד.

• ספריות פיתוח עיקריות:

- socket לתקשורת TCP/IP בין הלקוח לשרת
- json המרת אובייקטים לטקסט ולהפך בפורמט קריא ונוח
- bcrypt להצפנה חד-כיוונית (Hashing) של סיסמאות
- threading להרצת פעולות ברקע (למשל שליחת מיילים)
- smtplib שליחת הודעות דוא"ל דרך SMTP
- PIL (Pillow) להצגת תמונות בממשק (לוגו)
- re, datetime לבדיקת תקינות קלט, עיבוד זמנים ותאריכים

כלים וסביבה הדרושים לבדיקות:

1. **CMD / PowerShell (שורת פקודה)**
לבדיקת כתובת IP של השרת (ipconfig) בעת בדיקות בין מחשבים שונים ברשת מקומית.
2. **DB Browser for SQLite:**
כלי גרפי לצפייה בתוכן קובץ מסד הנתונים, בדיקת טבלאות, שליפות ועדכונים – חשוב לאימות תקינות הפעולות מול המאגר.
3. **בדיקות מקומיות: (Local Testing)**
הפעלת קובצי השרת והלקוח על אותה מכונה או מחשבים נפרדים, בדיקת זרימת בקשות ותגובות, תוך שימוש בהודעות print() לקלאיתור תקלות.
4. **בדיקות תקשורת בין מחשבים:**
בדיקת יציבות חיבור TCP, טיפול בניתוק בלתי צפוי של לקוח, בדיקת תגובות לקלטים לא תקינים, והרצת מספר לקוחות במקביל.
5. **סביבת ההפעלה:**
 - מערכת הפעלה Windows 10/11 :
 - פייתון מותקן מראש (גרסה 3.10+)
 - הפעלת המערכת כקבצים עצמאיים (server.py, gui.py)
 - ללא צורך בחיבור אינטרנט או שרת חיצוני – מבוסס רשת מקומית (LAN / localhost)

תיאור פרוטוקול התקשורת

תיאור מילולי:

המערכת מבוססת על **מודל TCP/IP** ומיישמת תקשורת דו-כיוונית בין הלקוח לשרת באמצעות ספריית socket של Python. התקשורת מבוצעת באמצעות פרוטוקול **TCP** – המבטיח אמינות, סדר ושלמות הנתונים.

פורמט ההודעות בין הלקוח לשרת, ובין השרת ללקוח:

הלקוח שולח לשרת הודעות בפורמט **JSON** כל הודעה כוללת לפחות את המפתח "action" שמתאר את סוג הפעולה המבוקשת (למשל: "login", "register", "search") שדות נוספים נכללים בהתאם לפעולה.

" action": "login,"

" username": "david123,"

" encrypted_password": "AF8B7C2"...

השרת מחזיר תשובה גם היא בפורמט **JSON** כל תשובה כוללת לפחות את המפתח "status" עם הערכים "success" או "error", ולרוב גם מפתח "message".

"message": "Login successful",

"user_id": 12,

"email_address": "user@example.com"

קידוד ופענוח

- בצד השולח: ההודעה מקודדת באמצעות `UTF-8 (.encode("utf-8"))`
 - בצד המקבל: ההודעה מפוענחת באמצעות `.decode("utf-8")`.
- כך ניתן להעביר הודעות בכל שפה ולשמור על תאימות מלאה בין רכיבי המערכת.

זיהוי סוף הודעה (END)

כיוון שההודעות נקלטות ב־ chunks (בקריאות של `recv(1024)`) לא תמיד כל ההודעה מגיעה במכה אחת. לכן, כל הודעה מסתיימת בסימן ייחודי `__END__` –

צד המקבל מקבל את ההודעה בלולאה – וממשיך לצרף חלקים עד שהוא מזהה את הסוף. `(__END__)` זהו פתרון פשוט, גמיש, ואינו דורש שליחת אורך ההודעה מראש.

הצפנה

- בעת התחברות או הרשמה, שדה הסיסמה **מוצפן בעזרת RSA** – הלקוח משתמש במפתח הציבורי של השרת כדי להצפין את הסיסמה.
- השרת מפענח את הסיסמה באמצעות המפתח הפרטי ששמור אצלו בלבד.

- בדרך זו, גם אם ייורט המידע – הסיסמה לא ניתנת לפענוח.
 - יתר שדות ההודעה (כמו שם משתמש, תאריך, אזור, סוג אוכל וכו') **אינם מוצפנים**, שכן אינם נחשבים רגישים.
 - בעת שליחת אימיילים (למשל, קוד אימות או אישור הזמנה), נעשה שימוש בפרוטוקול TLS באמצעות הפעלת `starttls()` כך שהתעבורה בין האפליקציה לשרת הדוא"ל מוצפנת ובטוחה.
- פרוטוקול תקשורת זה נבחר בזכות:
- **גמישות גבוהה** – העברת מבני מידע מורכבים בפשטות.
 - **אמינות מלאה** – בזכות TCP.
 - **הגנה על סיסמאות** – בזכות הצפנת RSA.
 - **זיהוי ברור של סוף הודעה** – ללא צורך בניהול אורך.
 - **פשטות תחזוקה** – מבנה אחיד לכל הבקשות והתשובות.
- ניתן להרחיב את המערכת בעתיד לשימוש בפרוטוקולי אבטחה נוספים (כגון TLS) אך במתכונתה הנוכחית היא נותנת מענה יעיל ומוגן לסוגי המידע שעוברים בה.

פירוט כלל ההודעות הזורמות במערכת:

הודעות מהלקוח לשרת:

שם ההודעה	שדות שמועברים בהודעה
get public key	action
register	action, first_name, last_name, phone, email, username, encrypted_password
login	action, username, encrypted_password
search	action, table_size, time, date, food_types, districts, price, rating, category kosher,
restaurant_info	action, restaurant
order	action, restaurant, category, table_size, reservation_time, date, user_id
cancel_reservation	action, reservation_id
admin_view_orders_by_user	action, user_name

הודעות מהשרת ללקוח:

שם ההודעה	שדות שמועברים בהודעה
response	status, public key
register_response	status, message
login_response	status, user_id, first_name, email_address, message
search_response	name, food_type, city, district, average_price_per_person, kosher, rating, seating_availability, restaurant_id, matched_tables, available_tables, category
restaurant_info_response	status, city, food_type, rating
order_response	status, end_reservation, message
cancel_response	status, message
orders_response	status, orders

הודעות מאדמין לשרת:

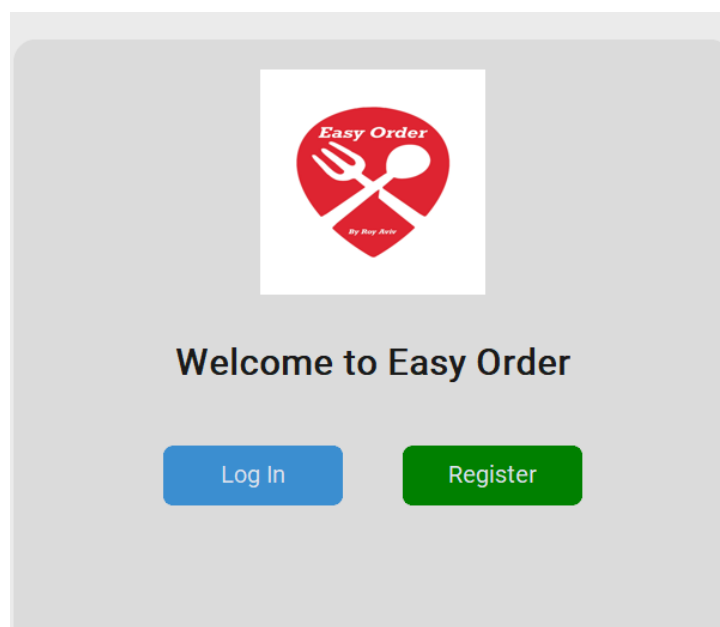
שם ההודעה	שדות שמועברים בהודעה
admin_view_orders_by_date	action, date
admin_view_orders_by_user	action, user_name
admin_get_popular_restaurants	action
admin_get_active_users	action
admin_add_restaurant	action, name, city, food_type, district, average_price, kosher, rating
admin_delete_restaurant	action, restaurant_name
admin_edit_restaurant	action, restaurant_name, field, new_value
admin_add_tables	action, restaurant_name, category, size, quantity
admin_remove_tables	action, restaurant_name, category, size, quantity

הודעות מהשרת לאדמין:

שם ההודעה	שדות שמועברים בהודעה
orders_by_date_response	status, orders
orders_by_user_response	status, orders
popular_restaurants_response	status, restaurants
active_users_response	status, users
add_restaurant_response	status, message
delete_restaurant_response	status, message
edit_restaurant_response	status, message
add_tables_response	status, message
remove_tables_response	status, message

תיאור מסכי המערכת:מסך פתיחה (Welcome Screen) :

- **כותרת** Welcome to Easy Order
- **תפקיד** : זהו המסך הראשי עם שתי אפשרויות – התחברות (Log In) או הרשמה (Register).
- **תיאור** : במסך זה מוצג הלוגו של המערכת וכפתורים למעבר למסכי ההתחברות או ההרשמה. המשתמש בוחר את הפעולה הרצויה.



מסך התחברות (Log In):

- **Welcome Back! כותרת**
- **תפקיד:** התחברות של משתמש קיים.
- **תיאור:**
 - כולל שדות שם משתמש וסיסמה.
 - לאחר ההתחברות (והזדהות עם אימות דו-שלבי), המשתמש מועבר למסך החיפוש.
 - כולל אימות סיסמה מוצפנת ב־ RSA ובדיקת תקינות עם bcrypt בצד השרת.

**Welcome Back!**

Username:

Password:

Log In

Back

מסך הרשמה (Register):

- **Create Your Account כותרת**
- **תפקיד:** יצירת משתמש חדש במערכת.
- **תיאור:**
 - כולל שדות לפרטים אישיים: שם פרטי, שם משפחה, טלפון, אימייל, שם משתמש וסיסמה.
 - כוללת בדיקת חוזק סיסמה גרפית.
 - הנתונים נשלחים לשרת לאחר הצפנה ב־ RSA.

**Create Your Account**

First Name:

Last Name:

Phone Number:

Email:

Username:

Password:

Confirm Password:

Register

Back

מסכי משתמש:**מסך אימות דו-שלבי (Two-Factor Authentication)**

• **כותרת** Verification Required• **תפקיד** : לאמת את זהות המשתמש ע"י קוד אימות במייל.• **תיאור** :

○ מופיע לאחר התחברות מוצלחת.

○ המשתמש מזין קוד שקיבל באימייל.

○ יש טיימר של 120 שניות ואפשרות לשלוח קוד חדש.

מסך חיפוש מסעדות (Restaurant Search Form)• **כותרת** Search Restaurants• **תפקיד** : הגדרת קריטריונים לחיפוש מסעדות.• **תיאור** :

○ טופס שמאפשר לבחור :

▪ מספר אנשים, תאריך ושעה

▪ אזור, סוג אוכל, טווח מחירים, כשרות ודירוג

▪ העדפת ישיבה (פנים / חוץ)

○ לחיצה על כפתור "Search" שולחת בקשה לשרת ומביאה תוצאות תואמות.

Search Results

Restaurants found:

- ☐ Pizza Chain - Indoor and Outdoor
- ☐ Hummus Abu-Obaid - Indoor and Outdoor
- ☐ Gan Ha'ir Cafe - Indoor and Outdoor
- ☐ McDonalds - Indoor and Outdoor
- ☐ Taco El Paso - Indoor and Outdoor
- ☐ Italy Roma - Indoor and Outdoor
- ☐ Fish Restaurant by the Sea - Indoor and Outdoor
- ☐ The Oriental Restaurant - Indoor and Outdoor

Continue

Restaurant Info



Details for Fish Restaurant by the Sea

City: Tel Aviv

Food Type: Fish

Rating: 4.3

Continue

מסך תוצאות חיפוש (Search Results)

- **Restaurants found** כותרת
- **תפקיד:** הצגת רשימת המסעדות התואמות.
- **תיאור:**
 - מוצגת רשימה של מסעדות עם זמינות ישיבה.
 - המשתמש בוחר מסעדה להמשך התהליך.

מסך פרטי מסעדה (Restaurant Info)

- **Details for [Restaurant Name]** כותרת
- **תפקיד:** הצגת פרטים על המסעדה שנבחרה.
- **תיאור:**
 - מציג עיר, סוג אוכל, דירוג.
 - מאפשר להמשיך להזמנה של שולחן במסעדה זו.

מסך בחירת מקום ישיבה (Seating Preference Selection)

- **Seating Preference** כותרת
- **תפקיד:** בחירת העדפת מקום ישיבה במסעדה.
- **תיאור:**

- לאחר בחירת מסעדה מוצגת למשתמש אפשרות לבחור ישיבה בפנים (Indoor), בחוץ (Outdoor) אם לא בחר בסינון הראשוני ולמסעדה יש מקום גם בפנים וגם בחוץ.

Select Seating Preference

Select seating for Fish Restaurant by the Sea:

- ☐ Indoor
- ☐ Outdoor

Confirm

Order Your Table



Reserve a table at Fish Restaurant by the Sea

 **Date:** 2025-05-22

 **Time:** 20:00

 **People:** 2

 **Seating:** Outdoor

Book Table

מסך סיכום הזמנה (Reservation Summary)

- **כותרת** Review Your Reservation
- **תפקיד**: הצגת פרטי ההזמנה לפני אישור סופי.
- **תיאור**:

- מוצג סיכום של המסעדה, תאריך, שעה, מספר אנשים והעדפת ישיבה.
- מאפשר למשתמש לאשר את ההזמנה או לחזור אחורה.

מסך אישור הזמנה (Reservation Confirmation)

- **כותרת** Reservation Confirmed
- **תפקיד**: הצגת אישור להזמנה שבוצעה בהצלחה.
- **תיאור**:

- מוצג חלון עם כל פרטי ההזמנה כולל זמן התחלה וזמן סיום ההזמנה.
- נשלח מייל אישור עם פרטי ההזמנה.

Reservation Confirmed



Reservation Confirmed!

 **Restaurant:** Fish Restaurant by the Sea

 **Date:** 2025-05-22

 **Time:** 20:00 - 21:30

 **People:** 2

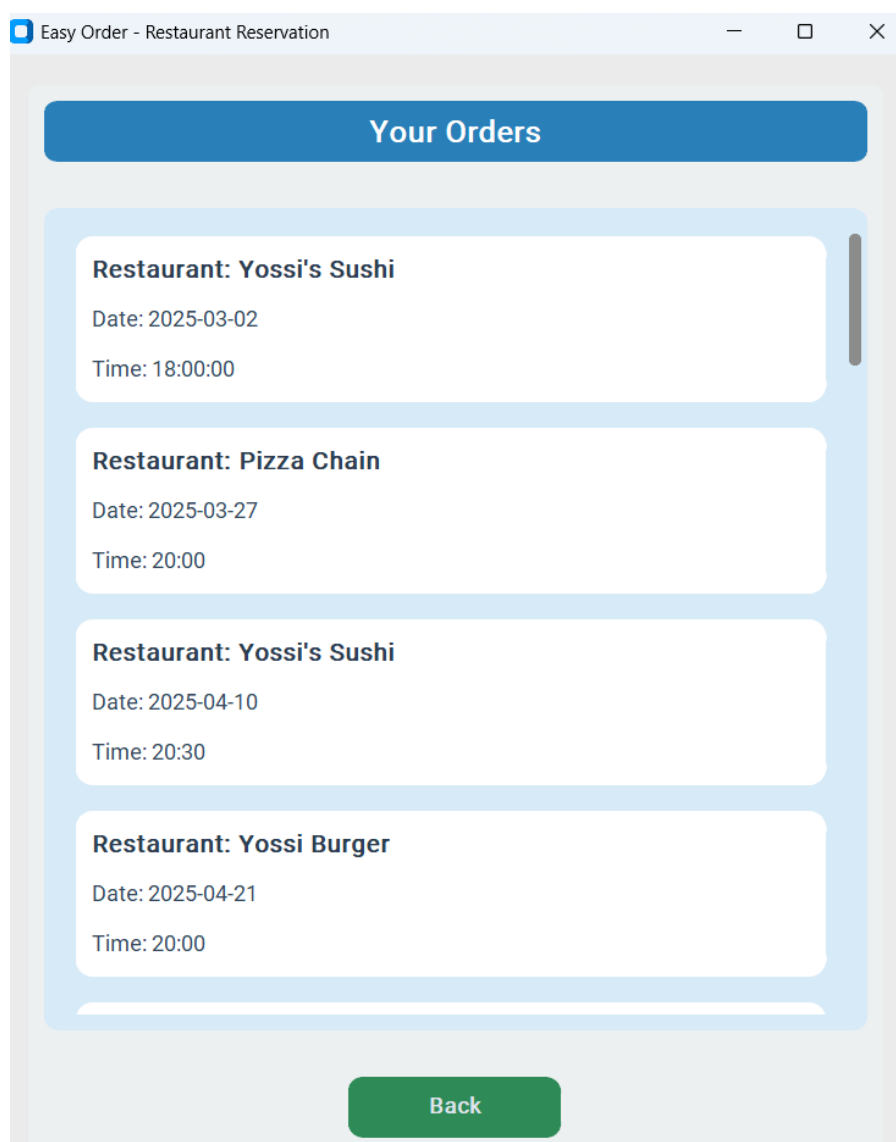
 **Seating:** Outdoor

A confirmation email has been sent to your inbox.

Close

מסך צפייה בהזמנות (Your Orders)

- **Your Orders** כותרת
- **תפקיד** : צפייה בהיסטוריית ההזמנות של המשתמש.
- **תיאור** :
 - מוצגות ההזמנות לפי תאריך עם אפשרות גלילה.
 - כולל כפתור לביטול הזמנה עתידית.
 - כולל כפתור חזרה למסך הבית של הממשק.
 - כל כרטיס של הזמנה כולל : שם מסעדה, תאריך, שעה.



מסכי אדמין :**מסך ניהול ראשי (Admin Panel)**

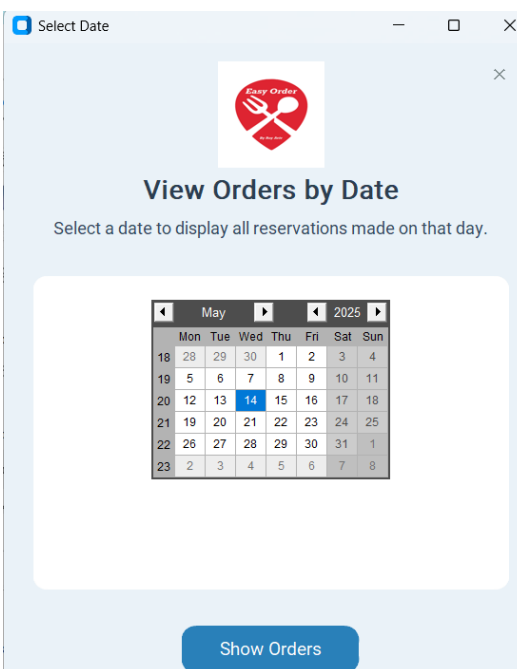
- **כותרת** Admin Panel
- **תפקיד** : ניהול מערכת ע"י אדמין.
- **תיאור** :

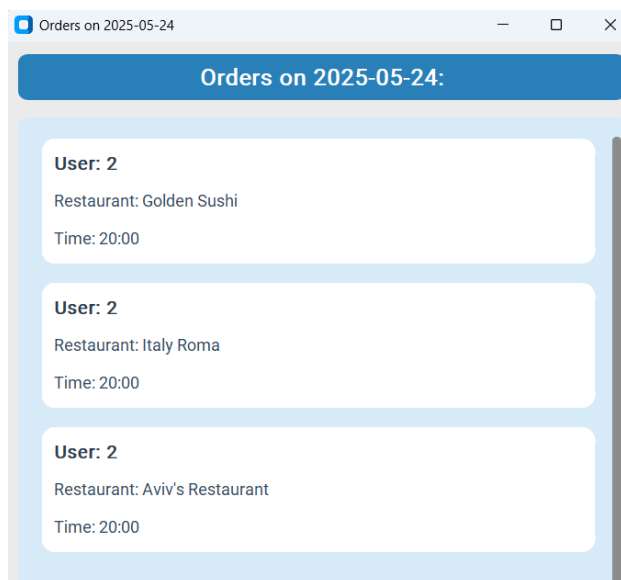
○ מציג תפריט פעולות ניהוליות :

- צפייה בהזמנות לפי תאריך
- צפייה בהזמנות לפי משתמש
- צפייה במסעדות הפופולריות ביותר
- צפייה במשתמשים הפעילים ביותר
- עריכת מסעדות
- עריכת שולחנות במסעדות

מסך בחירת תאריך להצגת הזמנות (Select Date for Orders)

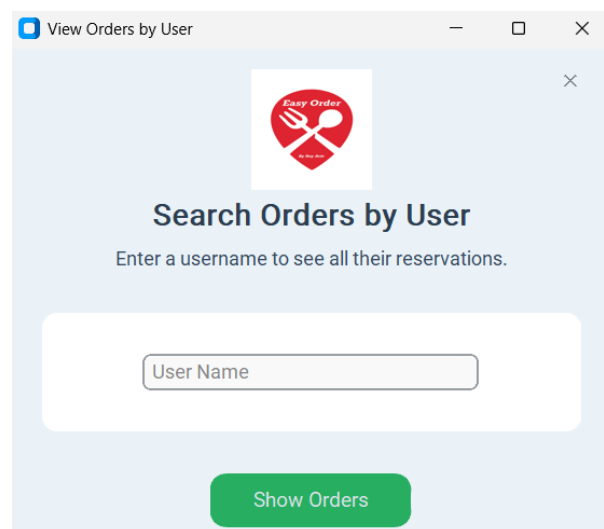
- **כותרת** Select a Date
 - **תפקיד** : לבחור תאריך להצגת ההזמנות של אותו יום.
 - **תיאור** :
- האדמין בוחר תאריך על ידי לחיצה עליו בלוח השנה שמוצג לו.
- לחיצה על כפתור "Show Orders" תציג לאדמין את כל ההזמנות באותו התאריך.





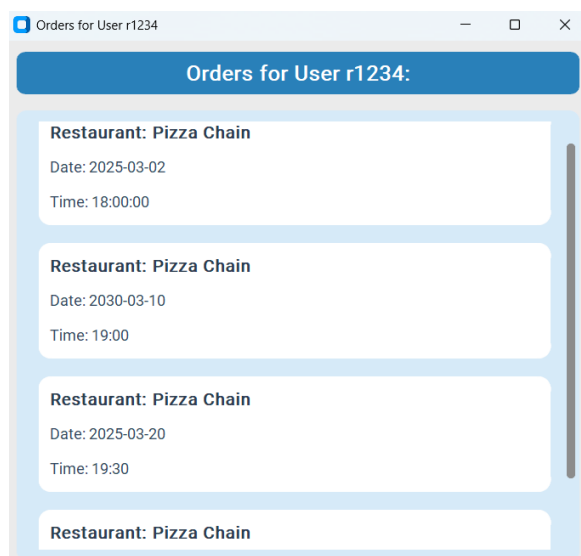
מסך תוצאות הזמנות לפי תאריך (Orders on Selected Date)

- **כותרת** Orders on Selected Date
- **תפקיד** : הצגת ההזמנות בתאריך שנבחר.
- **תיאור** :
 - מוצגת רשימה של הזמנות הכוללת :
 - מזהה משתמש
 - שם מסעדה
 - שעת ההזמנה
 - יש אפשרות לחזור למסך בחירת תאריך



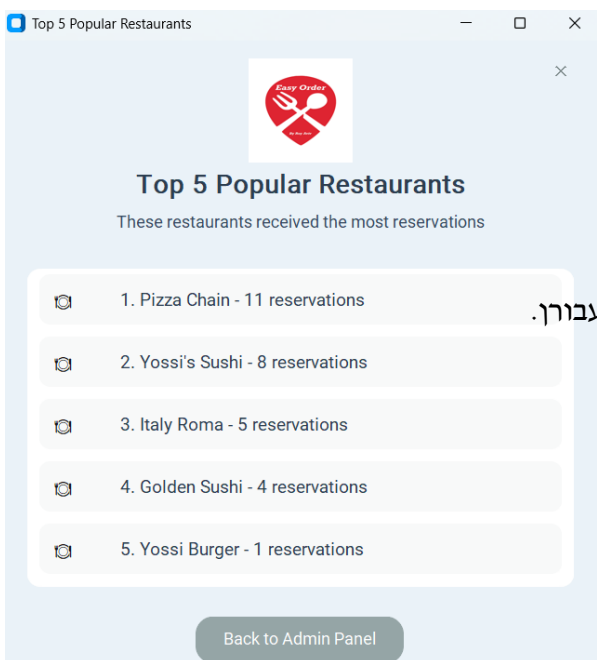
מסך הזנת שם משתמש להצגת הזמנות (Enter Username)

- **כותרת** View Orders by User
- **תפקיד** : הזנת שם משתמש להצגת ההיסטוריה שלו.
- **תיאור** :
 - האדמין מזין שם משתמש של לקוח.
 - לאחר לחיצה על "Show Orders" מתקבלות כל ההזמנות של המשתמש הזה.



מסך תוצאות הזמנות לפי משתמש (Orders of Selected User)

- **כותרת** Orders for Selected User
- **תפקיד** : הצגת רשימת ההזמנות של המשתמש.
- **תיאור** :
 - לכל שורה מוצגים :
 - מזהה ההזמנה
 - תאריך ההזמנה
 - שעת ההזמנה
 - שם המסעדה

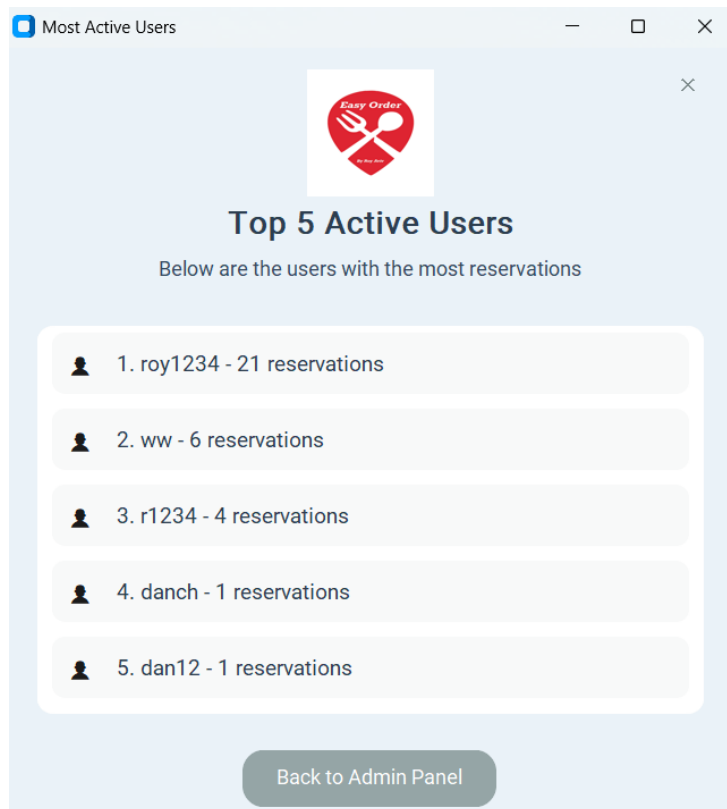


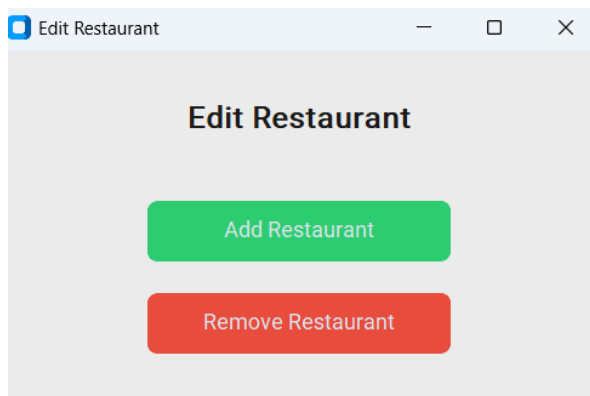
מסך צפייה במסעדות הפופולריות (Most Popular Restaurants)

- כותרת Top 5 Popular Restaurants
- תפקיד: להציג את המסעדות עם הכי הרבה הזמנות.
- תיאור:
 - מוצגות מסעדות ממוינות לפי כמות הזמנות שהתקבלו עבורן.
 - כפתור חזרה למסך אדמין ראשי

מסך צפייה במשתמשים הפעילים (Most Active Users)

- כותרת Top Users
- תפקיד: להציג את המשתמשים שביצעו הכי הרבה הזמנות.
- תיאור:
 - מוצגים שמות משתמש וכמות ההזמנות שביצעו.
 - כפתור חזרה למסך אדמין ראשי





מסך ניהול מסעדות (Edit Restaurant)

- **כותרת** Edit Restaurant
- **תפקיד** : הצגת אפשרויות לניהול מסעדות.
- **תיאור** :
 - מוצגים שני כפתורים :
 - Add Restaurant מעבר למסך הוספת מסעדה.
 - Remove Restaurant מעבר למסך מחיקת מסעדה.
 - מאפשר לאדם לבצע פעולות תחזוקה על מאגר המסעדות.

מסך הוספת מסעדה (Add New Restaurant)

- **כותרת** Add New Restaurant
- **תפקיד** : הוספת מסעדה חדשה למערכת.
- **תיאור** :
 - טופס מפורט להזנת פרטי המסעדה :
 - שם, סוג אוכל, עיר, מחוז, מחיר ממוצע, דירוג, שעות פתיחה/סגירה, כשרות, משך הזמנת שולחן.
 - כולל בדיקות תקינות :
 - פורמט שעה, ערכים חוקיים למחוז וסוג אוכל.
 - לחיצה על Add Restaurant שולחת את הנתונים לשרת.

מסך מחיקת מסעדה (Delete Restaurant)

- **כותרת** Delete Restaurant
- **תפקיד** : מחיקת מסעדה קיימת מהמערכת.
- **תיאור** :

- האדמין מזין את שם המסעדה שברצונו למחוק.
- לחיצה על Delete Restaurant שולחת בקשה לשרת למחיקת המסעדה מהמאגר.
- מתקבלת הודעת אישור או שגיאה בהתאם לתוצאה.

מסך ניהול שולחנות (Edit Tables)

- **כותרת** Edit Tables
- **תפקיד** : ניהול שולחנות של מסעדות במערכת.
- **תיאור** :

- מוצגים שני כפתורים :
 - Add Tables – מעבר למסך הוספת שולחנות.
 - Remove Tables – מעבר למסך הסרת שולחנות.
- הממשק מאפשר להוסיף או להסיר שולחנות למסעדה לפי פרמטרים מותאמים

מסך הוספת שולחנות (Add Tables)

- **כותרת** Add Tables to Restaurant
- **תפקיד** : הוספת שולחנות חדשים למסעדה קיימת.
- **תיאור** :

- האדמין מזין :
 - שם המסעדה, קטגוריית השולחן, (Indoor/Outdoor) גודל השולחן, וכמות שולחנות להוספה.

- לאחר לחיצה על Add Tables הבקשה נשלחת לשרת.
- מתקבלת תגובה בהתאם להצלחה או כשל.

מסך הסרת שולחנות (Remove Tables)

- **כותרת** Remove Tables from Restaurant

- **תפקיד** : הסרת שולחנות ממסעדה מסוימת.

- **תיאור** :

- כמו במסך הוספה, יש להזין :

- שם המסעדה, קטגוריה, גודל שולחן, וכמות שולחנות להסרה.

- לחיצה על Remove Tables תשלח את הבקשה לשרת.

- מתקבלת תגובה בהתאם להצלחה או כשל.

The screenshot shows a web browser window titled "Remove Tables". The form has a light gray background and contains the following elements:

- Restaurant Name**: A text input field with the placeholder "Restaurant Name".
- Table Category**: Two radio button options: "Indoor" (selected) and "Outdoor".
- Table Size (Number of Seats)**: A text input field with the placeholder "e.g. 4".
- Number of Tables**: A text input field with the placeholder "e.g. 3".
- Remove Tables**: A red button with white text.



תיאור מבני הנתונים-פירוט כללי:

בפרויקט זה נעשה שימוש במסד נתונים מסוג SQLite3. זהו מסד נתונים קובצי (DB) שמוטמע כחלק מהשרת וניגש אליו באמצעות ספריית sqlite3 של פייתון. מסד הנתונים מכיל מספר טבלאות המקושרות ביניהן באמצעות מפתחות זרים (Foreign Keys) ונעשה בו שימוש ב-Triggers (טריגרים) לאוטומציה של פעולות.

שם בסיס הנתונים - Tablesטבלאות:

טבלה מספר 1 - restaurants טבלה בה נמצאות כל המסעדות והפרטים שלהם.

מאפיינים	דוגמאות	מפתח זר	מפתח ראשי	טיפוס	תיאור השדה	שם השדה
NOT NULL, AUTOINCREMENT	1, 2, 3		✓	INTEGER	מזהה ייחודי של המסעדה	id
NOT NULL	Cafe Greg, SushiBar			TEXT	שם המסעדה	name
NOT NULL	Italian, Sushi			TEXT	סוג אוכל	food_type
	10:00, 13:00			TEXT	שעת פתיחה	opening_time
	18:00, 23:00			TEXT	שעת סגירה	closing_time
				INTEGER	זמן שמירת שולחן	Table_reservation_duration
NOT NULL	Tel Aviv, Haifa			TEXT	עיר	city
	Gush Dan, North			TEXT	אזור	district
	50.0, 75.5			REAL	מחיר ממוצע לסועד	average_price_per_person
	True, False			BOOLEAN	כשרות	kosher
	4.5, 3.9			REAL	דירוג	rating

טבלה מספר 2 - tables טבלה המקושרת לטבלת המסעדות המציגה את כלל השולחנות של כל המסעדות.

שם השדה	תיאור השדה	טיפוס	מפתח ראשי	מפתח זר	דוגמאות	מאפיינים
id	מזהה ייחודי של שולחן	INTEGER	✓		1, 2, 3	NOT NULL, AUTOINCREMENT
restaurant_id	מזהה מסעדה	INTEGER		restaurants.id	1, 2	NOT NULL
table_size	גודל שולחן	INTEGER			2, 4, 6	NOT NULL
category	קטגוריית ישיבה	TEXT			Indoor, Outdoor	NOT NULL
Number_of_tables	כמות שולחנות מסוג זה	INTEGER			4,8,2	NOT NULL

טבלה מספר 3 - users טבלה בה נמצאים כלל המשתמשים במאגר

שם השדה	תיאור השדה	טיפוס	מפתח ראשי	מפתח זר	דוגמאות	מאפיינים
id	מזהה משתמש	INTEGER	✓		1, 2, 3	NOT NULL, AUTOINCREMENT
first_name	שם פרטי	TEXT			David, Dana	NOT NULL
last_name	שם משפחה	TEXT			Cohen, Levi	NOT NULL
phone	טלפון	TEXT			0532802549	NOT NULL
email_address	דוא"ל	TEXT			example@gmail.com	NOT NULL
user_name	שם משתמש	TEXT			david123	UNIQUE, NOT NULL
user_password	סיסמה מוצפנת	TEXT			\$2b\$12\$vRtlPe0/...	NOT NULL
createt_at	זמן יצירת המשתמש	DATETIME			2025-03-09 14: 19: 05	CURRENT_TIMESTAMP

טבלה מספר 4 - reservations טבלת כלל ההזמנות הקיימות.

שם השדה	תיאור השדה	טיפוס	מפתח ראשי	מפתח זר	דוגמאות	מאפיינים
id	מזהה הזמנה	INTEGER	✓		1, 2	NOT NULL, AUTOINCREMENT
user_id	מזהה משתמש	INTEGER		users.id	1, 2	NOT NULL
table_id	מזהה שולחן	INTEGER		tables.id	1, 3	NOT NULL
reservation_date	תאריך הזמנה	DATE			2024-05-20	NOT NULL
reservation_time	שעת הזמנה	TIME			18:00	NOT NULL
end_time	שעת סיום	TIME			19:30	

בטבלה זו ישנו טריגר: Set_end_time_after_insert

מטרת הטריגר: הטריגר נועד לחשב ולהזין אוטומטית את שעת סיום ההזמנה (end_time) בטבלת reservations, על בסיס שעת התחלת ההזמנה (reservation_time) ומשך זמן ההזמנה שנקבע במסעדה (reservation_duration), ללא צורך בהתערבות של השרת או הלקוח.

אופן הפעולה:

כאשר מתווספת שורה חדשה לטבלת reservations (כלומר, מתבצעת הזמנה חדשה), הטריגר פועל לפי השלבים הבאים:

1. מאחזר את table_id מהשורה החדשה.
2. באמצעות table_id מאתר את restaurant_id המתאים בטבלת tables.
3. מאתר את reservation_duration (משך ההזמנה בדקות) של אותה מסעדה בטבלת restaurants.
4. מחשב את זמן הסיום על ידי הוספת reservation_duration לשעת ההזמנה (reservation_time).
5. מעדכן את השדה end_time של ההזמנה החדשה עם תוצאת החישוב.

חשיבות הטריגר:

- **אוטומציה מלאה** – אין צורך לחשב את זמן הסיום בצד הלקוח או השרת.
- **מניעת טעויות אנוש** – חישוב מדויק ואחיד שמתבצע תמיד על בסיס נתוני המסעדה.
- **שיפור הביצועים בבדיקת זמינות שולחנות** – מאפשר לבצע שאילתות המבוססות על זמן התחלה וסיום, וכך לזהות התנגשויות בין הזמנות בצורה אמינה.

דוגמה:

נניח שמשתמש מזמין שולחן במסעדה בשעה 18:00, והמסעדה מגדירה reservation_duration של 90 דקות:

- זמן ההזמנה (reservation_time) שנרשם הוא 18:00.
- הטריגר מוסיף 90 דקות end_time => יתעדכן אוטומטית ל-19:30.

סקירת חולשות ואיומים**שכבת האפליקציה:****• עבודה עם בסיס נתונים SQL Injection –****איום:**

חדירת קוד זדוני דרך שאילתות SQL עלולה לאפשר קריאת נתונים, שינוי או מחיקה שלהם, גם ללא הרשאות מתאימות.

הפתרון:

- כל השאילתות בפרויקט מבוצעות בשימוש בפרמטרים מאובטחים (?) ולא ע"י מחרוזות שעוברות format או f-string.
- שימוש נכון ב־sqlite עם prepared statements.

• עבודה עם אתרי Web**איום:**

בפרויקט זה אין אינטגרציה עם אתרים חיצוניים או טעינת תכנים מ־Web, ולכן האיום נמנע מראש.

הפתרון:

- הפרויקט הוא אפליקציית Desktop בלבד (מבוססת GUI) ללא גישה לדפדפן או ל־Web APIs.

• תהליך ה־Login אימות ווידוא זהות**איום:**

גישה לחשבון של משתמש אחר, אם האימות אינו מבוסס היטב.

הפתרון:

- אימות דו-שלבי (Two Factor Authentication) אחרי התחברות עם שם משתמש וסיסמה – נדרש גם קוד חד-פעמי מהמייל.
- סיסמאות מוצפנות ב־ RSA בשליחה, ולאחר מכן ב־ bcrpt בעת שמירה במסד.
- הסיסמה לעולם אינה מועברת בטקסט גלוי או נשמרת בפורמט ניתן לפענוח.
- כל בקשה נבדקת על ידי השרת מבחינת תקינות מבנה, סוג פעולה, ותכולת השדות.
- יש בקרת גישה – רק משתמשים מאומתים (כולל אימות דו-שלבי) יכולים לבצע פעולות רגישות כמו הזמנה או צפייה בפרטי משתמשים.

● האזנה או שינוי תוכן בזמן מעבר - MITM (Man-in-the-Middle)

איום:

האקר באמצע התקשורת עלול לקרוא את הסיסמה או לשנות את הבקשות.

הפתרון:

- כל סיסמה שמועברת מהלקוח לשרת מוצפנת באמצעות RSA (הצפנה איסימטרית)
- גם אם מאזינים לתקשורת – הסיסמה לא ניתנת לפענוח בלי המפתח הפרטי.
- כלל המידע הרגיש מועבר בפורמט JSON, עם אמצעי זיהוי סיום (__END__), מה שמונע התקפות מסוג injection או שינוי מבנה ההודעה.
- שימוש ב־ "END" בסוף כל הודעה נועד לוודא שההודעה התקבלה בשלמותה – גם אם חולקה לחלקים (chunks) של 1024 בתים. השרת קורא בלולאה עד שהוא מזהה את הסוף, כך שלא תתבצע פענוח לפני שהתקבל כל המידע – מה שמונע קריאות חלקיות ושיבוש מבני.

● עומס על השרת - DDoS / DOS

איום:

שליחת מספר רב של בקשות לשרת עלולה לגרום לקריסתו או אי-זמינותו למשתמשים רגילים.

הפתרון:

- קיימת מערכת לניטור וניהול של ניסיונות התחברות לפי כתובת IP.
- כתובת שמבצעת מעל 10 ניסיונות התחברות שגויים – נחסמת זמנית.
- ניתן להרחיב זאת גם לניהול תעבורה ברמת firewall בעתיד.

● העלאת קבצים – ניתוח Hash

איום:

העלאת קבצים זדוניים או שינוי תוכן לאחר ההעלאה.

המצב בפרויקט:

- אין העלאת קבצים מצד המשתמש (למעט קריאת תמונת לוגו מקומית על ידי המערכת).

- במידה ויתווסף רכיב כזה בעתיד – ניתן יהיה לחשב Hash של הקובץ למשל (SHA-256) ולוודא אמינותו מול רשימה מאושרת.

שכבת התעבורה:

(Three-Way Handshake) לחיצת יד משולשת – TCP פרוטוקול

הסבר כללי:

הפרויקט מבוסס על פרוטוקול **TCP (Transmission Control Protocol)** שנועד להעברת מידע אמין ורציף בין הלקוח לשרת. דואג לכך שכל המידע יועבר בשלמותו, לפי הסדר הנכון, וללא אובדן מידע, גם במצבים של עומס או שיבושי רשת.

כדי להבטיח את אמינות הקשר TCP, משתמש במנגנון פתיחת חיבור הנקרא **"לחיצת יד משולשת"**

(Three-Way Handshake):

1. **SYN** – הלקוח שולח בקשה לפתיחת חיבור.
 2. **SYN-ACK** – השרת מאשר את הבקשה ומציע חיבור משלו.
 3. **ACK** – הלקוח מאשר את קבלת תשובת השרת.
- רק לאחר תהליך זה – מתחילה שליחת המידע בפועל.

Header של TCP – שדות חשובים:

בעת כל העברת מידע TCP, כולל כותרת (Header) המכילה שדות חיוניים הבאים:

- **Sequence Number** – מזהה את מספר הסדרה של החבילה, כדי להרכיב את המידע לפי הסדר הנכון.
- **Acknowledgment Number** – מציין איזו חבילה התקבלה בהצלחה, כהוכחה לכך שלא הייתה אובדן מידע.
- **Window Size** – גודל החלון שמציין כמה מידע ניתן לשלוח מבלי לחכות לאישור נוסף – תומך בזרימה חלקה של מידע.
- **Flags** – ביטים מיוחדים המציינים את מצב החיבור:

1. **SYN (Synchronize):** משמש לפתיחת חיבור בין הלקוח לשרת. נשלח בתחילת תהליך "לחיצת היד המשולשת" כדי לבקש התחברות.
2. **ACK (Acknowledgment):** מאשר שקיבלת בהצלחה מידע מהצד השני. כל הודעה לאחר פתיחת החיבור מכילה את הדגל הזה.
3. **FIN (Finish):** מציין שהשולח סיים לשלוח מידע – משמש לסגירה מסודרת של החיבור (כמו ניתוק תקין).

למה TCP נבחר לפרויקט זה?

- **אמינות גבוהה** – מבטיח שכל הודעה תגיע ליעד במלואה ובסדר הנכון.
- **אימות חיבור** – חיבור נוצר רק לאחר אימות הדדי, מה שמונע פתיחה אקראית של קשרים.
- **תמיכה בהודעות גדולות** – הפרוטוקול מחלק ומרכיב את המידע לחבילות לפי הצורך.

- **אבטחה בסיסית** – Flags – כמו FIN ו RST-מבטיחים סיום תקשורת מסודר ולא פתאומי.

• Subnet:

המערכת רצה בתוך רשת פנימית – (LAN) לדוגמה Subnet מסוג X.X.192.168.

- הדבר מאפשר הרצה על מספר מחשבים מבלי חשיפה מיידית לאינטרנט.
- החיבורים בין הלקוח לשרת מתבצעים לפי כתובת IP פנימית שמזוהה ב־ CMD (באמצעות ipconfig)
- בכך מצטמצם הסיכון לחשיפה חיצונית ומוקדם ניתוב רק של כתובות ידועות מראש.

• הצפנה: (Encryption)

יישום בפרויקט:

כל סיסמה נשלחת כשהיא **מוצפנת בעזרת RSA** מפתח ציבורי משותף.

ההצפנה היא **אסימטרית** – רק השרת שמחזיק את המפתח הפרטי (המפתח הפרטי שמור רק אצל השרת ולעולם אינו נחשף או נשלח) מסוגל לפענח את ההודעה.

מניעת MITM (Man-in-the-Middle) גם אם תוקף יאזין למידע, הוא לא יוכל לקרוא את הסיסמה.

שיטת RSA מאפשרת ללקוח להצפין מידע עם מפתח ציבורי שהוגדר מראש, כאשר רק השרת (שמחזיק במפתח הפרטי) מסוגל לפענח.

מפתחות נבחרים על בסיס שני מספרים ראשוניים גדולים, גודל המפתח נקבע ל־2048 ביט, והוא מספיק חזק לעמידה במתקפות ברוטליות במשך שנים רבות, כל עוד המפתח הפרטי נשמר בסוד בצד השרת בלבד, וכוללים תהליך של אוילר ו־ modular inverse מה שמבטיח שגם מחשבים חזקים לא יצליחו לפצח את ההצפנה בזמן סביר.

שכבת ההצפנה הנוכחית היא יישומית (application-level), אך ניתן להרחיב בעתיד עם הצפנת TLS / SSL לתקשורת כללית מאובטחת.

• הפעלת המערכת: (System Boot)

תהליך הפעלה – כולל אבטחה:

- השרת מופעל עם כתובת IP קבועה ופורט ידוע.
- בעת ההפעלה נבדק האם ה־IP-כבר משמש, ואם כן – המערכת מתריעה.
- בצד הלקוח, מתבצעת התחברות לשרת ופתיחת GUI.
- הגישה לשירותים מותנית באימות דו-שלבי.

חולשות וטיפולן-

● חולשת SQL Injection

תיאור:

מתקפה בה המשתמש מזין קלט זדוני למשל (OR 1=1 --) 'בשדה טופס, על מנת להריץ שאילתות לא מאובטחות על בסיס הנתונים.

הטיפול בפרויקט:

במערכת זו נעשה שימוש אך ורק בשאילתות פרמטריות (?) ולא בהרכבה דינמית של מחרוזות SQL. כך גם אם יוזן ערך זדוני – הוא יתפרש כערך ולא כחלק מהשאילתה.

● מתקפת MITM (Man-In-The-Middle)

תיאור:

תוקף יכול להאזין או לשנות את המידע שמועבר בין השרת ללקוח.

הטיפול בפרויקט:

רכיב רגיש כמו סיסמה מוצפן באופן חד-כיווני בעזרת RSA (המפתח הציבורי נמצא אצל הלקוח, והפרטי רק אצל השרת), כך שגם אם ההודעה יורטה – לא ניתן לפענח אותה.

● מתקפת Brute-force על סיסמאות

תיאור:

ניסיון לנחש סיסמה על ידי ניסוי שיטתי של סיסמאות רבות.

הטיפול בפרויקט:

בפרויקט מיושם מנגנון נעילה לפי כתובת IP לאחר חמישה ניסיונות התחברות שגויים, השרת חוסם זמנית את הכתובת למשך שעה.

● מתקפת DOS / DDoS

תיאור:

הצפת השרת בבקשות רבות כדי להפילו או לגרום לאיטיות קיצונית.

הטיפול בפרויקט:

בפרויקט הוכנס מנגנון לספירת בקשות לפי כתובת IP, ונעילה זמנית של לקוחות חריגים. בנוסף, משאבים כמו שליחת מייל נעשים בתהליכי רקע (thread), דבר זה מבטיח שהשרת יוכל להמשיך לטפל בבקשות נוספות בזמן שמיל נשלח – במקום "להיתקע" בזמן השליחה, שיכולה להימשך עד 3 שניות, כדי לא להעמיס על הלולאה הראשית של השרת.

● פגיעות כתוצאה מהעלאת קבצים

תיאור:

במערכות רבות קיימת אפשרות להעלות קבצים אשר עלולים להכיל קוד זדוני או תוכן לא רצוי.

הטיפול בפרויקט:

המערכת אינה כוללת שום אפשרות להעלאת קבצים מצד המשתמש, כך שהסיכון הזה אינו רלוונטי לפרויקט.

● הצפנה חלשה או לא עקבית

תיאור:

שימוש באלגוריתמים לא בטוחים, כמו MD5 או שימוש לא עקבי בהצפנה רק בחלק מהמערכת.

הטיפול בפרויקט:

- נעשה שימוש ב־RSA להצפנת הסיסמה לפני שליחתה לשרת.
- לאחר מכן, בשרת מתבצעת הצפנת bcrypt עם Salt אוטומטי (רכיב אקראי שמוסף לסיסמה לפני הצפנתה במאגר), מה שמקשה מאוד על פריצת הסיסמה גם במקרה של פריצה למסד הנתונים.
- bcrypt משתמש בפונקציית Hash איטית יחסית עם salt מובנה – דבר המקשה מאוד על מתקפות מילון או brute-force.
- כל סיסמה נשמרת עם salt ייחודי, כך שגם אם שתי סיסמאות זהות – הן ייראו שונות לחלוטין במסד.
- יתר המידע אינו רגיש ולכן אינו מוצפן – מה שמאזן בין אבטחה ליעילות.

הפרויקט נבנה עם התייחסות לא רק לשכבת האפליקציה אלא גם לשכבת התעבורה:

- שימוש ב־TCP לתקשורת אמינה ומובנית.
- הצפנת מידע רגיש (RSA).
- טיפול בחסימת תוקפים לפי IP.
- מניעת חולשות נפוצות כמו SQL injection או האזנת צד שלישי.

המערכת נבנתה לפי עקרונות ההגנה: Defense in Depth: כל שכבת מערכת (אפליקציה, תקשורת, בסיס נתונים) כוללת מנגנוני הגנה ייעודיים – כך שגם אם שכבה אחת תיפרץ, שאר המערכת תישאר מאובטחת. גישה זו שומרת על אבטחה גבוהה תוך שמירה על יעילות תפקודית.

מימוש הפרויקט פירוט מימוש המערכת (המשך שלב עיצוב)

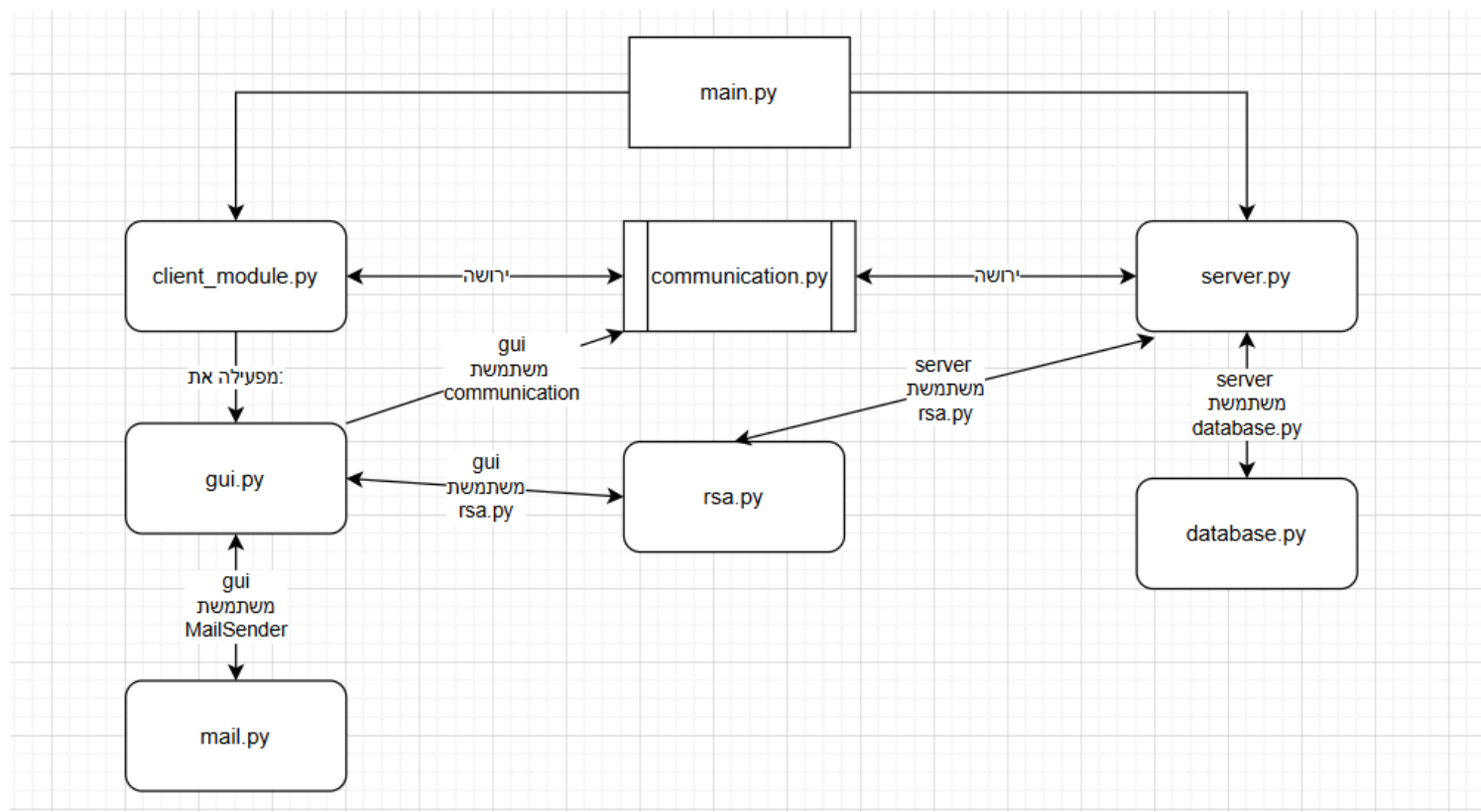
חלק א' - סקירת כל המודולים / מחלקות המרכיבים את המערכת וקשרי הגומלין ביניהם:

מודולים / מחלקות מיובאים:

שם המודול/מחלקה המיובא	ייעוד המודול / שורת הסבר
customtkinter	יצירת ממשק גרפי מתקדם ויפה יותר מ-Tkinter הרגיל.
tkinter.messagebox	הצגת הודעות קופצות למשתמש (שגיאות, אישור, מידע).
tkcalendar.Calendar	לוח שנה גרפי לבחירת תאריכים.
PIL.Image	טעינת תמונות להצגה (כגון לוגו) בממשק הגרפי.
threading	הפעלת תהליכים מקבילים (כמו שליחת מייל ברקע).
json	המרת נתונים בין פורמט JSON למילונים – לשליחת וקבלת נתונים מול השרת.
socket	ניהול תקשורת בין השרת ללקוח בפרוטוקול TCP.
sqlite3	גישה וניהול למסד הנתונים מסוג SQLite.
time	שימוש בהשהיות (sleep) לניהול תזמון פעולות כמו המתנה בין הפעלת שרת ללקוח, וכן למדידת זמני חסימה בשרת (למשל בהגנה מפני DDoS).
datetime	עבודה עם תאריכים ושעות (לצורך השוואה, בדיקה, תצוגה וכו').
random	הגרלות של מספרים אקראיים (למשל בקוד RSA ובשליחת קוד אימות).
bcrypt	הצפנת סיסמאות בצורה בטוחה במאגר הנתונים.
re	בדיקת תבניות – (regex) למשל אימות אימיילים או סיסמאות.
smtplib	שליחת מיילים דרך שרת (SMTP) משמש לשליחת קוד אימות ואישור וביטול הזמנה).
email.mime.text / multipart	בניית גוף הודעת האימייל בתצורה טקסטואלית.

מודולים / מחלקות שפיתחתי :

תרשים המתאר את המחלקות שיש ביניהן קשרים של ירושה או הכללה



Communication

תפקיד:

לטפל בתקשורת TCP בין השרת ללקוח – כולל התחברות, שליחה, קבלה וסגירה של חיבורים.

תכונות:

- **ip**
כתובת ה-IP אליה השרת מאזין או הלקוח מתחבר.
- **port**
מספר הפורט שאליו מתבצעת התקשורת.
- **is_server**
משתנה בוליאני שמציין האם מדובר בשרת (True) או לקוח (False).
- **socket**
אובייקט socket שמבצע את החיבור ודרכו מתבצעת התקשורת.

פעולות:

- **__init__(ip, port, is_server=False)**
טענת כניסה: כתובת IP, מספר פורט, דגל המזהה אם זה שרת או לקוח.
טענת יציאה: מאתחלת אובייקט socket, אם זה שרת – מאזין ומחכה לחיבורים.
- **connect()**
טענת כניסה: אין.
טענת יציאה: כלקוח – מתחבר לשרת לפי ה-IP והפורט שנקבעו.
- **accept_connection()**
טענת כניסה: אין.
טענת יציאה: כשרת – מקבל חיבור נכנס ומחזיר את ה-socket והכתובת של הלקוח. במקרה של כשל – מחזיר None.
- **send(message)**
טענת כניסה: מחרוזת (str) או מחרוזת מקודדת – ההודעה לשליחה.
טענת יציאה: שולחת את ההודעה דרך החיבור הפעיל. במקרה של כשל – סוגרת את החיבור.
- **receive()**
טענת כניסה: אין.
טענת יציאה: קוראת נתונים מהחיבור עד שמגיעה לסימון, "____END____" ומחזירה את המידע שקיבלה כמחרוזת. אם יש כשל – סוגרת את החיבור ומחזירה None.
- **close()**
טענת כניסה: אין.
טענת יציאה: סוגרת את החיבור ומאפסת את הסוקט.

Server

תפקיד:

מטפלת בכל הבקשות המתקבלות מהלקוח – התחברות, הרשמה, חיפוש מסעדות, הזמנות, צפייה בנתונים ועוד.
המחלקה יורשת מהמחלקה Communication את יכולות התקשורת הבסיסיות (שליחה, קבלה, התחברות וכו').

תכונות:

- **db**
אובייקט מסוג Database, דרכו מתבצעת הגישה למסד הנתונים.
- **rsa**
אובייקט מסוג RSA המשמש להצפנה/פענוח של סיסמאות ומפתחות ציבוריים/פרטיים.
- **ip_requests**
מילון שבו נשמרים הזמנים של בקשות שהתקבלו מכל כתובת IP – משמש להגנת DDoS.
- **failed_logins**
מילון שמכיל את כמות ניסיונות הכניסה הכושלים לפי כתובת IP.
- **blocked_ips**
כתובות IP שנחסמו זמנית עקב ניסיונות התחברות מרובים.
- **request_limit**
מספר הבקשות המקסימלי המותר בפרק זמן – להגנה על השרת (למשל 50).
- **time_window**
פרק הזמן שבו סופרים את הבקשות עבור הגנת DDoS (למשל 10 שניות).
- **failed_attempts_limit**
מספר ניסיונות התחברות שגויים מותר ל IP-לפני חסימה.
- **block_duration**
משך זמן (בשניות) לחסימת IP שנכשל שוב ושוב בהתחברות.

פעולות:

- **__init__(ip, port)**
טענת כניסה: כתובת IP ופורט.
טענת יציאה: קורא ל־ __init__, Communication יוצר את אובייקטי Database ו־ RSA ומאתחל את מערכות ההגנה.
- **start_server()**
טענת כניסה: אין.
טענת יציאה: מאזין לחיבורים נכנסים ובכל חיבור חדש מפעיל thread שמריץ את handle_client.
- **handle_client(client_socket)**
טענת כניסה: הפונקציה מקבלת אובייקט socket פתוח ומחובר של לקוח (Client) שדרכו מתבצעת התקשורת בין הלקוח לשרת.

- **טענת יציאה:** מטפלת בכל הבקשות של הלקוח/ אדמין – קוראת הודעות שהלקוח שולח, מפענחת את המידע (כולל פענוח RSA של סיסמאות אם נדרש), מפעילה את הלוגיקה המתאימה לפי הפעולה (action) מחזירה תגובה ללקוח, וממשיכה להאזין עד סיום החיבור או תקלה.

תיאור מפורט של תהליך הפעולה:

1. קבלת מידע מהלקוח:

הפונקציה מאזינה דרך `client_socket.recv()` ומקבלת את המידע שנשלח מהלקוח.
קריאת המידע מתבצעת בלולאה עד שמתקבל הסימן `__END__` שמסמן את סיום ההודעה.

2. פענוח ההודעה:

ההודעה מתקבלת כ־string מוצפן ומפוענחת ל־JSON.
אם יש שדה של סיסמה מוצפנת (כמו בהרשמה או התחברות) – הסיסמה מפוענחת באמצעות המפתח הפרטי של השרת. (RSA)

3. זיהוי הפעולה: (action)

הפונקציה בודקת את ערך השדה "action" (למשל: "login", "register", "search", "order" וכו').

4. הפניית הטיפול לפונקציה מתאימה:

לדוגמה:

אם `action == "login"` – בדיקת שם משתמש וסיסמה מול מסד הנתונים.
אם `action == "register"` – יצירת משתמש חדש במסד.
אם `action == "search"` → חיפוש מסעדות במסד לפי הקריטריונים שהתקבלו.
אם `action == "order"` → ביצוע ההזמנה ושמירתה במסד.
אם `action == "admin_view"` → שליפת נתונים מהמסד עבור אדמין.

5. שליחה חוזרת של תשובה ללקוח:

התשובה נשלחת תמיד בפורמט JSON, עם שדה "status" שמכיל ("success" או "error") ולפעמים גם מידע נוסף.
ההודעה נחתמת בסיומת `__END__` כדי לאותת ללקוח מתי ההודעה הסתיימה.

6. ניהול שגיאות:

הפונקציה משתמשת ב־try-except כדי לתפוס חריגות ולמנוע קריסת השרת במקרה של שגיאה בבקשה.
במקרה של שגיאה נשלחת תגובה מתאימה ללקוח עם "status": "error" והודעה מסבירה.

7. סגירת החיבור בסיום:

לאחר שהלקוח סיים לשלוח בקשות או במקרה של ניתוק, החיבור נסגר בעזרת `client_socket.close()`.

- **תפקידים עיקריים שממומשים בפונקציה:**
- ניהול תקשורת TCP עם לקוח בודד.
- ניתוח הודעות JSON והצפנה/פענוח לפי צורך.
- קריאה לפונקציות ייעודיות שמבצעות את הלוגיקה הרצויה.
- שליחת תשובה מתאימה ללקוח (כולל טיפול בחריגות).
- שמירה על ביצועים ויציבות – באמצעות חוטי ריצה (Thread) שמאפשרים שירות ברזמני ללקוחות רבים.
- **send_response(client_socket, response)**
טענת כניסה: socket של לקוח ומילון תגובה (dictionary)
טענת יציאה: שולח ללקוח את התגובה כ־ JSON עם הסימון "___END___"
- **check_ddos_protection(client_ip)**
טענת כניסה: כתובת IP של לקוח.
טענת יציאה: בודק האם מותר ל־IP לבצע בקשה נוספת בהתאם למגבלת הבקשות. מחזיר True אם מותר False אם לא.
- **handle_failed_login(client_ip)**
טענת כניסה: כתובת IP של לקוח.
טענת יציאה: מגדיל את מונה הכשלונות של ה־IP ובמידת הצורך חוסם אותו.
- **hash_password(password)**
טענת כניסה: סיסמה רגילה. (string)
טענת יציאה: מחזירה את הסיסמה כשהיא מוצפנת באמצעות bcrypt.
- **verify_password(password, hashed)**
טענת כניסה: סיסמה רגילה וסיסמה מוצפנת.
טענת יציאה: מחזירה True אם הסיסמאות תואמות אחרת False.

Client module

- תפקיד:** מייצגת את צד הלקוח – יוצרת חיבור לשרת, ומאתחלת את ממשק המשתמש הגרפי (GUI) דרכו המשתמשים מבצעים פעולות (כמו התחברות, חיפוש, הזמנה וכו').
- המחלקה יורשת מהמחלקה Communication**, ולכן מקבלת את כל יכולות התקשורת (connect, send, receive, close).

תכונות:

- **ip**
כתובת ה־IP של השרת.
- **port**
הפורט של השרת.

• gui

אובייקט מסוג GUI – ממשק המשתמש הגרפי שנטען לאחר ההתחברות.

פעולות:

- **__init__(ip, port)**
טענת כניסה: כתובת IP ופורט של השרת.
טענת יציאה: קורא ל־ `Communication.__init__`, יוצר חיבור לשרת (`connect`) ומפעיל את הממשק הגרפי ע"י `start_gui`.
- **start_gui()**
טענת כניסה: אין.
טענת יציאה: יוצר אובייקט מסוג GUI ומפעיל את הממשק באמצעות `mainloop`.

Database

תפקיד:

מטפלת בכל הפעולות מול מסד הנתונים: התחברות למסד, רישום משתמשים, אימות התחברות, חיפוש מסעדות, ביצוע הזמנות, ניהול מסעדות, צפייה בהזמנות ועוד.

תכונות:

- **db_name**
שם קובץ מסד הנתונים (ברירת מחדל: `'restaurant_database.db'`).
- **conn**
אובייקט החיבור למסד הנתונים (`sqlite3.Connection`).
- **cursor**
אובייקט להפעלת שאילתות SQL על מסד הנתונים (`sqlite3.Cursor`).

פעולות:

- **connect()**
טענת כניסה: אין.
טענת יציאה: פותחת חיבור למסד הנתונים ושומרת את ה־`cursor`.
- **authenticate_user(username)**
טענת כניסה: שם משתמש (`string`).
טענת יציאה: מחזירה את פרטי המשתמש אם קיים (`user_id, first_name, hashed_password`) אחרת `None` (`email`).
- **register_user(first_name, last_name, email, phone, username, password)**
טענת כניסה: פרטי משתמש כולל סיסמה מוצפנת.
טענת יציאה: מוסיף את המשתמש למסד. מחזיר `True` אם הצליח, `False` אם שם המשתמש או האימייל כבר קיימים.
- **search_restaurants(...)**
טענת כניסה: כל פרמטרי הסינון: גודל שולחן, שעה, תאריך, סוג אוכל, אזור, מחיר, כשרות, דירוג, קטגוריית ישיבה.
טענת יציאה: מחזיר רשימה של מסעדות זמינות התואמות לקריטריונים, אם לא נמצאה מסעדה בעלת מקום פנוי התואמת לקריטריונים מחזיר רשימה ריקה.

- find_tables(restaurant_name, category, table_size)**
טענת כניסה: שם המסעדה, קטגוריית ישיבה, גודל שולחן
טענת יציאה: מאתרת את כל השולחנות התואמים את הפרמטרים הנ"ל מתוך טבלת tables.
 מחזירה רשימה של שורות (tuples) שמייצגות שולחנות פנויים במסעדה עם קטגוריה וגודל תואמים.
- **get_restaurant_info(restaurant_name)**
טענת כניסה: שם מסעדה.
טענת יציאה: מחזיר את סוג האוכל, העיר והדירוג של המסעדה.
 - **order_a_table(restaurant, category, table_size, reservation_time, reservation_date, user_id)**
טענת כניסה: כל פרטי ההזמנה.
טענת יציאה: מבצע את ההזמנה בפועל ומחזיר את זמן הסיום המשוער לפי חישוב הטריגר.
 - **get_user_orders(user_id)**
טענת כניסה: מזהה משתמש.
טענת יציאה: מחזיר את ההזמנות העתידיות של המשתמש.
 - **cancel_order(order_id)**
טענת כניסה: מזהה ההזמנה.
טענת יציאה: מבטל את ההזמנה במסד הנתונים. מחזיר True אם הצליח.
 - **view_orders_by_date(date)**
טענת כניסה: תאריך בפורמט YYYY-MM-DD.
טענת יציאה: מחזיר את כל ההזמנות בתאריך זה (משמש באדמין).
 - **view_orders_by_user(user_name)**
טענת כניסה: שם משתמש.
טענת יציאה: מחזיר את ההזמנות של המשתמש לפי שמו. אם שם המשתמש לא קיים מחזיר None
 - **get_popular_restaurants()**
טענת כניסה: אין.
טענת יציאה: מחזיר את חמשת המסעדות עם הכי הרבה הזמנות.
 - **get_active_users()**
טענת כניסה: אין.
טענת יציאה: מחזיר את חמשת המשתמשים שביצעו הכי הרבה הזמנות.
 - **add_restaurant(data)**
טענת כניסה: מילון עם כל פרטי המסעדה להוספה.
טענת יציאה: מוסיף את המסעדה למסד הנתונים. מחזיר True אם הצליח, אחרת False.
 - **delete_restaurant(name)**
טענת כניסה: שם מסעדה.
טענת יציאה: מוחק את המסעדה וכל ההזמנות העתידיות אליה, בנוסף את רשימת ההזמנות שבוטלו, כולל: מזהה הזמנה, מזהה משתמש, תאריך, שעה וכתובת אימייל של המשתמש בכדי לעדכן את המשתמשים על ביטול הזמנתם. מחזיר True אם הצליח.
 - **add_table(info)**
טענת כניסה: מילון עם פרטי השולחן (מסעדה, גודל, קטגוריה, כמות).
טענת יציאה: מוסיף את השולחנות לטבלת השולחנות. מחזיר True אם הצליח.

- **remove_table(info)**
טענת כניסה: מילון עם פרטי השולחן להסרה.
טענת יציאה: מוחק את השולחנות לפי הנתונים. מחזיר True אם הצליח.

RSA

תפקיד:

ביצוע הצפנה ופענוח של סיסמאות וטקסטים באמצעות אלגוריתם RSA. משמש בעיקר להצפנת סיסמאות בלקוח לפני שליחתן לשרת, ולפענוח בצד השרת.

תכונות:

- **p, q**
שני מספרים ראשוניים שנבחרים אקראית מתוך רשימה קבועה לצורך יצירת המפתחות.
- **n**
מכפלת $p * q$ – משמשת גם להצפנה וגם לפענוח.
- **phi**
פונקציית אוילר של n : $(p-1)*(q-1)$ – נדרשת לחישוב המפתח הפרטי.
- **public_key**
זוג המספרים (e, n) – המפתח הציבורי להצפנת הודעות.
- **private_key**
זוג המספרים (d, n) – המפתח הפרטי לפענוח הודעות.

פעולות:

- **__init__(p=None, q=None)**
טענת כניסה: ניתן להעביר מספרים ראשוניים ידניים או לא להעביר כלל.
טענת יציאה: מגריל מספרים ראשוניים (אם לא הועברו), יוצר את n , phi , ומייצר את המפתחות $public_key$, $private_key$.
- **generate_random_primes()**
טענת כניסה: אין.
טענת יציאה: בוחר אקראית שני מספרים ראשוניים שונים מתוך רשימה קבועה ומחזיר אותם.
- **gcd(a, b)**
טענת כניסה: שני מספרים שלמים.
טענת יציאה: מחזיר את המחלק המשותף הגדול ביותר ביניהם (אלגוריתם אוקלידס).
- **multiplicative_inverse(e, phi)**
טענת כניסה: e – המפתח הציבורי – phi – פונקציית אוילר.
טענת יציאה: מחזיר את d – המפתח הפרטי (ההופכי של e מודולו phi).
- **generate_keypair()**
טענת כניסה: אין.
טענת יציאה: בוחר e ו- d שמקיימים את התנאים ומחזיר את הזוגות (e, n) , (d, n) .

- **encrypt(plaintext, key)**
טענת כניסה: טקסט להצפנה ומפתח (בד"כ public_key)
טענת יציאה: מחזיר רשימת מספרים – ההודעה כשהיא מוצפנת.
- **decrypt(ciphertext, key)**
טענת כניסה: רשימת מספרים מוצפנת ומפתח (בד"כ private_key)
טענת יציאה: מפענח ומחזיר את ההודעה המקורית (מחרוזת).
- **encrypt_as_str(plaintext, key)**
טענת כניסה: טקסט רגיל ומפתח.
טענת יציאה: כמו encrypt – אבל הפלט הוא מחרוזת אחת עם מספרים מופרדים בפסיקים.
- **decrypt_from_str(encrypted_str, key)**
טענת כניסה: מחרוזת מוצפנת שמכילה מספרים מופרדים בפסיקים, ומפתח.
טענת יציאה: מפענח את המחרוזת ומחזיר את הטקסט המקורי.

MailSender

תפקיד:

שליחת הודעות אימייל למשתמשים – כולל קוד אימות בעת התחברות, אישור הזמנה, והודעת ביטול הזמנה.

תכונות:

- **sender_email**
כתובת האימייל שממנה נשלחות ההודעות (לדוגמה "easy.ordermail1@gmail.com")
- **sender_password**
סיסמת אפליקציה (App Password) של Gmail – מאפשרת שליחת אימיילים דרך SMTP.
- **smtp_server**
כתובת שרת – SMTP לרוב. "smtp.gmail.com":
- **smtp_port**
פורט לשליחת דוא"ל מאובטח. (587)
- **last_code_sent**
הקוד האחרון שנשלח באימות דו-שלבי – נשמר בזיכרון לבדיקה מקומית.

פעולות:

- **generate_verification_code()**
טענת כניסה: אין.
טענת יציאה: מגריל קוד אימות באורך 8 ספרות (מחרוזת מספרית), שומר אותו במשתנה last_code_sent ומחזיר אותו.
- **send_verification_code(recipient_email)**
טענת כניסה: כתובת אימייל של המשתמש.
טענת יציאה: שולח מייל עם קוד אימות לכתובת. מחזיר את הקוד אם הצליח, או None אם נכשל.
- **send_reservation_confirmation(recipient_email, restaurant, city, date, start_time, end_time, people, seating_category)**

טענת כניסה: כל פרטי ההזמנה + כתובת אימייל של המקבל.
טענת יציאה: שולח אימייל עם פרטי ההזמנה המאושרת. מחזיר True אם הצליח False, אם נכשל.

- **send_cancel_confirmation(recipient_email, order)**
טענת כניסה: כתובת אימייל ואובייקט order (מילון) המכיל את פרטי ההזמנה שבוטלה על ידי הלקוח.

טענת יציאה: שולח מייל על ביטול הזמנה. מחזיר True אם הצליח, False אם נכשל.

- **send_cancel_restaurant_closed(recipient_email, order)**
טענת כניסה: כתובת אימייל ואובייקט order (מילון) המכיל את פרטי ההזמנה שבוטלה על ידי המסעדה.

טענת יציאה: שולח מייל על ביטול הזמנה. מחזיר True אם הצליח False, אם נכשל.

- **get_last_code_sent()**
טענת כניסה: אין.
טענת יציאה: מחזיר את הקוד האחרון שנשלח – לשם אימות בצד הלקוח (מקומית).

GUI

תפקיד:

ניהול ממשק המשתמש הגרפי של המערכת – כולל התחברות, הרשמה, אימות באמצעות מייל, חיפוש מסעדות, ביצוע הזמנות, ביטולים, צפייה בהזמנות, וניהול מסעדות ושולחנות על ידי אדמין.

תכונות:

- **client_comm**
תקשורת עם השרת – שליחת וקבלת בקשות ותגובות.
- **root**
חלון GUI ראשי (Tkinter).
- **rsa**
אובייקט להצפנת סיסמאות בטכנולוגיית RSA.
- **mail_sender**
אובייקט לשליחת הודעות אימייל (אימות, אישור, ביטול).
- **server_public_key**
המפתח הציבורי של השרת להצפנת RSA.
- **username, user_id, user_email, user_first_name**
פרטי המשתמש שהתחבר בהצלחה.
- **restaurants_mapping**
מיפוי בין תצוגת המסעדות לבין מזהים פנימיים.
- **selected_restaurant_var, seating_var, available_seating**
משתנים לאחסון בחירת מסעדה וקטגוריית ישיבה.
- **verification_code**
קוד אימות שנשלח למייל בעת התחברות.

- `date_entry, time_entry, people_entry, price_entry, food_listbox, districts_listbox, kosher_var, rating_var`
שדות טופס קלט להזמנה.

פעולות:

- `__init__(client_comm)`
טענת כניסה: אובייקט תקשורת לשרת.
טענת יציאה: מאתחל את הממשק, התקשורת, ההצפנה, וחלון ה-GUI.
- `get_server_public_key()`
טענת כניסה: אין.
טענת יציאה: מקבל את המפתח הציבורי של השרת ומאחסן אותו.
- `create_login_register_screen()`
טענת כניסה: אין.
טענת יציאה: מציג את מסך הפתיחה עם כפתורי התחברות והרשמה.
- `show_login_screen()`
טענת כניסה: אין.
טענת יציאה: מציג את טופס ההתחברות.
- `show_register_screen()`
טענת כניסה: אין.
טענת יציאה: מציג את טופס ההרשמה עם שדות הפרטים.
- `login()`
טענת כניסה: אין.
טענת יציאה: מצפין את הסיסמה, שולח לשרת, ואם הצליח – מציג חלון אימות.
- `send_code_thread()`
טענת כניסה: אין.
טענת יציאה: שולח קוד אימות למייל באמצעות Thread נפרד.
- `open_verification_window()`
טענת כניסה: אין.
טענת יציאה: פותח חלון להזנת קוד אימות שנשלח במייל.
- `verify_code()`
טענת כניסה: אין.
טענת יציאה: בודק אם הקוד שהוזן תואם לקוד שנשלח למייל.
- `resend_verification_code()`
טענת כניסה: אין.
טענת יציאה: שולח קוד אימות חדש למייל.
- `start_verification_timer()`
טענת כניסה: אין.
טענת יציאה: מציג טיימר של 120 שניות עד אפשרות לשליחה חוזרת.
- `register()`
טענת כניסה: אין.
טענת יציאה: מבצע בדיקות תקינות לשדות, חוזק סיסמה, ושולח את הנתונים לשרת.
- `evaluate_password_strength(password)`
טענת כניסה: מחרוזת סיסמה.
טענת יציאה: מחזיר את רמת חוזק הסיסמה "Strong", "Medium", או "Weak".

- **update_password_strength(event)**
טענת כניסה: אירוע Tkinter.
טענת יציאה: מעדכן את התצוגה הגרפית לפי חוזק הסיסמה.
- **is_valid_email(email)**
טענת כניסה: כתובת אימייל.
טענת יציאה: מחזיר True אם הפורמט תקין, אחרת False.
- **is_valid_date_format(date)**
טענת כניסה: מחרוזת תאריך.
טענת יציאה: בודק אם הפורמט תואם YYYY-MM-DD.
- **is_valid_time_format(time)**
טענת כניסה: מחרוזת שעה.
טענת יציאה: בודק אם הפורמט תואם HH:MM.
- **is_date_time_valid(date, time)**
טענת כניסה: תאריך ושעה.
טענת יציאה: בודק אם הם עתידיים ותקינים להזמנה.
- **back_to_login_register_screen()**
טענת כניסה: אין.
טענת יציאה: חוזר למסך הראשי של התחברות/הרשמה.
- **create_widgets()**
טענת כניסה: אין.
טענת יציאה: יוצר את המסך הראשי למשתמש המחובר.
- **_add_form_entry(parent, label, attr, font)**
טענת כניסה: קונטיינר, תווית, שם תכונה, גופן.
טענת יציאה: מוסיף שדה טופס קלט מותאם אישית.
- **_add_date_picker(parent, font)**
טענת כניסה: קונטיינר וגופן.
טענת יציאה: מוסיף רכיב בחירת תאריך עם לוח שנה.
- **show_user_orders()**
טענת כניסה: אין.
טענת יציאה: מציג את ההזמנות של המשתמש הנוכחי.
- **confirm_cancel_order(order)**
טענת כניסה: מילון פרטי הזמנה.
טענת יציאה: מציג חלון אישור לביטול ההזמנה.
- **send_cancelled_reservation_thread(order)**
טענת כניסה: מילון פרטי הזמנה.
טענת יציאה: שולח מייל ביטול בתוך Thread.
- **search_restaurant()**
טענת כניסה: אין.
טענת יציאה: שולח לשרת בקשת חיפוש מסעדות לפי קריטריונים.
- **show_restaurants(restaurants)**
טענת כניסה: רשימת מסעדות.
טענת יציאה: מציג את תוצאות החיפוש ברשימה.
- **on_restaurant_select()**
טענת כניסה: אין.
טענת יציאה: מזהה את המסעדה שנבחרה ברשימה ומציג מידע עליה.
- **seating_category()**
טענת כניסה: אין.
טענת יציאה: מחזיר את סוג הישיבה שנבחר להזמנה.

- **show_seating_preference_window()**
טענת כניסה: אין.
טענת יציאה: מציג חלון בחירה בין Indoor ו Outdoor.
- **show_order_table_button()**
טענת כניסה: אין.
טענת יציאה: מציג סיכום ההזמנה לפני שליחה.
- **show_order_confirmation_window()**
טענת כניסה: אין.
טענת יציאה: מציג חלון אישור על הזמנה שנשלחה ופרטים.
- **send_confirmed_reservation_thread(order)**
טענת כניסה: מילון פרטי ההזמנה.
טענת יציאה: שולח מייל אישור בתוך Thread נפרד.
- **open_admin_panel()**
טענת כניסה: אין.
טענת יציאה: פותח תפריט אדמין לצפייה בניתוח נתונים וניהול.
- **open_date_selection_window()**
טענת כניסה: אין.
טענת יציאה: פותח חלון לבחירת תאריך לצפייה בהזמנות.
- **show_orders_result_window(orders, date)**
טענת כניסה: הזמנות ותאריך.
טענת יציאה: מציג את ההזמנות לפי התאריך שהתקבל.
- **view_orders_by_user()**
טענת כניסה: אין.
טענת יציאה: מציג שדה להזנת שם משתמש לצפייה בהזמנות.
- **show_user_orders_result_window(orders, user_name)**
טענת כניסה: הזמנות ושם משתמש.
טענת יציאה: מציג את ההזמנות של אותו משתמש.
- **view_popular_restaurants()**
טענת כניסה: אין.
טענת יציאה: שולח בקשה לשרת לקבלת המסעדות הפופולריות.
- **show_popular_restaurants_window(restaurants)**
טענת כניסה: רשימת מסעדות.
טענת יציאה: מציג את המסעדות עם הכי הרבה הזמנות.
- **view_active_users()**
טענת כניסה: אין.
טענת יציאה: מבקש מהשרת את המשתמשים הכי פעילים.
- **show_active_users_result_window(users)**
טענת כניסה: רשימת משתמשים.
טענת יציאה: מציג את המשתמשים הפעילים ביותר.
- **edit_restaurants()**
טענת כניסה: אין.
טענת יציאה: פותח תפריט עריכת מסעדות (הוספה / מחיקה).
- **add_restaurant()**
טענת כניסה: אין.
טענת יציאה: מציג טופס להזנת פרטי מסעדה חדשה ושולח את הפרטים שהוזנו לשרת שיוסיף אותה.
- **delete_restaurant()**
טענת כניסה: אין.
טענת יציאה: מוחק מסעדה מהמערכת, כולל שליחת מייל ביטול לכל המשתמשים אשר יש להם הזמנות עתידיות לאותה המסעדה.

- `edit_tables()`
טענת כניסה: אין.
טענת יציאה: פותח תפריט עריכת שולחנות (הוספה / מחיקה).
- `add_tables()`
טענת כניסה: אין.
טענת יציאה: מציג טופס להוספת שולחנות למסעדה ושולח את הפרטים לשרת.
- `remove_tables()`
טענת כניסה: אין.
טענת יציאה: מציג טופס להסרת שולחנות קיימים ושולח את הפרטים לשרת.
- `close_connection()`
טענת כניסה: אין.
טענת יציאה: סוגר את התקשורת של הלקוח עם השרת.
- `close_application()`
טענת כניסה: אין.
טענת יציאה: סוגר את האפליקציה בצורה מסודרת.
- `run()`
טענת כניסה: אין.
טענת יציאה: מריץ את הלולאה הראשית של הממשק (mainloop).

חלק ב' - קטעי קוד ייחודי ואלגוריתמים מרכזיים בפרויקט:

נושא: שליחת מיילים בצורה מהירה ולא חוסמת (Threading)

יכולת: שליחת מייל לאישור או ביטול הזמנה – מבלי לעכב את ממשק המשתמש.
הסבר: שליחת מיילים דרך SMTP יכולה לקחת מספר שניות. כדי שלא לחסום את זרימת הקוד הראשי (במיוחד ב-GUI), מבצע את השליחה בתוך thread נפרד.

קטע קוד לדוגמה מתוך `gui.py`

```
def delete_restaurant(self): 1 usage
def on_submit():
    try:
        response_data = json.loads(response)
        if response_data.get("status") == "success":
            cancelled_orders = response_data.get("cancelled_orders", [])

            def send_cancellation_emails():
                self.mail_sender = MailSender()
                for order in cancelled_orders:
                    self.mail_sender.send_cancel_restaurant_closed(
                        recipient_email=order["email"],
                        order=order
                    )

            threading.Thread(target=send_cancellation_emails).start()
```

הסבר:

- `send_cancelled_reservation_thread` מופעלת לאחר ביטול הזמנה.
- השליחה מתבצעת דרך `Thread` חדש שמריץ את הפונקציה `send_email`.
- זה מאפשר לממשק להמשיך לפעול מבלי "להיתקע" או להמתין.

נושא: הצפנת סיסמאות כפולה (RSA → bcrypt)

יכולת: הגנה כפולה על סיסמאות המשתמש – גם בעת שליחה וגם בעת שמירה.

הסבר:

- בצד הלקוח, הסיסמה מוצפנת באמצעות RSA (מפתח ציבורי של השרת).
- בשרת, היא מפוענחת באמצעות המפתח הפרטי.
- לאחר מכן, הסיסמה **מוצפנת שוב** באמצעות `bcrypt` ונשמרת במסד הנתונים.

קטע קוד מתוך server.py:

```
def handle_client(self, client_socket): 1 usage
    self.send_response(client_socket, response)

    elif action == "register":
        first_name = data.get('first_name')
        last_name = data.get('last_name')
        email = data.get('email')
        phone = data.get('phone')
        username = data.get('username')
        encrypted_password = data.get('encrypted_password')

        password = self.rsa.decrypt_from_str(encrypted_password, self.rsa.private_key)
        hashed_password = self.hash_password(password)
        result = self.db.register_user(first_name, last_name, email, phone, username, hashed_password)
```

הסבר:

- השימוש בשתי שכבות הצפנה מונע פענוח הסיסמה – גם אם הנתונים נפרצים או "יורטו".

נושא: קבלת הודעות שלמות בתקשורת Socket

יכולת: קבלת הודעות ארוכות בצורה חלקית והשלמתן – עד לקבלת ההודעה כולה.

קטע קוד מ: communication.py

```
def receive(self): 17 usages (17 dynamic)
    """קבלת הודעה מלאה עם סימון סוף הודעה"""
    if self.socket:
        try:
            data = b""
            while True:
                part = self.socket.recv(1024)
                if not part:
                    break
                data += part
                if b"__END__" in part:
                    break
            return data.decode('utf-8').replace(__old: "__END__", __new: "")
        except Exception as e:
            print(f"Error receiving message: {e}")
            self.close()
    return None
```

הסבר:

- הודעות מגיעות במקטעים – כל מקטע בגודל 1024 בתים.
- הלולאה ממשיכה לקלוט חלקים עד שמופיעה הסיומת __END__.
- כך מובטחת קבלת ההודעה השלמה – גם אם היא נשלחה במספר חלקים.

נושא: חסימת IP לאחר ניסיונות התחברות כושלים

יכולת: ההגנה מפני brute-force על ידי מעקב אחרי IP.

קטע קוד מ: server.py

```
def handle_failed_login(self, client_ip: str): 2 usages
    # אם ה-IP לא נמצא בכשלונות, יתחיל עם 0
    if client_ip not in self.failed_logins:
        self.failed_logins[client_ip] = 0

    self.failed_logins[client_ip] += 1

    if self.failed_logins[client_ip] >= self.failed_attempts_limit:
        self.blocked_ips[client_ip] = time.time() # חוסם את ה-IP
        print(f"Blocked IP: {client_ip} due to too many failed login attempts")
```


הסבר:

- כל IP נשמר עם מונה ניסיונות וכמה זמן הוא חסום.
- אם חרג מהמכסה (10 ניסיונות למשל), הוא נחסם לשעה.
- כך נמנעת מתקפה ממוחשבת של ניסוי סיסמאות.

נושא: בדיקת זמינות שולחן לפי מסעדה, גודל, קטגוריה וזמן

יכולת: חיפוש שולחן פנוי במסעדה – בהתחשב בגודל השולחן, סוג הישיבה והתאריך והשעה.

קטע קוד מתוך database.py (search_restaurants):

```
def search_restaurants(self, table_size, time, date, food_types, districts, price, kosher, rating, category):
    # HH:MM פורמט במחזוריות משתמשים אנחנו השאלתה כאשר אנחנו משתמשים במחזוריות פורמט HH:MM
    reservation_query = """SELECT COUNT(*)
                            FROM reservations
                            WHERE table_id = ?
                            AND reservation_date = ?
                            AND ((reservation_time <= ? AND end_reservation > ?)
                                OR (reservation_time < ? AND end_reservation >= ?))"""
    self.cursor.execute(reservation_query,
                        (table_id, reservation_date, reservation_time_str, reservation_time_str,
                         reservation_end_time_str, reservation_end_time_str))
    reservation_count = self.cursor.fetchone()[0]

    # בדיקה אם יש מספיק מקומות פנויים לשולחן הזה
    if reservation_count < number_of_tables:
        is_restaurant_available = True
```

הסבר:

- מבצע הצטרפות בין מסעדות לשולחנות.
- מדלג על שולחנות שכבר הוזמנו באותו תאריך ושעה.
- מאפשר למצוא רק את המסעדות הרלוונטיות עבור ההזמנה הספציפית.
- מאוד קריטי למערכת – כדי שלא יהיו "התנגשויות" בין הזמנות.

נושא: שליחת הודעת אישור הזמנה יפה במייל

יכולת: שליחת מייל מעוצב עם פרטי הזמנה בשפה ברורה ואישית למשתמש.

קטע קוד מתוך mail.py:

```
def send_reservation_confirmation(self, recipient_email, restaurant, city, date, start_time, end_time, people,
                                seating_category):
    subject = "Reservation Confirmation - EasyOrder"
    body = (
        f"Hello,\n\n"
        f"Your reservation is confirmed!\n\n"
        f"📍 Restaurant: {restaurant} ({city})\n"
        f"📅 Date: {date}\n"
        f"🕒 Time: {start_time} - {end_time}\n"
        f"👥 Number of People: {people}\n"
        f"🪑 Seating: {seating_category}\n\n"
        f"Thank you for using EasyOrder!"
    )
```

הסבר:

- כל מייל כולל אייקונים, פרטי ההזמנה, תאריך, שעה, קטגוריית ישיבה ועוד.
- גורם למשתמש להרגיש ביטחון בכך שההזמנה התקבלה.
- המייל נשלח גם בביטול הזמנה – בגישה זהה.

חלק ג' - מסך בדיקות מלא:

הבדיקות שתוכננו בשלב האפיון:

בדיקה: שליחת מייל ביטול הזמנה לאחר מחיקת מסעדה (Admin)

מטרת הבדיקה: לוודא שכאשר מנהל מוחק מסעדה, כל ההזמנות העתידיות שלה נמחקות וכל המשתמשים מקבלים מייל ביטול.

מה בוצע בפועל: נוצרה מסעדת טסט עם מספר הזמנות עתידיות, ולאחר מכן המנהל מחק אותה דרך ממשק Admin.

תוצאת הבדיקה:

- ההזמנות העתידיות הוסרו ממסד הנתונים.
 - המשתמשים קיבלו מייל על ביטול ההזמנה.
- פתרון בעיות:** בתחילה לא נשלחו המיילים בגלל קריאה ישירה לפונקציה. זה תוקן ע"י הפעלת שליחת המיילים בתוך Thread ברקע.

בדיקה: טיפול בכתובת אימייל שגויה בשליחת קוד אימות

מטרת הבדיקה: לבדוק שהמערכת מתריעה כאשר לא ניתן לשלוח מייל לקוד אימות (למשל אם כתובת המייל לא תקינה).

מה בוצע בפועל: נרשם משתמש עם כתובת מייל שגויה ולא קיימת, ונעשה ניסיון לשלוח קוד אימות.

תוצאת הבדיקה: התקבלה שגיאת SMTP והוצגה הודעה מתאימה למשתמש.
פתרון בעיות: נוסף טיפול ב־ try/except והמערכת ממשיכה לפעול מבלי להיתקע.

בדיקה: הצפנת סיסמה כפולה RSA + bcrypt –

מטרת הבדיקה: לוודא שהסיסמה אינה נשמרת לא מוצפנת במאגר הנתונים במפורש או מועברת בטקסט קריא.
מה בוצע בפועל:

- הוזנה סיסמה במסך ההרשמה ונבדקה השליחה לשרת (מודגש: מועברת מוצפנת ב־RSA)
- בשרת בוצעה בדיקה שבמסד הנתונים הסיסמה שמורה ב־ bcrypt.
תוצאת הבדיקה:
- הסיסמה לא ניתנת לפענוח מהמסד.
- גם אם מאזינים לשידור, לא ניתן לקרוא את הסיסמה.
פתרון בעיות: נבדק גם מקרה של סיסמה ריקה או קצרה – נוספה בדיקת תקינות קלט בצד הלקוח.

בדיקות נוספות:

בדיקה: חיפוש מסעדה ללא מילוי שדות חובה

מטרת הבדיקה: לבדוק שהמערכת מתריעה כאשר המשתמש מנסה לבצע חיפוש בלי להזין תאריך, שעה ומספר סועדים.
מה בוצע בפועל: נלחץ כפתור "Search Restaurants" ללא מילוי שדות החובה.
מה היו תוצאות הבדיקה: הופיעה הודעת שגיאה מסוג `messagebox.showerror` עם הסבר מה חסר.
כיצד נפתרו בעיות: חלק מהשדות לא נבדקו בתחילה – הוספו תנאי בדיקה מוקדמים למניעת שליחת בקשת חיפוש לא תקינה לשרת.

בדיקה: ניסיון לרשום שם משתמש שכבר קיים

מטרת הבדיקה: לוודא שהמערכת חוסמת רישום כפול של אותו שם משתמש.
מה בוצע בפועל: נרשם משתמש חדש עם שם שכבר קיים במסד הנתונים.
מה היו תוצאות הבדיקה: התקבלה הודעת שגיאה מהשרת. "Username already exists":
כיצד נפתרו בעיות: לא נדרשו תיקונים – ההתנהגות תואמת לציפיות, אך חודדה ההודעה למשתמש להצגת הסיבה בצורה ברורה.

בדיקה: סגירה בטוחה של היישום והשרת

מטרת הבדיקה: לוודא שכאשר המשתמש סוגר את האפליקציה, התקשורת עם השרת נסגרת בצורה מסודרת ללא שגיאות או קריסות.

ימה בוצע בפועל: המשתמש לחץ על כפתור "close connection" של חלון ה-GUI לאחר התחברות פעילה.
ימה היו תוצאות הבדיקה: החיבור לשרת נותק, והשרת לא הציג חריגות או שגיאות.
יכיצד נפתרו בעיות: בהתחלה התקבלה חריגה (ConnectionResetError) בשרת – תוקן ע"י עטיפת קבלת הנתונים ב-try/except עם טיפול מותאם בניתוק לקוח.

בדיקה: טיפול בניתוק מהשרת במהלך פעולה

ימטרת הבדיקה: לבדוק שהמערכת לא קורסת אם השרת מתנתק או נסגר בזמן שהלקוח מנסה לשלוח בקשה.
ימה בוצע בפועל: הופעל הלקוח כרגיל, לאחר מכן השרת כובה באופן ידני. לאחר מכן נשלחה בקשה (למשל חיפוש מסעדה או התחברות).
ימה היו תוצאות הבדיקה: המשתמש קיבל הודעה מסוג Connection Error, והאפליקציה לא קרסה.
יכיצד נפתרו בעיות: בוצע טיפול בשגיאות תקשורת באמצעות try/except סביב כל שליחת בקשה (send) וקליטת תגובה, (receive) עם הצגת הודעה גרפית מתאימה למשתמש.

בדיקה: הצגת סיסמה חלשה בצורה גרפית בזמן אמת

ימטרת הבדיקה: לבדוק שהמשתמש מקבל חייווי חזותי על חוזק הסיסמה בזמן ההקלדה.
ימה בוצע בפועל: במסך ההרשמה הוזנו סיסמאות בדרגות שונות: קצרה, בלי אותיות גדולות, עם תווים מיוחדים וכו'.
ימה היו תוצאות הבדיקה: הצבע והטקסט ליד שדה הסיסמה התעדכנו לפי רמת חוזק (חלשה / בינונית / חזקה).
יכיצד נפתרו בעיות: בתחילה החיווי לא התעדכן עד ללחיצה על כפתור. תוקן באמצעות קישור של הפונקציה update_password_strength לאירוע KeyRelease.

בדיקה: שליחת הזמנה עם שעת סיום אוטומטית לפי משך ההזמנה

ימטרת הבדיקה: לבדוק שהמערכת מחשבת ומציגה למשתמש את שעת סיום ההזמנה בהתאם למשך ישיבה שהוגדר למסעדה.
ימה בוצע בפועל: בוצעה הזמנה במסעדה עם משך ישיבה של 90 דקות, ונבדק אם מופיעה גם שעת הסיום במסך האישור ובמייל.
ימה היו תוצאות הבדיקה: שעת הסיום חושבה אוטומטית ונכללה גם בחלון האישור וגם בגוף האימייל.
יכיצד נפתרו בעיות: בתחילה שעת הסיום לא הופיעה – הפונקציה עודכנה כך שתחבר בין שעת התחלה למשך ההזמנה לפי הנתון מהמסד.

מדריך למשתמש

פירוט כלל קבצי המערכת- עץ קבצים

client_module.py	—	# קוד לקוח ראשי
communication.py	—	# מחלקת תקשורת socket
database.py	—	# פעולות מול מסד נתונים (SQLite)
gui.py	—	# ממשק גרפי (GUI) של המשתמש
main.py	—	# קובץ הפעלה ראשי של המערכת
mail.py	—	# מחלקת שליחת מיילים (אימות, אישור הזמנה)
rsa.py	—	# הצפנת RSA
server.py	—	# קוד שרת מלא כולל בקרת גישה, חסימות וכו'
logo.jpeg	—	# קובץ לוגו לתצוגה ב-GUI
Database.db	—	# קובץ מסד נתונים

התקנת המערכת:

סביבה נדרשת:

- מערכת הפעלה: Windows 10/11
- גרסת פייתון: Python 3.10 ומעלה
- גישה לאינטרנט : חובה לשליחת מיילים דרך SMTP עם TLS

כלים נדרשים:

- סביבת פיתוח (IDE): PyCharm / VSCode
- DB Browser for SQLite : לצפייה במידע (לא חובה)

ספריית להתקנה:

- customtkinter
- bcrypt
- smtplib
- sqlite3
- Pillow
- tkcalendar

מיקומי קבצים :

- כל קבצי המערכת צריכים להימצא בתיקייה אחת משותפת.
- אין צורך בשרת – SQL נעשה שימוש ב־ SQLite מקומי.
- קובץ database.db ייוצר אוטומטית בריצה הראשונה.

נתונים התחלתיים :

- משתמש לדוגמה (אדמין) :
שם משתמש : admin
סיסמה : 0000
- משתמשים נוספים נרשמים דרך טופס ההרשמה ב-GUI

דרישות רשת :

- **הרצה מקומית או ברשת פנימית (LAN)**
הפעלת המערכת על אותו מחשב או בין כמה מחשבים בבית/בית ספר.
- **יש לוודא פתיחת פורט TCP (למשל : 12345)**
פורט = ערוץ תקשורת. חייב להיות פתוח כדי שהלקוח יוכל "לדבר" עם השרת.
- **כתובת IP של השרת נקבעת דרך ipconfig ומוזנת בצד הלקוח :**
ה IP-הוא כתובת המחשב של השרת. בודקים אותה ב־ CMD כדי שהלקוח יוכל להתחבר אליו.

ארכיטקטורה מינימלית נדרשת :

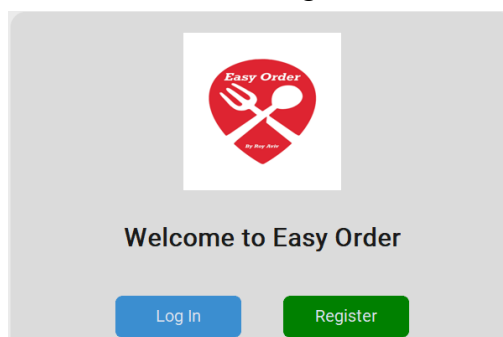
- **מחשב עם Python מותקן :**
נדרש שיהיה מותקן פייתון כדי להפעיל את הקבצים של הפרויקט.
- **לפחות 4 GB RAM (רצוי 8 GB לפיתוח חלק)**
RAM זה זיכרון שמריץ תוכנות. 4 גיגה זה מספיק להפעלה, אבל 8 גיגה מומלץ לפיתוח חלק.
- **חיבור יציב לאינטרנט :**
צריך אינטרנט כדי לשלוח מילים (למשל קוד אימות). לא חובה כל הזמן – רק בנקודות מסוימות.

משתמשי המערכת :משתמש קצה (משתמש רגיל)

1. **וודא התקנה תקינה :** ודא שכל קבצי הפרויקט נמצאים בתיקייה אחת, וכל הספריות הדרושות (כמפורט בסעיף "כלים נדרשים") מותקנות באמצעות pip
2. **הפעלת המערכת :** פתח את הקובץ main.py על ידי לחיצה כפולה או הפעלה מתוך סביבת פיתוח.
3. **תחילת עבודה :** עם ההפעלה, ייפתח מסך ראשי הכולל אפשרות להרשמה או התחברות למערכת.

מסכים עיקריים :

- **מסך כניסה :** מסך הכניסה לאפליקציה בו המשתמש יכול לבחור בין 2 אופציות : Log in במידה ורשום כבר לאתר ורוצה להיכנס, או register במידה והינו משתמש חדש אשר רוצה להירשם לאתר.



- **מסך הרשמה :** מסך בו המשתמש יכול להירשם לאפליקציה על ידי הזנת פרטים מזהים שם משתמש וסיסמה שחייבת להיות לפחות ברמת אבטחה בינונית (עונה על לפחות 2 מתוך 4 קריטריונים (אורך 8, אות קטנה, אות גדולה ותו מיוחד) אחרת לא יוכל להירשם).



Create Your Account

First Name:

Last Name:

Phone Number:

Email:

Username:

Password:

Confirm Password:

Password strength: Weak



Create Your Account

First Name:

Last Name:

Phone Number:

Email:

Username:

Password:

Confirm Password:

Password strength: Medium



Create Your Account

First Name:

Last Name:

Phone Number:

Email:

Username:

Password:

Confirm Password:

Password strength: Strong

☐ Input Error

Passwords do not match.

☐ Password Strength

Your password is too weak. Use uppercase, lowercase, symbols and at least 8 characters.

☐ Input Error

Please fill in all fields.

☐ Registration Successful

You have successfully registered!

- **מסך התחברות :** מסך בו המשתמש מזין את שם המשתמש והסיסמה שלו כדי להיכנס לאפליקציה.



Welcome Back!

Username:

Password:

☐ Login Error

Invalid username or password

- **מסך קוד אימות (אימות דו-שלבי):** אחרי ההתחברות תישלח הודעה למייל עם קוד אימות. יש להזין את הקוד כדי להיכנס.

Login Verified

Welcome back roy!

אישור

Two-Factor Authentication

Verification Required

A verification code was sent to roy.aviv25@gmail.com

Enter the 8-digit code:

22930504

Verify

Send Again

Code expires in: 00:46

Verification Failed

Incorrect verification code. Try again.

אישור

easy.ordermail1@gmail.com

אני

11:16 (לפני 21 דקות)

Hello!

Your verification code is: 25535491

- **מסך חיפוש מסעדות:** המשתמש מזין את הקריטריונים הרצויים (סוג אוכל, כשרות, תאריך, שעה וכו') ולוחץ על כפתור חיפוש. הקריטריונים היחידים שחובה להזין אותם הינם: תאריך, שעה וכמות סועדים.

Input Error

The selected date has already passed. Please choose a future date.

Choose a date

May 2025

Mon Tue Wed Thu Fri Sat Sun

18 28 29 30 1 2 3 4

19 5 6 7 8 9 10 11

20 12 13 14 15 16 17 18

21 19 20 21 22 23 24 25

22 26 27 28 29 30 31 1

23 2 3 4 5 6 7 8

Select

Input Error

You must enter the number of people, the time and the date.

Search Restaurants

Number of People (Required)

4

Time (HH:MM) (Required)

20:00

Select Date (Required)

2025-05-13

Open Calendar

Average Price per Person (Optional)

80

☒ **Kosher**

Rating (Optional)

3

Seating Preference:

☐ Indoor

☐ Outdoor

☒ Not specified

Food Type (Optional)

Italian

District (Optional)

Gush Dan

Input Error

Invalid time format. Please enter in HH:MM format.

Number of People (Required)

Time (HH:MM) (Required)

Select Date (Required)

Open Calendar

Average Price per Person (Optional)

☐ **Kosher**

Rating (Optional)

Seating Preference:

☐ Indoor

☐ Outdoor

☒ Not specified

Food Type (Optional)

No Preference

District (Optional)

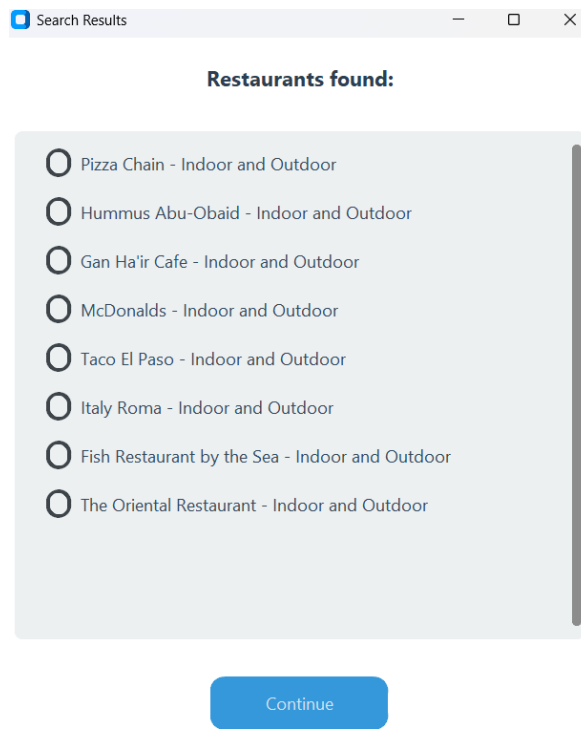
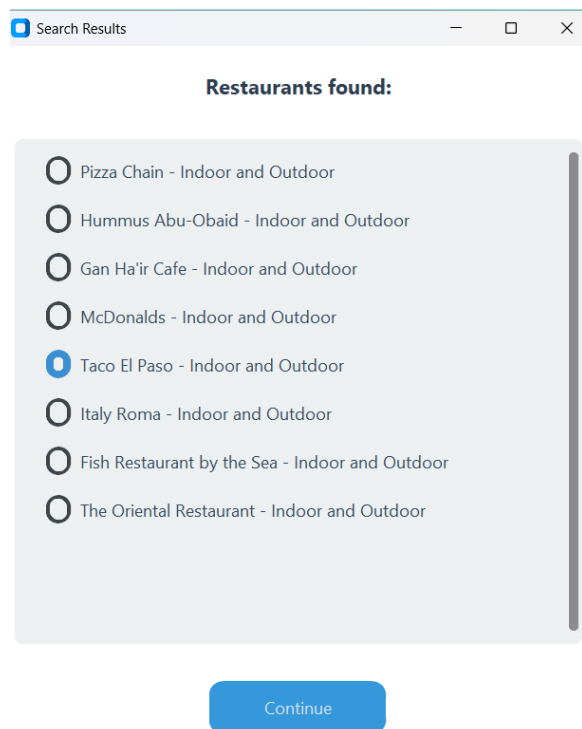
No Preference

View Your Orders

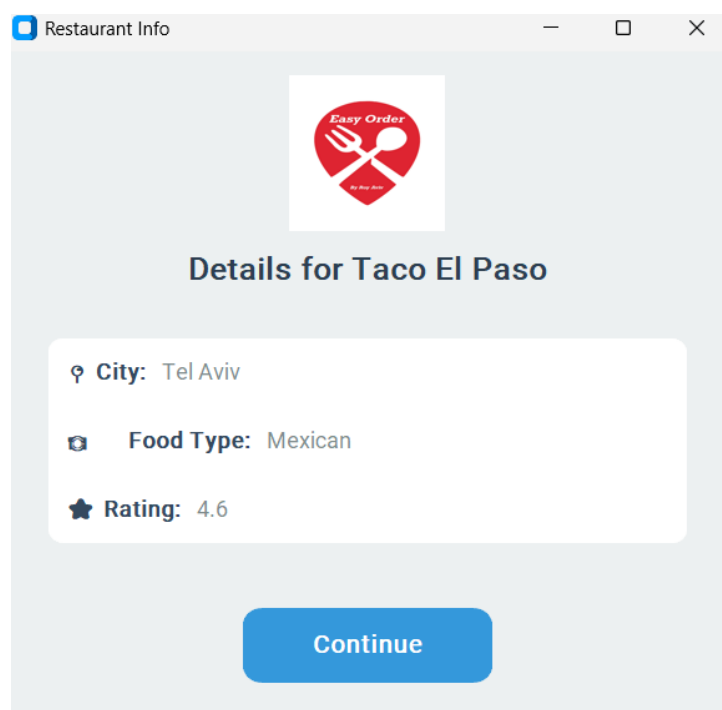
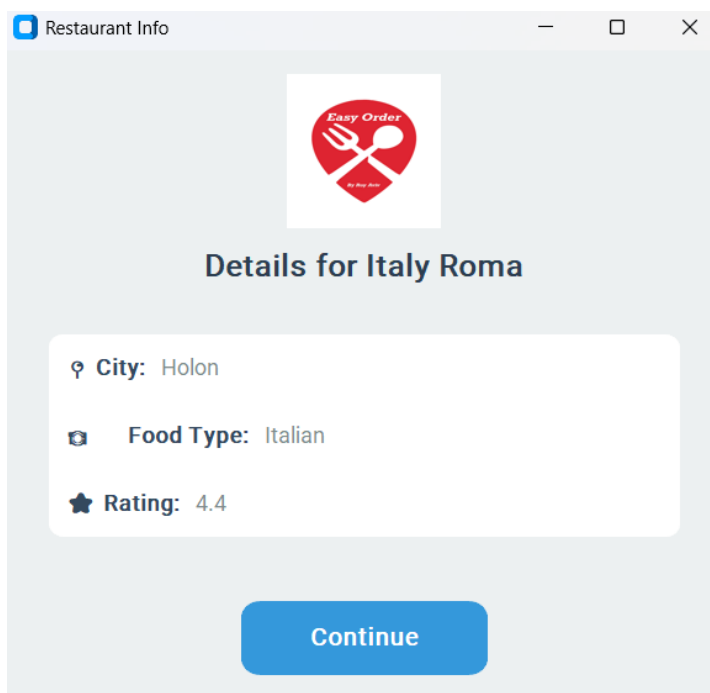
Close Connection

אישור

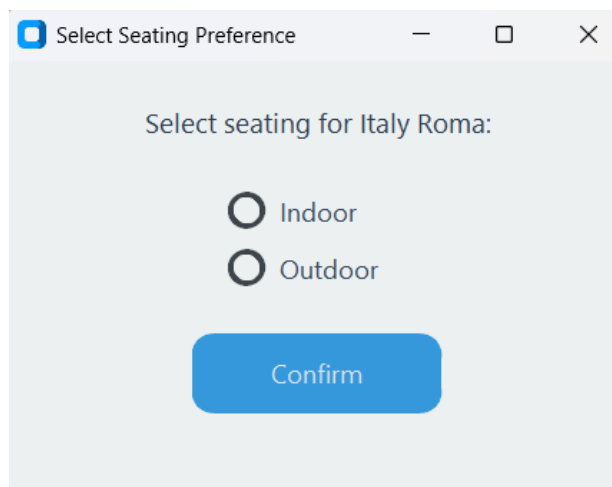
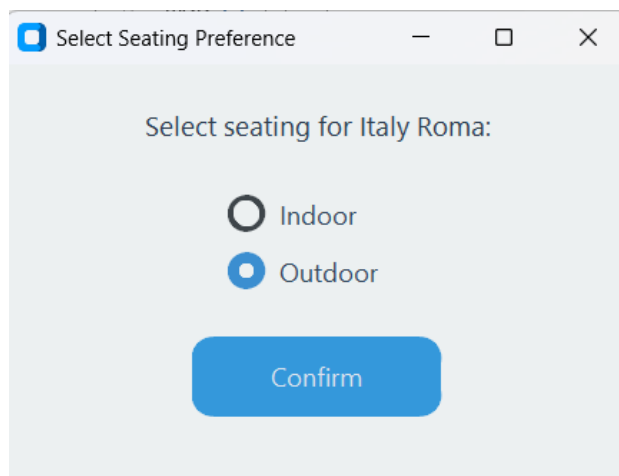
- **מסך תוצאות חיפוש:** מוצגות מסעדות מתאימות. לחיצה על אחת מהן תפתח פרטים נוספים עליה.



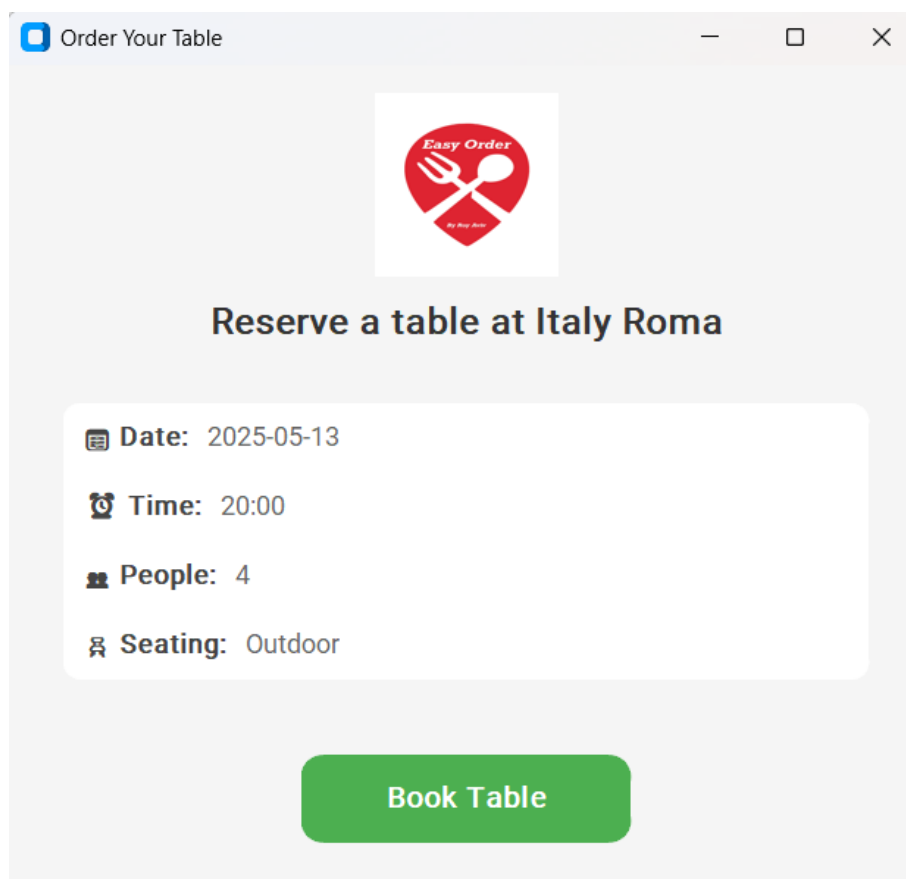
- **מסך פרטים על מסעדה:** מסך בו מוצגים פרטים על המסעדה שנבחרה.



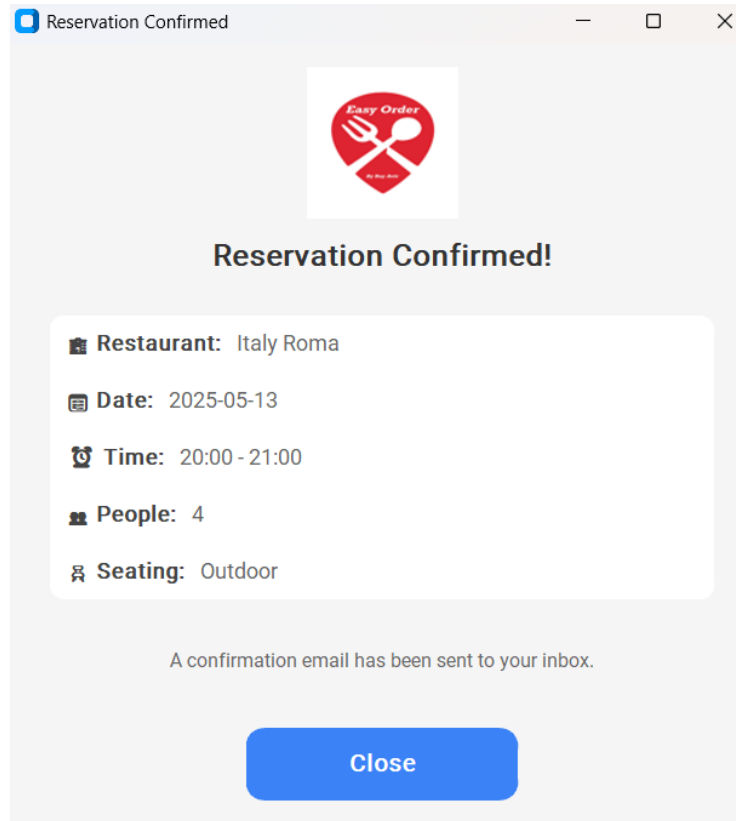
- **מסך בחירת מקום ישיבה (אם לא נבחר מראש):** במידה ויש אפשרות לבחירה – תוצג אפשרות לבחור בין ישיבה בפנים או בחוץ.



- **מסך סיכום והזמנה:** מוצגים פרטי ההזמנה וניתן ללחוץ על כפתור לביצוע ההזמנה באופן סופי.



- **מסך אישור הזמנה:** מסך בו הלקוח מקבל אישור סופי על ההזמנה כולל הודעה על כך שקיבל את כל פרטי ההזמנה במייל שלו.



Hello,

Your reservation is confirmed!

📍 Restaurant: Italy Roma (None)

📅 Date: 2025-05-13

🕒 Time: 20:00 - 21:00

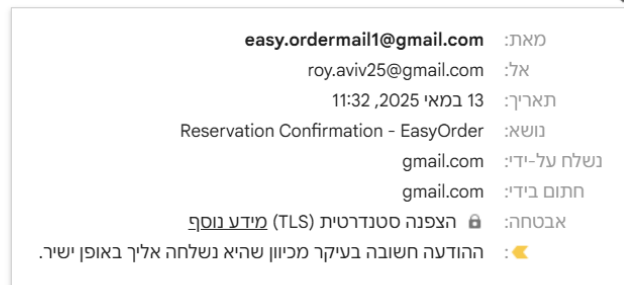
👥 Number of People: 4

🍽️ Seating: Outdoor

Thank you for using EasyOrder!

דואר נכנס x Reservation Confirmation - EasyOrder

easy.ordermail1@gmail.com



• מסך הזמנות שלי: המשתמש יכול לצפות בכל ההזמנות שביצע ולבטל עתידיות.

Easy Order - Restaurant Reservation

Your Orders

Restaurant: Pizza Chain
Date: 2025-05-04
Time: 20:00

Restaurant: Italy Roma
Date: 2025-05-13
Time: 20:00
[Cancel Reservation](#)

Restaurant: Italy Roma
Date: 2025-05-17
Time: 20:00
[Cancel Reservation](#)

Restaurant: Golden Sushi

[Back](#)

Easy Order - Restaurant Reservation

Your Orders

Restaurant: Yossi's Sushi
Date: 2025-03-02
Time: 18:00:00

Restaurant: Pizza Chain
Date: 2025-03-27
Time: 20:00

Restaurant: Yossi's Sushi
Date: 2025-04-10
Time: 20:30

Restaurant: Yossi Burger
Date: 2025-04-21
Time: 20:00

[Back](#)

Cancel Order

Are you sure you want to cancel your reservation at Italy Roma on 2025-05-13 at 20:00?

[כן](#) [לא](#)

Cancelled

Reservation cancelled successfully. A confirmation email was sent.

[אישור](#)

11:43 (לפני 0 דקות)

Hello,

Your reservation has been successfully cancelled.

Cancelled Reservation Details:

Restaurant: Italy Roma

Date: 2025-05-13

Time: 20:00

We're sorry to see you go. Hope to serve you again soon!

EasyOrder Team

Reservation Cancellation - EasyOrder

easy.ordermail1@gmail.com

מאת: easy.ordermail1@gmail.com

אל: roy.aviv25@gmail.com

תאריך: 13 במאי 2025, 11:43

נושא: Reservation Cancellation - EasyOrder

נשלח על-ידי: gmail.com

חתום ביד: gmail.com

אבטחה: הצפנה סטנדרטית (TLS) מידע נוסף

ההודעה חשובה בעיקר מכיוון שהיא נשלחה אליך באופן ישיר.

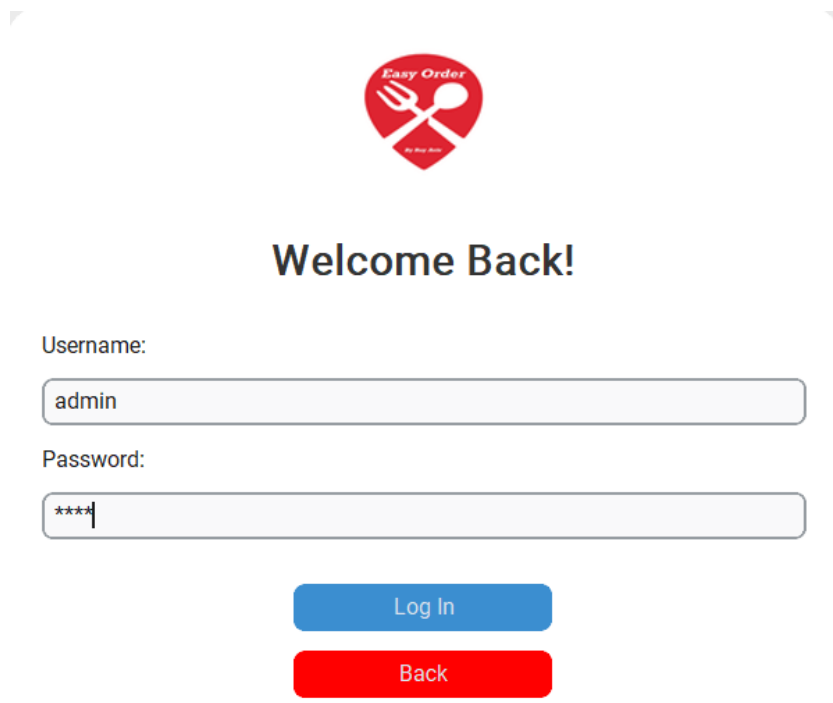
טיפים למשתמש:

- **אימות דו-שלבי (2FA):** הקוד שנשלח למייל תקף למשך **2 דקות בלבד**. במידה והקוד פג תוקפו או לא התקבל, ניתן לחוץ על כפתור "Send Again" לשליחה חוזרת.
- **בחירת סיסמה חזקה:** לצורך אבטחה, הסיסמה חייבת לעמוד **לפחות ב-2 מתוך 4 התנאים הבאים:**
 1. לפחות 8 תווים
 2. אות גדולה
 3. אות קטנה
 4. **תו מיוחד** (למשל %\$#@!)
סיסמה שלא עומדת בדרישות תידחה.
- **ביטול הזמנות:** ניתן לבטל רק **הזמנות עתידיות**. הזמנות מן העבר מוצגות לצורך תיעוד בלבד, ולא ניתנות לעריכה או מחיקה.
- **מניעת כפילויות בהזמנה:** אם תנסה להזמין לשעה בה **כבר אין מקומות פנויים**, תתקבל הודעת שגיאה. במקרים כאלה, כדאי לנסות שעה אחרת.
- **בחירת קטגוריית ישיבה:** במקרים בהם **לא נבחרה קטגוריה (פנים / חוץ)**, המערכת תבצע השלמה אוטומטית אם קיימת רק אפשרות אחת, או תפתח חלון ייעודי לבחירת העדפה.
- **שמור על חיבור לאינטרנט:** נדרש חיבור יציב לרשת בעת שליחת מיילים (למשל בעת התחברות או קבלת אישור הזמנה).
- **הזן פרטים מלאים:** המערכת לא תאפשר המשך פעולה אם אחד מהשדות החיוניים (שם משתמש, תאריך, שעה בחיפוש מסעדות או פרטים אישיים בתהליך ההרשמה) לא הוזן.
- **שימוש בפורמט נכון:** יש להזין את השעה בפורמט **HH:MM** ואת כמות האנשים כמספר שלם כדי להימנע משגיאות קלט.

מנהל מערכת (אדמין):

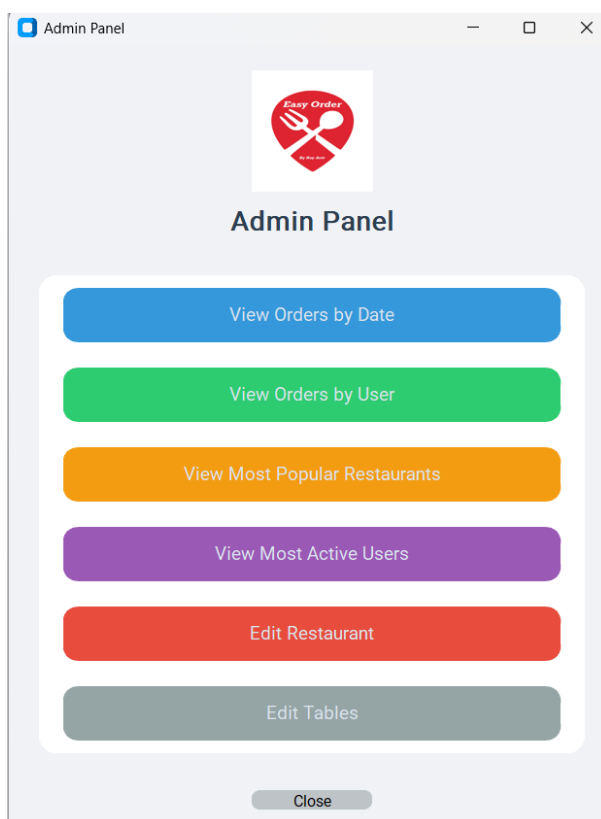
אופן הפעלת המערכת:

1. יש להתחבר עם שם המשתמש admin וסיסמה 0000 במסך ההתחברות.



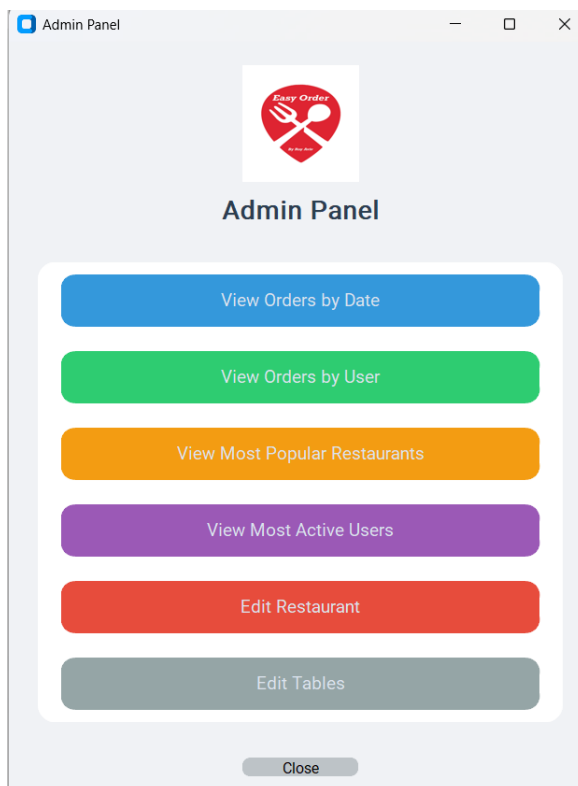
The login form features the 'Easy Order' logo at the top, which consists of a red shield with a white fork and spoon crossed inside. Below the logo, the text 'Welcome Back!' is displayed in a large, bold, black font. Underneath, there are two input fields: 'Username:' with the text 'admin' entered, and 'Password:' with four asterisks '****' entered. At the bottom of the form are two buttons: a blue 'Log In' button and a red 'Back' button.

2. לאחר לחיצה על log in תיפתח מערכת הניהול.

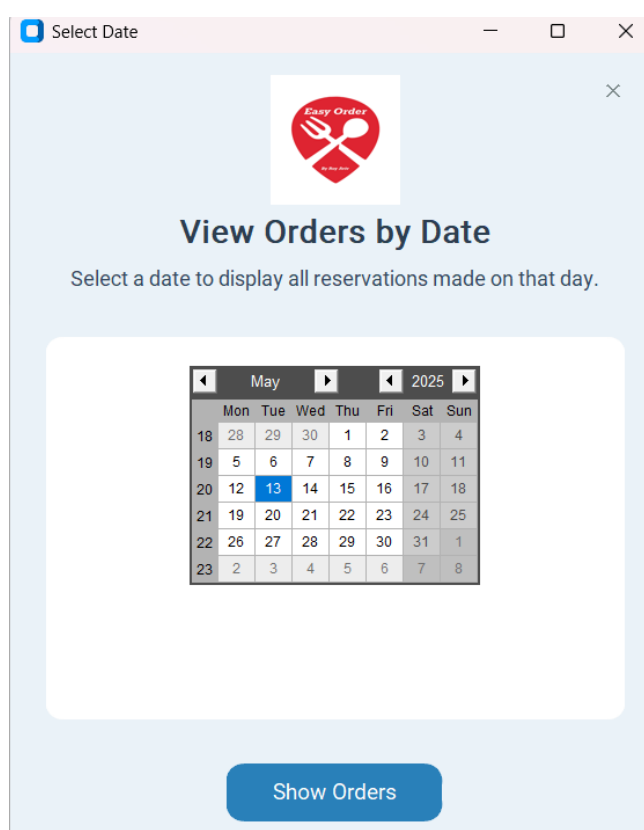
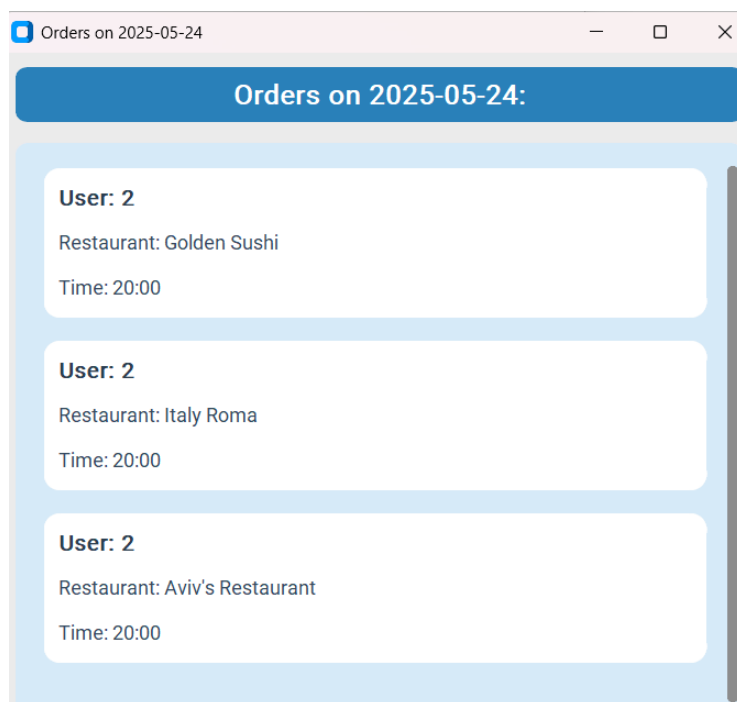


מסכים עיקריים:

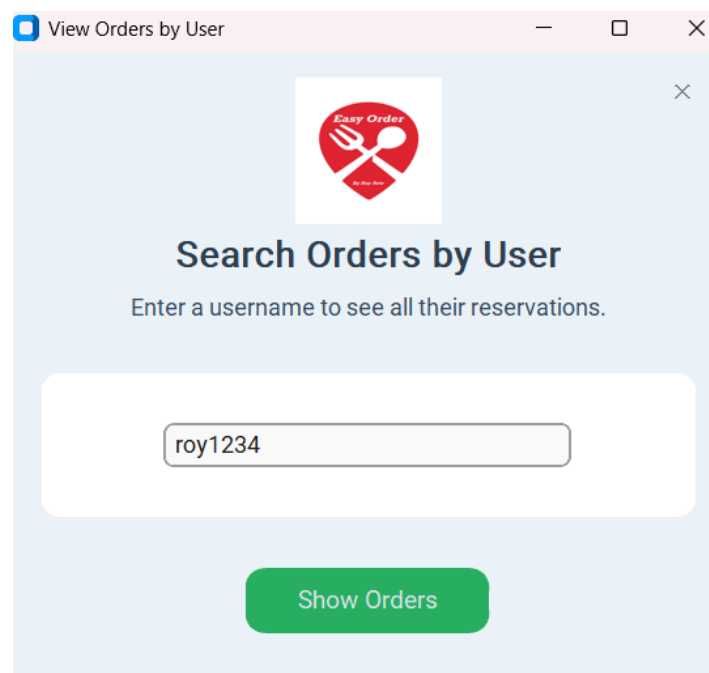
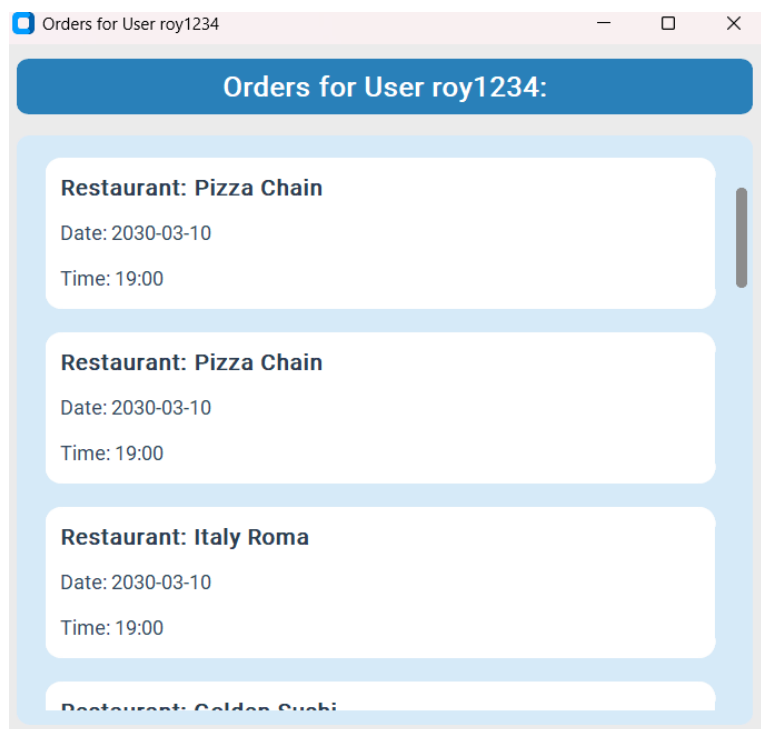
- מסך ראשי של האדמין: מציג כפתורים לביצוע פעולות ניהוליות.



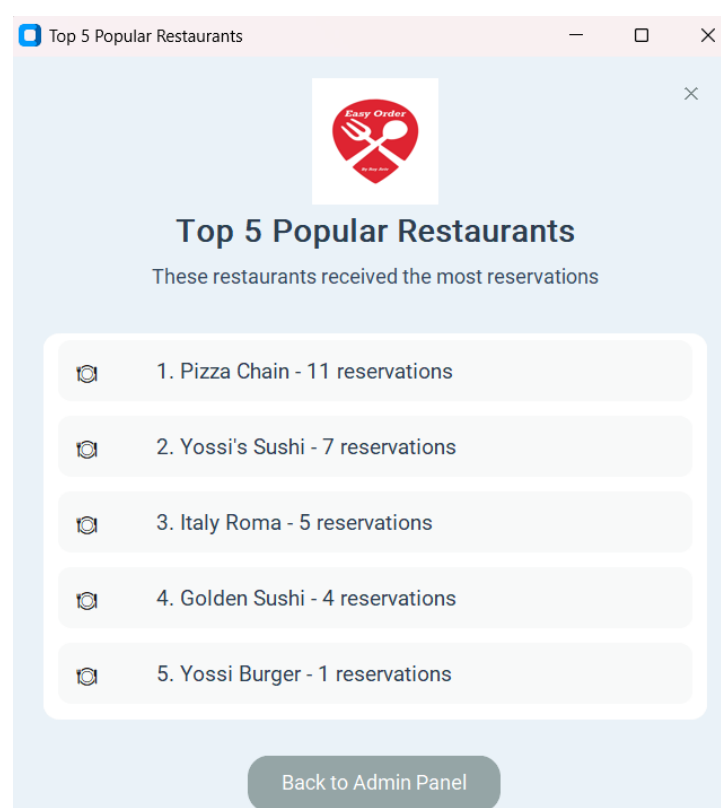
- צפייה בהזמנות לפי תאריך: האדמין בוחר תאריך ורואה את כל ההזמנות שנעשו ביום זה.



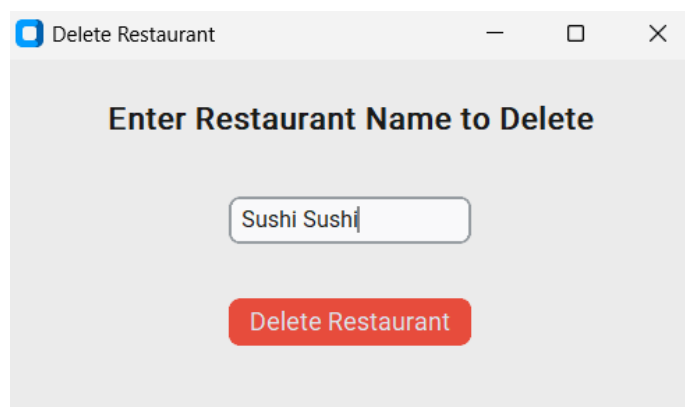
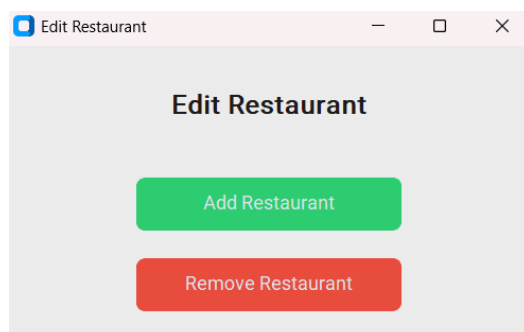
- צפייה בהזמנות לפי משתמש: ניתן להזין שם משתמש ולראות את ההזמנות שביצע.



- מסעדות פופולריות ומשתמשים הכי פעילים: חלונות אשר מציגים את חמשת המסעדות עם מספר ההזמנות הגבוה ביותר, ואת חמשת המשתמשים עם מספר ההזמנות הגבוה ביותר.

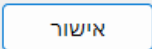


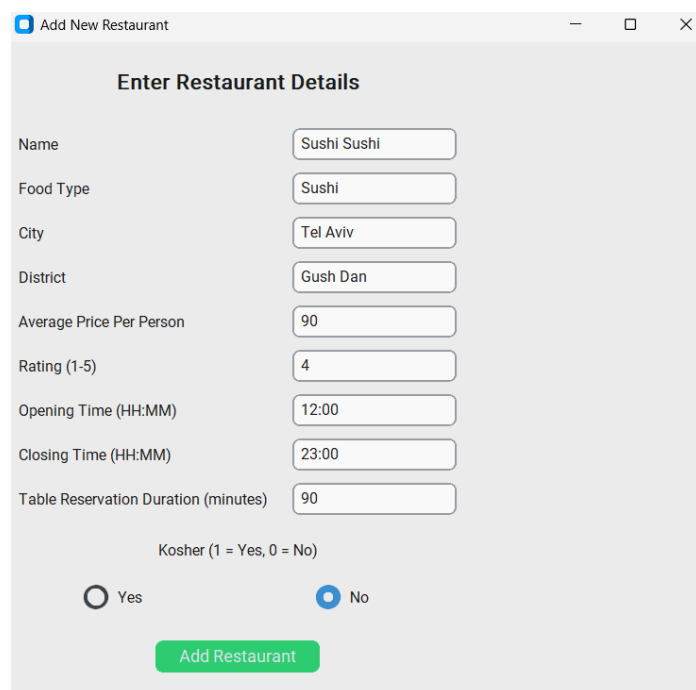
- הוספת/עריכת/מחיקת מסעדות: מאפשר לנהל את רשימת המסעדות במערכת.



 Success 

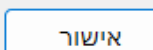
 Restaurant 'Sushi Sushi' and its future reservations deleted successfully.

 אישור

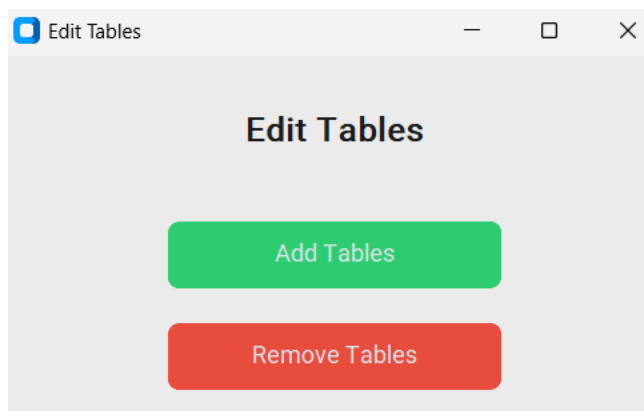
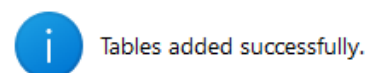
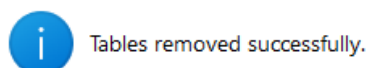
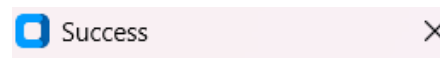


 Success 

 Restaurant added successfully.

 אישור

- ניהול שולחנות במסעדה: ניתן להוסיף ולהסיר שולחנות לכל מסעדה.

A window titled 'Remove Tables to Restaurant' with a light gray background. It contains the following fields: 'Restaurant Name' (text input with placeholder 'Restaurant Name'), 'Table Category' (radio buttons for 'Indoor' and 'Outdoor', with 'Indoor' selected), 'Table Size (Number of Seats)' (text input with placeholder 'e.g. 4'), 'Number of Tables' (text input with placeholder 'e.g. 3'), and a red 'Remove Tables' button at the bottom.A window titled 'Add Tables to Restaurant' with a light gray background. It contains the following fields: 'Restaurant Name' (text input with placeholder 'Pizza Chain'), 'Table Category' (radio buttons for 'Indoor' and 'Outdoor', with 'Indoor' selected), 'Table Size (Number of Seats)' (text input with placeholder '4'), 'Number of Tables' (text input with placeholder '3'), and a green 'Add Tables' button at the bottom.[אישור](#)[אישור](#)

סיכום אישי:

תהליך העבודה:

תהליך העבודה על הפרויקט היה מעניין, מאתגר וגם מהנה. קודם כל לקח לי הרבה זמן לחשוב על נושא מתאים לפרויקט ובחרתי בנושא זה בגלל חוויה אישית. אמא שלי ביקשה ממני להזמין מקום למסעדה בשישי בערב והתהליך למצוא מקום פנוי במסעדה כלשהי היה לי מעייף ומעצבן ולכן רציתי ליצור לזה פתרון.

התחלתי משלב אפיון של המערכת ואיך אני רואה שהיא תעבוד וגם תעמוד בדרישות הפרויקט. תחילה בניתי מאגר נתונים ויצרתי מסעדות מדומות כדי להפעיל את האפליקציה הדמוי מציאותית, ולאחר פיתחתי את התקשורת, ממשק המשתמש, ולבסוף לשכבת האבטחה והבדיקות. לאורך הדרך התמודדתי עם אתגרים טכניים כמו הצפנת סיסמאות באמצעות RSA שהיה לי קשה להבנה ומסובך בהתחלה, שליחת מיילים בצורה מאובטחת עם פרוטוקול חדש שהייתי צריך ללמוד (SMTP), ומניעת מתקפות כמו SQL Injection ו-DDoS שהייתי צריך ללמוד לעומק כדי להבין איך להגן על המערכת. כל אתגר חייב פתרון מותאם וחשיבה מעמיקה תוך התייעצות עם המורים וחיפוש עצמאי באינטרנט עד להצלחה.

תהליך הלמידה:

במהלך הפרויקט למדתי טכנולוגיות רבות שלא הכרתי לעומק קודם:

- עבודה עם **TCP Socket** ויצירת תקשורת Client-Server עצמאית בין כמה משתמשים שונים ממחשבים שונים.
- הצפנה בשני שלבים **RSA**: להעברת הסיסמה ו-**bcrypt** לאחסונה במאגר הנתונים.
- שליחת מיילים דרך **SMTP מאובטח עם TLS** כולל טיפול בחריגות.
- יצירת ממשק משתמש עם **CustomTkinter** ברמה מקצועית עם עיצוב יפה ונגיש לכולם.
- ביצוע בדיקות תוכנה (קופסה שחורה) וניהול מצבים חריגים.
- החיפוש אחר פתרונות, גם כשלא היו לי תשובות מיידיות, לימד אותי איך ללמוד לבד, לחפש מידע באינטרנט, ולהתייעץ כשצריך.

כלים שנלקחו להמשך:

אני יוצא מהפרויקט הזה עם כלים פרקטיים חשובים:

- איך בונים מערכת שלמה – לא רק הקוד אלא כל הדברים שמעבר התכנון (גם מלמעלה וגם תכנון ברמת הפרטים הקטנים ביותר) אבטחה, בדיקות רבות ושונות של סוגי קלטים שונים ואף מסוכנים והתמודדות איתם וממשק נוח ופשוט לכולם.
- איך מגינים על המערכת – מניהול הצפנה, אימות דו שלבי ועד חסימת תוקפים לפי IP
- עבודה עם מאגר נתונים רחב בעל טבלאות רבות מקושרות ביניהן ומנומרות.

תובנות:

- למדתי עד כמה חשוב לבדוק כל חלק במערכת – גם אם נראה "שוליי" ולא משמעותי.
- קיבלתי עזרה במידת הצורך – ממקורות באינטרנט, מצוות המורים ומהחברים לכיתה.
- שיתוף פעולה, גם אם מינימלי, עוזר לפתור בעיות מהר יותר.

בראייה לאחר:

- **תכנון מקדים מסודר יותר:** בתחילת הדרך התחלתי ישר בקידוד, מבלי להשקיע מספיק זמן באפיון מלא של המערכת ושל המבנה הפנימי של הקוד וכל הפיצ'רים השונים בו. אם הייתי מקצה זמן מסודר יותר לבניית דיאגרמות, חלוקה לשכבות ועיצוב – הייתי חוסך הרבה זמן וטעויות בהמשך.
- **שימוש מוקדם יותר בבדיקות ואימותים:** רק בשלבים מתקדמים יחסית התחלתי לבצע בדיקות שיטתיות על כל פעולה, ובדיעבד – הייתי מכניס את זה כבר בשלב הבנייה הראשוני, כדי לזהות בעיות מוקדם יותר.
- **ניסיון מוקדם יותר בטכנולוגיות חדשות:** לדוגמה, העבודה עם הצפנה ר' SMTP הייתה חדשה לי, ובהתחלה הובילה לעיכובים. אם הייתי לומד את הכלים האלו לעומק לפני תחילת המימוש – זה היה מייעל את הפיתוח.

שדרוגים עם משאבים נוספים:

אם היו ברשותי עוד זמן או משאבים, הייתי מוסיף:

- שימוש ב־ Machine Learning ללמידת העדפות המשתמש.
- אפשרות לכל משתמש במערכת ליצור לעצמו מסעדה שתהיה חלק מהמסעדות במאגר הנתונים.
- צ'ט בוט לעזרה למשתמשים במערכת.
- פיצ'ר של ביקורת על מסעדות שנעשות על ידי המשתמשים באתר.

שאלות חקר עצמי:

- איך הייתי יכול לשפר עוד יותר את אבטחת המידע במערכת?
- האם ממשק המשתמש מספיק ברור ונגיש לכל הגילאים?
- אילו תכונות נוספות היו הופכות את המערכת לשימושית יותר בעולם האמיתי?

תודות:

ברצוני להביע תודתה והערכה רבה **לצוות מורי מגמת הסייבר בתיכון אלון**, על הליווי הצמוד, התמיכה, ההכוונה וההשקעה הרבה לאורך כל שנותיי במגמה – ובעיקר במהלך השנה האחרונה בפרויקט.

צוות המורים סייע לי רבות – הן בהתמודדות עם אתגרים טכניים בקוד, והן במחשבה משותפת על רעיונות, שדרוגים ופיתוחים מיוחדים שהובילו את הפרויקט לרמה גבוהה ומקצועית.

תודה גדולה וייחודית **לעידן**, שמלווה אותי כבר מכיתה ט', ועמד לצידי במשך כל השנים בלמידת התחום. עידן הקדיש לי זמן אישי ומפגשים עקביים לאורך כל השנה – אחת לחודש או חודשיים – כדי לעבור איתי לעומק על שלבי הפרויקט, להציע רעיונות, לזהות בעיות ולשפר את הקוד. המפגשים האלו לא רק תרמו לתוצאה הטכנית, אלא גם העמיקו משמעותית את ההבנה שלי.

תודה רבה גם **לרום**, שלימד אותי מאז כיתה ח', על התמיכה והעזרה הרבה – ובעיקר בתחילת הדרך של הפרויקט, בשלב שבו הייתי עוד מבולבל ולא בטוח איך לגשת למשימה. רוֹם ישב איתי על מסד הנתונים, שהיה הבסיס לפרויקט כולו, ונתן לי כיוון ברור ונכון להתחלה.

תודה גם **לקובי ולעודי**, שלימדו אותי סייבר לראשונה השנה – על הידע שהעבירו לי ועל ההשפעה הישירה של הלמידה בכיתה על האופן שבו תכננתי ובניתי את הפרויקט.

תודה מיוחדת גם לליאת, שלמרות שלא לימדה אותי באופן ישיר, כמעט תמיד הייתה נוכחת בשיעורים שבהם עבדנו על הפרויקט, הציעה עזרה מיוזמתה, ותמיד הייתה זמינה לכל שאלה או צורך שעלה.

ולבסוף, תודה גדולה לגלית, שלימדה אותי כבר בכיתה ז'. אף על פי שכבר שנתיים איננה מלמדת אותי בפועל – היא תמיד מתעניינת, שואלת, מדריכה ותומכת. ההשפעה שלה על הדרך שלי בלמידת סייבר מורגשת עד היום.

ביבליוגרפיה

1. Wikipedia contributors. (n.d.). *RSA (cryptosystem)*. Wikipedia. Retrieved May 2025, from [https://en.wikipedia.org/wiki/RSA_\(cryptosystem\)](https://en.wikipedia.org/wiki/RSA_(cryptosystem))
2. Wikipedia contributors. (n.d.). *Bcrypt*. Wikipedia. Retrieved May 2025, from <https://en.wikipedia.org/wiki/Bcrypt>
3. Python Software Foundation. (n.d.). *random — Generate pseudo-random numbers*. Python documentation. Retrieved May 2025, from <https://docs.python.org/3/library/random.html>
4. GeeksforGeeks. (n.d.). *Python after() method in Tkinter*. Retrieved May 2025, from <https://www.geeksforgeeks.org/python-after-method-in-tkinter/>
5. Cloudflare. (n.d.). *What is a DDoS attack?*. Retrieved May 2025, from <https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>
6. OWASP Foundation. (n.d.). *Authentication Cheat Sheet*. Retrieved May 2025, from https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
7. Stack Overflow. (n.d.). *Python socket receive large amount of data*. Retrieved May 2025, from <https://stackoverflow.com/questions/17667903/python-socket-receive-large-amount-of-data>
8. Wikipedia contributors. (n.d.). *JSON*. Wikipedia. Retrieved May 2025, from <https://en.wikipedia.org/wiki/JSON>

Python Software Foundation. (n.d.). *socket — Low-level networking interface*. Python 9. documentation. Retrieved May 2025, from <https://docs.python.org/3/library/socket.html>

נספחים:

קוד הפרויקט:

Server:

```

from communication import Communication
from database import Database
import json
import threading
from rsa import RSA
import bcrypt
import time

class Server(Communication):
    def __init__(self, ip, port):
        super().__init__(ip, port, is_server=True)
        self.db = Database()
        self.rsa = RSA()

        self.ip_requests = { } # אוגרים את הזמן של כל בקשה
        self.failed_logins = { } # אוגרים את מספר הכשלונות
        self.blocked_ips = { } # אוגרים את ה-IPים החסומים
        self.request_limit = 50
        self.time_window = 10
        self.failed_attempts_limit = 10 # מספר כשלונות מותר לפני חסימה
        self.block_duration = 3600 # משך הזמן שכתובת ה-IP חסומה

    def send_response(self, client_socket, response):
        client_socket.sendall((json.dumps(response) + "__END__").encode('utf-8'))

    def check_ddos_protection(self, client_ip: str):
        if client_ip in self.blocked_ips:
            block_time = self.blocked_ips[client_ip]
            if time.time() - block_time < self.block_duration:
                return False # ה-IP עדיין חסום
            else:
                del self.blocked_ips[client_ip] # שחרור ה-IP מהחסימה אם עבר הזמן

        current_time = time.time()
        if client_ip in self.ip_requests:
            requests = self.ip_requests[client_ip]
            self.ip_requests[client_ip] = [timestamp for timestamp in requests if

```

```

        current_time - timestamp < self.time_window]
        if len(self.ip_requests[client_ip]) >= self.request_limit:
עבר את המגבלה        return False #
        else:
            self.ip_requests[client_ip] = []

מוסיף את הזמן הנוכחי למילון #
        self.ip_requests[client_ip].append(current_time)
        return True

def handle_failed_login(self, client_ip: str):
אם ה IP לא נמצא בכשלוניות, יתחיל עם #
        if client_ip not in self.failed_logins:
            self.failed_logins[client_ip] = 0

        self.failed_logins[client_ip] += 1

        if self.failed_logins[client_ip] >= self.failed_attempts_limit:
חוסם את ה IP        self.blocked_ips[client_ip] = time.time() #
            print(f"Blocked IP: {client_ip} due to too many failed login attempts")

def hash_password(self, password: str) -> str:
        salt = bcrypt.gensalt()
        hashed = bcrypt.hashpw(password.encode('utf-8'), salt)
        return hashed.decode('utf-8')

def verify_password(self, password: str, hashed: str) -> bool:
        return bcrypt.checkpw(password.encode('utf-8'), hashed.encode('utf-8'))

def start_server(self):
        print("Server is listening for connections...")
        while True:
            client_socket, addr = self.accept_connection()
            if client_socket:
יצירת Thread חדש לכל לקוח        #
                threading.Thread(target=self.handle_client, args=(client_socket,)).start()

def handle_client(self, client_socket):
שולפים את כתובת ה IP של הלקוח        client_ip = client_socket.getpeername()[0] #

בדיקת הגנה על DDoS לפני כל בקשה        #
        if not self.check_ddos_protection(client_ip):
            response = {
                "status": "error",
                "message": "Too many requests. Please try again later."
            }
            self.send_response(client_socket, response)
            client_socket.close()
        return

```

```

try:
    התחברות למסד נתונים ברגע שצריך לטפל בבקשה #
    if not self.db.cursor:
        self.db.connect()

    while True:
        request = client_socket.recv(1024)
        if not request:
            print("Client disconnected.")
            break

    try:
        פענוח הבקשה מלקוח data = json.loads(request.decode('utf-8')) #
        action = data.get('action')

        if action == "get_public_key":
            e, n = self.rsa.public_key
            response = {
                "status": "success",
                "public_key": {
                    "e": e,
                    "n": n
                }
            }
            self.send_response(client_socket, response)

        elif action == "search":
            num_people = data.get('table_size')
            reservation_time = data.get('time')
            date = data.get('date')
            food_types = data.get('food_types')
            districts = data.get('districts')
            price = data.get('price')
            kosher = data.get('kosher')
            rating = data.get('rating')
            category = data.get('category')

            קריאה לפונקציה לחפש מסעדות #
            response = self.db.search_restaurants(
                num_people, reservation_time, date, food_types, districts,
                price, kosher, rating, category
            )
            print(f"Received search request: {data}")

        -JSON קידוד התשובה כ #
            self.send_response(client_socket, response)

        elif action == "login":
            username = data.get('username')
            encrypted_password = data.get('encrypted_password')
            password = self.rsa.decrypt_from_str(encrypted_password, self.rsa.private_key)

```



```
user_info= self.db.authenticate_user(username)
if user_info:
    user_id, first_name, hashed_password, email = user_info
    if self.verify_password(password, hashed_password):
        response = {
            "status": "success",
            "message": "Login successful",
            "user_id": user_id,
            "first_name": first_name,
            "email_address": email
        }
    else:
        self.handle_failed_login(client_ip)
        response = {
            "status": "error",
            "message": "Invalid username or password"
        }
    else:
        self.handle_failed_login(client_ip)
        response = {
            "status": "error",
            "message": "Invalid username or password"
        }

self.send_response(client_socket, response)

elif action == "register":
    first_name = data.get('first_name')
    last_name = data.get('last_name')
    email = data.get('email')
    phone = data.get('phone')
    username = data.get('username')
    encrypted_password = data.get('encrypted_password')

    password = self.rsa.decrypt_from_str(encrypted_password, self.rsa.private_key)
    hashed_password = self.hash_password(password)
    result = self.db.register_user(first_name, last_name, email, phone, username,
hashed_password)
    if result:
        response = {"status": "success", "message": "Registration successful"}
    else:
        response = {"status": "error", "message": "Username or email already exists"}

    self.send_response(client_socket, response)

elif action == "restaurant_info":
    restaurant = data.get('restaurant')
    rest_info = self.db.get_restaurant_info(restaurant)
```

```

    if rest_info:
        food_type, city, rating = rest_info
        response = {
            "status": "success",
            "food_type": food_type,
            "city": city,
            "rating": rating
        }
    else:
        response = {
            "status": "error",
        }

    self.send_response(client_socket, response)

elif action == "order":
    restaurant = data.get('restaurant')
    category = data.get('category')
    table_size = data.get('table_size')
    reservation_time = data.get('reservation_time')
    reservation_date = data.get('date')
    user_id = data.get('user_id')

    order_response = self.db.order_a_table(
        restaurant, category, table_size, reservation_time, reservation_date, user_id)

    if order_response:
        response = {
            "status": "success",
            "message": "Order placed successfully",
            "end_reservation": order_response
        }
    else:
        response = {
            "status": "error",
            "message": "Error placing order"
        }

    self.send_response(client_socket, response)

elif action == "admin_view_orders_by_date":
    date = data.get("date")
    reservation_on_date = self.db.view_orders_by_date(date)
    if reservation_on_date:
        orders = []
        for order in reservation_on_date:
            user_id, reservation_time, restaurant_name = order #
            orders.append({
                "user_id": user_id,

```

```

        "reservation_time": reservation_time,
        "restaurant_name": restaurant_name
    })
    response = {
        "status": "success",
        "orders": orders # מחזירים את כל ההזמנות שנמצאו
    }
else:
    response = {
        "status": "error",
        "message": "No reservations found for this date."
    }
self.send_response(client_socket, response)

elif action == "admin_view_orders_by_user":
    user_name = data.get("user_name")
    reservations_by_user = self.db.view_orders_by_user(user_name)

    if reservations_by_user is None:
        response = {
            "status": "error",
            "message": "User not found."
        }
    elif not reservations_by_user:
        response = {
            "status": "success",
            "orders": []
        }
    else:
        orders = []
        for order in reservations_by_user:
            reservation_id, reservation_date, reservation_time, restaurant_name = order
            orders.append({
                "reservation_id": reservation_id,
                "reservation_date": reservation_date,
                "reservation_time": reservation_time,
                "restaurant_name": restaurant_name
            })
        response = {
            "status": "success",
            "orders": orders
        }

    self.send_response(client_socket, response)

elif action == "cancel_reservation":
    reservation_id = data.get("reservation_id")
    success = self.db.cancel_reservation(reservation_id)

```

```
if success:
    response = {
        "status": "success",
        "message": "Reservation cancelled successfully."
    }
else:
    response = {
        "status": "error",
        "message": "Reservation not found or could not be cancelled."
    }
self.send_response(client_socket, response)

elif action == "admin_view_popular_restaurants":
    most_popular_rest = self.db.get_most_popular_restaurant()
    if most_popular_rest:
        response = {
            "status": "success",
            "restaurants": [
                {"name": row[0], "reservation_count": row[1]}
                for row in most_popular_rest
            ]
        }
    else:
        response = {
            "status": "error",
            "message": "No restaurant data found."
        }
    self.send_response(client_socket, response)

elif action == "admin_view_active_users":
    most_popular_users = self.db.get_most_popular_users()
    if most_popular_users:
        users = []
        for user_name, reservation_count in most_popular_users:
            users.append({
                "user_name": user_name,
                "reservation_count": reservation_count
            })
        response = {
            "status": "success",
            "users": users
        }
    else:
        response = {
            "status": "error",
            "message": "No users found."
        }
    self.send_response(client_socket, response)

elif action == "admin_add_restaurant":
```

```

        restaurant = data.get("restaurant")
הנתונים שהתקבלו מהלקוח) כל פרמטר בנפרד) #
        name = restaurant.get("name")
        food_type = restaurant.get("food_type")
        opening_time = restaurant.get("opening_time")
        closing_time = restaurant.get("closing_time")
        table_reservation_duration = restaurant.get("table_reservation_duration")
        city = restaurant.get("city")
        district = restaurant.get("district")
        average_price_per_person = restaurant.get("average_price_per_person")
        kosher = restaurant.get("kosher")
        rating = restaurant.get("rating")

        response = self.db.add_restaurant(name, food_type, opening_time, closing_time,
                                           table_reservation_duration, city, district,
                                           average_price_per_person, kosher, rating)
שליחת התגובה ללקוח #
        self.send_response(client_socket, response)

elif action == "admin_delete_restaurant":
    name = data.get("restaurant_name")
    if name:
        success, message, cancelled_orders = self.db.delete_restaurant(name)
        response = {
            "status": "success" if success else "error",
            "message": message,
            "cancelled_orders": []
        }
    if success:
        for order in cancelled_orders:
            reservation_id, user_id, date, time, email = order
            response["cancelled_orders"].append({
                "restaurant_name": name,
                "reservation_date": date,
                "reservation_time": time,
                "email": email
            })
    else:
        response = {
            "status": "error",
            "message": "No restaurant name provided."
        }
    self.send_response(client_socket, response)

elif action == "admin_add_table":
    table_info = data.get("table_info")
    restaurant_name = table_info.get("restaurant_name")
    category = table_info.get("category")
    table_size = table_info.get("table_size")

```

```

        number_of_tables = table_info.get("number_of_tables")
        success = self.db.add_table(restaurant_name, category, table_size,
number_of_tables)
        if success:
            response = {"status": "success", "message": "Tables added successfully."}
        else:
            response = {"status": "error", "message": "Restaurant not found. Cannot add
tables."}
        client_socket.sendall((json.dumps(response) + "__END__").encode('utf-8'))

    elif action == "admin_remove_table":
        table_info = data.get("table_info")
        restaurant_name = table_info.get("restaurant_name")
        category = table_info.get("category")
        table_size = table_info.get("table_size")
        number_of_tables = table_info.get("number_of_tables")
        success = self.db.remove_table(restaurant_name, category, table_size,
number_of_tables)
        if success:
            response = {"status": "success", "message": "Tables removed successfully."}
        else:
            response = {"status": "error", "message": "Restaurant not found. Cannot remove
tables."}
        self.send_response(client_socket, response)

    else:
        response = {"status": "error", "message": "Invalid action"}
        self.send_response(client_socket, response)

    except json.JSONDecodeError:
        print("Error decoding JSON request.")
        break
    except Exception as e:
        print(f"Error processing request: {e}")
        break

    except Exception as e:
        print(f"Error handling client: {e}")

    finally:
        client_socket.close()

```

Database:

```

import sqlite3
from datetime import datetime, timedelta

class Database:
    מסמכים\רועי\בית ספר\כיתה
    \tabels.db"): יבס ייבר
    def __init__(self, db_name=r"C:\Users\aviva\Dropbox\
        self.db_name = db_name
        self.conn = None
        self.cursor = None

    def connect(self):
        try:
            self.conn = sqlite3.connect(self.db_name, check_same_thread=False)
            self.cursor = self.conn.cursor()
            print("Connection to database established.")
        except sqlite3.Error as e:
            print(f"Error connecting to database: {e}")
            return None
        return self.cursor

    def close(self):
        if self.conn:
            self.conn.close()
            print("Connection closed.")

    def authenticate_user(self, username):
        query = "SELECT id, first_name, user_password, email_address FROM users WHERE
        user_name = ?"
        self.cursor.execute(query, (username,))
        result = self.cursor.fetchone()
        return result if result else None

    def register_user(self, first_name, last_name, email, phone, username, password):
        self.cursor.execute("SELECT * FROM users WHERE user_name = ? OR email_address = ?",
        (username, email))
        if self.cursor.fetchone():
            return False # Duplicate username or email
        self.cursor.execute(
            "INSERT INTO users (first_name, last_name, email_address, phone, user_name,
        user_password) VALUES (?, ?, ?, ?, ?, ?)",
            (first_name, last_name, email, phone, username, password)
        )
        self.conn.commit()
        return True

    def find_tables(self, restaurant_id, table_size, category):
        פונקציה שמחזירה את כל השולחנות המתאימים לפי מסעדה, גודל שולחן וקטגוריה"
        if category and category != "Not specified":

```

```

        table_query = """SELECT id, number_of_tables, category
                           FROM tables
                           WHERE restaurant_id = ?
                           AND category = ?
                           AND table_size >= ?"""
        self.cursor.execute(table_query, (restaurant_id, category, table_size))
    else:
        # במקרה שאין קטגוריה - נבדוק את כל השולחנות המתאימים בגודל
        table_query = """SELECT id, number_of_tables, category
                           FROM tables
                           WHERE restaurant_id = ?
                           AND table_size >= ?"""
        self.cursor.execute(table_query, (restaurant_id, table_size))

    return self.cursor.fetchall()

def search_restaurants(self, table_size, time, date, food_types, districts, price, kosher, rating,
category):
    if not self.cursor:
        print("חיבור למסד הנתונים לא הוקם.")
        return []

    query = "SELECT DISTINCT id, name, opening_time, closing_time,
table_reservation_duration FROM restaurants WHERE 1=1"
    params = []

    if food_types:
        query += " AND food_type IN ({})".format(','.join('?' for _ in food_types))
        params.extend(food_types)

    if districts:
        query += " AND district IN ({})".format(','.join('?' for _ in districts))
        params.extend(districts)

    if price:
        query += " AND average_price_per_person <= ?"
        params.append(price)

    if kosher == 1:
        query += " AND kosher = ?"
        params.append(1)

    if rating:
        query += " AND rating >= ?"
        params.append(rating)

    try:
        self.cursor.execute(query, tuple(params))
        restaurants = self.cursor.fetchall()
        valid_restaurants = []

```



```

המרת תאריך ושעה לפורמט מתאים #
reservation_time = datetime.strptime(time, "%H: %M").time()
reservation_date = datetime.strptime(date, "%Y-%m-%d").date()

מילון לשמירה על מסעדות שכבר נוספו #
restaurant_map = { }

for restaurant in restaurants:
    restaurant_id, name, opening_time, closing_time, reservation_duration = restaurant

    opening_time = datetime.strptime(opening_time, "%H: %M").time()
    closing_time = datetime.strptime(closing_time, "%H: %M").time()
    reservation_end_time = (datetime.combine(reservation_date, reservation_time) +
                             timedelta(minutes=reservation_duration)).time()

HH: MM המרת הזמן למחרוזת בפורמט #
reservation_time_str = reservation_time.strftime("%H: %M")
reservation_end_time_str = reservation_end_time.strftime("%H: %M")

בדיקת זמינות במסעדה לפי שעות פתיחה #
if not (opening_time <= reservation_time <= closing_time and reservation_end_time <=
closing_time):
    continue

חיפוש שולחנות רלוונטיים במסעדה #
tables = self.find_tables(restaurant_id, table_size, category)
ברירת מחדל במקרה שאין הגדרה לישיבה seating_availability = "No seating available" #

for table in tables:
    כל שורה ב tables - היא טופל עם 3 ערכים #
    table_id, number_of_tables, table_category = table

דגל לבדיקה אם לפחות שולחן אחד זמין is_restaurant_available = False #

HH: MM שליחת השאילתה כאשר אנחנו משתמשים במחרוזות בפורמט #
reservation_query = """SELECT COUNT(*)
                        FROM reservations
                        WHERE table_id = ?
                        AND reservation_date = ?
                        AND ((reservation_time <= ? AND end_reservation > ?)
                        OR (reservation_time < ? AND end_reservation >= ?))"""
self.cursor.execute(reservation_query,
                    (table_id, reservation_date, reservation_time_str, reservation_time_str,
                     reservation_end_time_str, reservation_end_time_str))
reservation_count = self.cursor.fetchone()[0]

בדיקה אם יש מספיק מקומות פנויים לשולחן הזה #
if reservation_count < number_of_tables:
    is_restaurant_available = True

```

```

        if is_restaurant_available:
            אם המסעדה לא קיימת במילון, הוסף אותה #
            if name not in restaurant_map:
                restaurant_map[name] = {
                    'name': name,
                    'seating_availability': seating_availability
                }

            עדכון זמינות ישיבה, אם יש קטגוריה חדשה #
            if table_category and table_category not in
            restaurant_map[name]['seating_availability']:
                if restaurant_map[name]['seating_availability'] == "No seating available":
                    restaurant_map[name]['seating_availability'] = table_category
                else:
                    restaurant_map[name]['seating_availability'] += ' and ' + table_category

            לאחר שמצאנו את כל המסעדות, ניצור רשימה מסודרת #
            valid_restaurants = list(restaurant_map.values())

            print(f"Found restaurants: {valid_restaurants}")
            החזרת רשימה של מילונים return valid_restaurants #

    except sqlite3.Error as e:
        print(f"Error executing query: {e}")
        return []

    def get_restaurant_info(self, restaurant):
        query = "SELECT food_type, city, rating FROM restaurants WHERE name = ?"
        self.cursor.execute(query, (restaurant,))
        result = self.cursor.fetchone()
        return result if result else None

    def order_a_table(self, restaurant, category, table_size, reservation_time, reservation_date,
user_id):
        try:
            חיפוש ה-ID של המסעדה לפי השם שלה #
            self.cursor.execute("SELECT id FROM restaurants WHERE name = ?", (restaurant,))
            restaurant_id = self.cursor.fetchone()[0]

            חיפוש השולחנות המתאימים #
            tables = self.find_tables(restaurant_id, table_size, category)
            table_id = tables[0][0]

            הכנסת ההזמנה #
            self.cursor.execute(
                "INSERT INTO reservations (user_id, table_id, reservation_time, reservation_date)
VALUES (?, ?, ?, ?)",
                (user_id, table_id, reservation_time, reservation_date)
            )

```

```

    )

    AUTOINCREMENT)שית (בהנחה שיש ID של ההזמנה שנוצרה) קבלת ה ID-#
        reservation_id = self.cursor.lastrowid

    self.conn.commit()

    שליפת זמן הסיום מהשורה שנוצרה #
        self.cursor.execute("SELECT end_reservation FROM reservations WHERE id = ?",
(reservation_id,))
        end_reservation = self.cursor.fetchone()[0]

    print("Table reserved successfully!")
    return end_reservation

except sqlite3.Error as e:
    print(f"Error executing reservation query: {e}")
    return None

def view_orders_by_date(self, date):
    query = """
        SELECT res.user_id, res.reservation_time, r.name AS restaurant_name
        FROM reservations res
        JOIN tables t ON res.table_id = t.id
        JOIN restaurants r ON t.restaurant_id = r.id
        WHERE res.reservation_date = ?
    """
    self.cursor.execute(query, (date,))
    return self.cursor.fetchall()

def view_orders_by_user(self, user_name):
    שליפת ה user_id-מתוך טבלת users לפי user_name #
    query_user_check = """
        SELECT id FROM users WHERE user_name = ?
    """
    self.cursor.execute(query_user_check, (user_name,))
    user = self.cursor.fetchone()

    if not user:
        return None

    user_id = user[0]

    שליפת ההזמנות כולל reservation_id #
    query_reservations = """
        SELECT res.id AS reservation_id, res.reservation_date, res.reservation_time, r.name AS
restaurant_name
        FROM reservations res
        JOIN tables t ON res.table_id = t.id
        JOIN restaurants r ON t.restaurant_id = r.id
    """

```

```

        WHERE res.user_id = ?
        """
    self.cursor.execute(query_reservations, (user_id,))
    return self.cursor.fetchall()

def get_most_popular_restaurant(self):
    query = """
        SELECT r.name, COUNT(*) AS reservation_count
        FROM reservations res
        JOIN tables t ON res.table_id = t.id
        JOIN restaurants r ON t.restaurant_id = r.id
        GROUP BY r.id
        ORDER BY reservation_count DESC
        LIMIT 5
    """
    self.cursor.execute(query)
    return self.cursor.fetchall()

def get_most_popular_users(self):
    query = """
        SELECT u.user_name, COUNT(r.id) as reservation_count
        FROM users u
        JOIN reservations r ON u.id = r.user_id
        GROUP BY u.id
        ORDER BY reservation_count DESC
        LIMIT 5
    """
    self.cursor.execute(query)
    return self.cursor.fetchall()

def cancel_reservation(self, reservation_id):
    try:
        query = "DELETE FROM reservations WHERE id = ?"
        self.cursor.execute(query, (reservation_id,))
        self.conn.commit()

        אם אכן נמחקה שורה – הפעולה הצליחה #
        return self.cursor.rowcount > 0

    except Exception as e:
        print("Error cancelling reservation: ", e)
        return False

def add_restaurant(self, name, food_type, opening_time, closing_time,
                    table_reservation_duration, city, district,
                    average_price_per_person, kosher, rating):
    try:
        הוספת המסעדה לבסיס הנתונים #
        query = """

```

```

        INSERT INTO restaurants
        (name, food_type, opening_time, closing_time, table_reservation_duration,
        city, district, average_price_per_person, kosher, rating)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """

        self.cursor.execute(query, (name, food_type, opening_time, closing_time,
        table_reservation_duration,
        city, district, average_price_per_person, kosher, rating))
        self.conn.commit()

    החזרת תגובת הצלחה      #
    return {
        "status": "success",
        "message": "Restaurant added successfully."
    }

    except Exception as e:
    החזרת הודעת שגיאה אם משהו השתבש      #
    return {
        "status": "error",
        "message": f"Failed to add restaurant: {str(e)}"
    }

    def delete_restaurant(self, name):
        cursor = self.conn.cursor()

    בדיקה אם המסעדה קיימת      #
    cursor.execute("SELECT id FROM restaurants WHERE name = ?", (name,))
    restaurant = cursor.fetchone()
    if not restaurant:
        return False, f"Restaurant '{name}' not found.", []

    restaurant_id = restaurant[0]

    שליפת פרטי ההזמנות העתידיות כדי שנוכל לשלוח מיילים      #
    cursor.execute("""
        SELECT r.id, r.user_id, r.reservation_date, r.reservation_time, u.email_address
        FROM reservations r
        JOIN tables t ON r.table_id = t.id
        JOIN users u ON r.user_id = u.id
        WHERE t.restaurant_id = ? AND r.reservation_date >= DATE('now')
        """, (restaurant_id,))
    cancelled_orders = cursor.fetchall()

    מחיקת ההזמנות      #
    cursor.execute("""
        DELETE FROM reservations
        WHERE table_id IN (
            SELECT id FROM tables WHERE restaurant_id = ?

```

```

        ) AND reservation_date >= DATE('now')
        """, (restaurant_id,))

# מחיקת השולחנות והמסעדה
cursor.execute("DELETE FROM tables WHERE restaurant_id = ?", (restaurant_id,))
cursor.execute("DELETE FROM restaurants WHERE id = ?", (restaurant_id,))
self.conn.commit()

return True, f'Restaurant '{name}' and its future reservations deleted successfully.',
cancelled_orders

def add_table(self, restaurant_name, category, table_size, number_of_tables):
    try:
        cursor = self.conn.cursor()

# שליפת restaurant_id לפי שם המסעדה
        cursor.execute("SELECT id FROM restaurants WHERE name = ?", (restaurant_name,))
        result = cursor.fetchone()

        if not result:
            return False # לא נמצאה מסעדה בשם הזה

        restaurant_id = result[0]

# בדיקה אם כבר קיימת שורה עם אותם נתונים
        cursor.execute("""
            SELECT id, number_of_tables FROM tables
            WHERE restaurant_id = ? AND category = ? AND table_size = ?
            """, (restaurant_id, category, table_size))

        existing_table = cursor.fetchone()

        if existing_table:
            # אם קיימת – עדכון כמות השולחנות
            table_id, current_count = existing_table
            new_count = current_count + number_of_tables
            cursor.execute("""
                UPDATE tables SET number_of_tables = ?
                WHERE id = ?
                """, (new_count, table_id))
        else:
            # אחרת – הכנסה חדשה
            cursor.execute("""
                INSERT INTO tables (restaurant_id, category, table_size, number_of_tables)
                VALUES (?, ?, ?, ?)
                """, (restaurant_id, category, table_size, number_of_tables))

        self.conn.commit()
        return True

```

```

except sqlite3.Error as e:
    print("Database error in add_table: ", e)
    return False

def remove_table(self, restaurant_name, category, table_size, number_to_remove):
    try:
        cursor = self.conn.cursor()

        # שליפת restaurant_id לפי שם המסעדה
        cursor.execute("SELECT id FROM restaurants WHERE name = ?", (restaurant_name,))
        result = cursor.fetchone()

        if not result:
            # לא נמצאה מסעדה
            return False

        restaurant_id = result[0]

        # חיפוש השורה בטבלה לפי המסעדה, קטגוריה וגודל שולחן
        cursor.execute("""
            SELECT id, number_of_tables FROM tables
            WHERE restaurant_id = ? AND category = ? AND table_size = ?
            """, (restaurant_id, category, table_size))

        table_row = cursor.fetchone()

        if not table_row:
            # לא נמצאה שורה עם הפרטים האלו
            return False

        table_id, current_count = table_row
        new_count = current_count - number_to_remove

        if new_count <= 0:
            # מחיקה של השורה
            cursor.execute("DELETE FROM tables WHERE id = ?", (table_id,))
        else:
            # עדכון הכמות
            cursor.execute("UPDATE tables SET number_of_tables = ? WHERE id = ?",
                (new_count, table_id))

        self.conn.commit()
        return True

    except sqlite3.Error as e:
        print("Database error in remove_table: ", e)
        return False

```

Communication:

import socket

class Communication:

def __init__(self, ip, port, is_server=False):

self.ip = ip

self.port = port

self.is_server = is_server

self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

if is_server:

try:

self.socket.bind((self.ip, self.port))

self.socket.listen(5)

print(f"Server listening on {self.ip}: {self.port}")

except Exception as e:

print(f"Server error: {e}")

def connect(self):

"""מתחבר לשרת כלקוח"""

if not self.is_server:

try:

self.socket.connect((self.ip, self.port))

print("Connected to server.")

except Exception as e:

print(f"Connection error: {e}")

self.socket = None

def accept_connection(self):

"""השרת מקבל חיבור חדש"""

if self.is_server:

try:

client_socket, addr = self.socket.accept()

print(f"New connection from {addr}")

return client_socket, addr

except Exception as e:

print(f"Error accepting connection: {e}")

return None, None

def send(self, message):

"""שליחת הודעה"""

if self.socket:

try:

if isinstance(message, str):

message = message.encode('utf-8')

self.socket.sendall(message)

except Exception as e:

print(f"Error sending message: {e}")

self.close()


```
def receive(self):
    """קבלת הודעה מלאה עם סימון סוף הודעה"""
    if self.socket:
        try:
            data = b""
            while True:
                part = self.socket.recv(1024)
                if not part:
                    break
                data += part
                if b"__END__" in part:
                    break
            return data.decode('utf-8').replace("__END__", "")
        except Exception as e:
            print(f"Error receiving message: {e}")
            self.close()
    return None

def close(self):
    """סגירת חיבור"""
    if self.socket:
        try:
            self.socket.close()
            print("Connection closed.")
        except Exception as e:
            print(f"Error closing connection: {e}")
    self.socket = None
```

Rsa:

import random

class RSA:

def __init__(self, p=None, q=None):

if not p or not q:

p, q = self.generate_random_primes()

self.p = p

self.q = q

self.n = p * q

self.phi = (p - 1) * (q - 1)

self.public_key, self.private_key = self.generate_keypair()

def generate_random_primes(self):

prime_list = [53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101]

p = random.choice(prime_list)

q = random.choice(prime_list)

while q == p:

q = random.choice(prime_list)

return p, q

def gcd(self, a, b):

while b != 0:

a, b = b, a % b

return a

def multiplicative_inverse(self, e, phi):

d = 0

x1, x2 = 0, 1

y1 = 1

temp_phi = phi

while e > 0:

temp1 = temp_phi // e

temp2 = temp_phi - temp1 * e

temp_phi, e = e, temp2

x = x2 - temp1 * x1

y = d - temp1 * y1

x2, x1 = x1, x

d, y1 = y1, y

if temp_phi == 1:

return d + phi

def generate_keypair(self):

e = random.randrange(2, self.phi)

```
while self.gcd(e, self.phi) != 1:
    e = random.randrange(2, self.phi)
d = self.multiplicative_inverse(e, self.phi)
return ((e, self.n), (d, self.n))

def encrypt(self, plaintext, key):
    e, n = key
    byte_data = plaintext.encode('utf-8')
    encrypted = [(byte ** e) % n for byte in byte_data]
    return encrypted

def decrypt(self, ciphertext, key):
    d, n = key
    decrypted = [chr((byte ** d) % n) for byte in ciphertext]
    return ''.join(decrypted)

def encrypt_as_str(self, plaintext, key):
    encrypted_list = self.encrypt(plaintext, key)
    return ','.join(map(str, encrypted_list))

def decrypt_from_str(self, encrypted_str, key):
    encrypted_list = list(map(int, encrypted_str.split(',')))
    return self.decrypt(encrypted_list, key)
```

Mail:

```

import smtplib
import random
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

class MailSender:
    def __init__(self):
        self.sender_email = "easy.ordermail1@gmail.com"
        self.sender_password = "dxjv hmjt blwq getl"
        self.smtp_server = "smtp.gmail.com"
        self.smtp_port = 587
        self.last_code_sent = None

    def generate_verification_code(self):
        self.last_code_sent = "".join([str(random.randint(0, 9)) for _ in range(8)])
        return self.last_code_sent

    def send_verification_code(self, recipient_email):
        code = self.generate_verification_code()

        message = MIMEMultipart()
        message["From"] = self.sender_email
        message["To"] = recipient_email
        message["Subject"] = "Your Verification Code"

        body = f"Hello!\n\nYour verification code is: {code}\n\nIf you did not request this, please ignore this email."
        message.attach(MIMEText(body, "plain"))

        try:
            with smtplib.SMTP(self.smtp_server, self.smtp_port) as server:
                server.starttls()
                server.login(self.sender_email, self.sender_password)
                server.sendmail(self.sender_email, recipient_email, message.as_string())
        except Exception as e:
            print("Failed to send email: ", e)
            return None

    def send_cancel_confirmation(self, recipient_email, order):
        subject = "Reservation Cancellation - EasyOrder"
        body = (
            f"Hello,\n\n"
            f"Your reservation has been successfully cancelled.\n\n"
            f"✗ Cancelled Reservation Details: \n"
            f"📍 Restaurant: {order.get('restaurant_name', 'N/A')}\n"

```

כאן מחזיר את הקוד עצמו return code # ←

```

f"📅 Date: {order.get('reservation_date', 'N/A')}\n"
f"🕒 Time: {order.get('reservation_time', 'N/A')}\n\n"
f"We're sorry to see you go. Hope to serve you again soon!\n\n"
f"EasyOrder Team"
)

message = MIMEMultipart()
message["From"] = self.sender_email
message["To"] = recipient_email
message["Subject"] = subject
message.attach(MIMEText(body, "plain"))

try:
    with smtplib.SMTP(self.smtp_server, self.smtp_port) as server:
        server.starttls()
        server.login(self.sender_email, self.sender_password)
        server.sendmail(self.sender_email, recipient_email, message.as_string())
    return True
except Exception as e:
    print("Failed to send cancellation email: ", e)
    return False

def send_cancel_restaurant_closed(self, recipient_email, order):
    subject = "Reservation Cancellation - EasyOrder"
    body = (
        f"Hello,\n\n"
        f"Unfortunately Your reservation has been cancelled.\n\n"
        f"Because {order.get('restaurant_name', 'N/A')} is closed.\n\n"
        f"❌ Cancelled Reservation Details: \n"
        f"📍 Restaurant: {order.get('restaurant_name', 'N/A')}\n"
        f"📅 Date: {order.get('reservation_date', 'N/A')}\n"
        f"🕒 Time: {order.get('reservation_time', 'N/A')}\n\n"
        f"Hope to serve you again soon!\n\n"
        f"EasyOrder Team"
    )

    message = MIMEMultipart()
    message["From"] = self.sender_email
    message["To"] = recipient_email
    message["Subject"] = subject
    message.attach(MIMEText(body, "plain"))

    try:
        with smtplib.SMTP(self.smtp_server, self.smtp_port) as server:
            server.starttls()
            server.login(self.sender_email, self.sender_password)
            server.sendmail(self.sender_email, recipient_email, message.as_string())
        return True

```

```

except Exception as e:
    print("Failed to send cancellation email: ", e)
    return False

def send_reservation_confirmation(self, recipient_email, restaurant, city, date, start_time,
end_time, people,
                                seating_category):
    subject = "Reservation Confirmation - EasyOrder"
    body = (
        f"Hello,\n\n"
        f"Your reservation is confirmed!\n\n"
        f"📍 Restaurant: {restaurant} ({city})\n"
        f"📅 Date: {date}\n"
        f"🕒 Time: {start_time} - {end_time}\n"
        f"👥 Number of People: {people}\n"
        f"🪑 Seating: {seating_category}\n\n"
        f"Thank you for using EasyOrder!"
    )

    message = MIMEMultipart()
    message["From"] = self.sender_email
    message["To"] = recipient_email
    message["Subject"] = subject
    message.attach(MIMEText(body, "plain"))

    try:
        with smtplib.SMTP(self.smtp_server, self.smtp_port) as server:
            server.starttls()
            server.login(self.sender_email, self.sender_password)
            server.sendmail(self.sender_email, recipient_email, message.as_string())
        return True
    except Exception as e:
        print("Failed to send confirmation email: ", e)
        return False

def get_last_code_sent(self):
    return self.last_code_sent

```

client_module:

```
from communication import Communication
from gui import GUI
```

```
class Client(Communication):
```

```
    def __init__(self, ip, port):
```

```
        super().__init__(ip, port, is_server=False)
```

```
-Clientת יצירת השרת מתחבר לשרת אחרי יצירת השרת self.connect() #
```

```
-GUIה את מפעיל את self.start_gui() #
```

```
    def start_gui(self):
```

```
-GUIה את הלקוח נעביר את #
```

```
של Communication-ה מעביר את ה Client-ל, GUI-כדי ש GUI-יכול להשתמש ב Communication-של gui = GUI(self) #
```

```
הלקוח
```

```
-GUIה את מתחיל את הלולאת gui.run() #
```

Gui:

```

import customtkinter as ctk
from tkinter import messagebox
import re
import time
import json
from datetime import datetime
from rsa import RSA
from tkcalendar import Calendar
from mail import MailSender
from PIL import Image
import threading

```

```

class GUI:

```

```

    def __init__(self, client_comm):
        self.client_comm = client_comm
        self.root = ctk.CTk()
        self.rsa = RSA()
        self.root.title("Easy Order - Restaurant Reservation")
        self.root.geometry("600x750")
        ctk.set_appearance_mode("light")
        ctk.set_default_color_theme("blue")

```

```

===== הגדרות מסכים ראשיים # =====
        self.login_register_frame: ctk.CTkFrame | None = None
        self.login_frame: ctk.CTkFrame | None = None
        self.register_frame: ctk.CTkFrame | None = None

```

```

===== שדות התחברות # =====
        self.username_entry: ctk.CTkEntry | None = None
        self.password_entry: ctk.CTkEntry | None = None

```

```

===== שדות הרשמה # =====
        self.first_name_entry: ctk.CTkEntry | None = None
        self.last_name_entry: ctk.CTkEntry | None = None
        self.phone_entry: ctk.CTkEntry | None = None
        self.email_entry: ctk.CTkEntry | None = None
        self.reg_username_entry: ctk.CTkEntry | None = None
        self.reg_password_entry: ctk.CTkEntry | None = None
        self.reg_confirm_password_entry: ctk.CTkEntry | None = None
        self.password_strength_label: ctk.CTkLabel | None = None

```

```

        self.server_public_key = self.get_server_public_key()

```

```

===== פרטי משתמש מחובר # =====
        self.user_id: int | None = None
        self.user_first_name: str = ""
        self.username: str = ""
        self.user_email: str = ""

```



```

self.verification_code: str | None = None
self.loading_window: ctk.CTkToplevel | None = None

=====טופס חיפוש מסעדה=====
self.food_listbox: ctk.CTkOptionMenu | None = None
self.districts_listbox: ctk.CTkOptionMenu | None = None
self.kosher_var: ctk.BooleanVar | None = None
self.rating_var: ctk.StringVar | None = None
self.rating_menu: ctk.CTkOptionMenu | None = None
self.seating_var: ctk.StringVar | None = None

=====מסך הזמנות משתמש=====
לשמירת האינדקס הנבחר self.selected_restaurant_var = ctk.IntVar(value=-1) #
self.restaurants_mapping = { }

=====שדות מילוי טופס הזמנה=====
self.date_entry: ctk.CTkEntry | None = None
self.time_entry: ctk.CTkEntry | None = None
self.people_entry: ctk.CTkEntry | None = None
self.price_entry: ctk.CTkEntry | None = None

=====בחירת מסעדה=====

=====פרטי מסעדה נבחרת=====
self.restaurant: str | None = None
self.available_seating: str | None = None
self.city: str | None = None
self.food_type: str | None = None
self.rating: str | None = None

=====אימות דו-שלבי=====
self.mail_sender: MailSender | None = None
self.verification_code: str | None = None
self.verification_window: ctk.CTkToplevel | None = None
self.timer_label: ctk.CTkLabel | None = None
self.code_entry: ctk.CTkEntry | None = None
self.remaining_seconds: int = 120
self.timer_running: bool = False

=====הפעלת מסך פתיחה=====
self.create_login_register_screen()

def get_server_public_key(self):
    request = {
        "action": "get_public_key"
    }
    self.client_comm.send(json.dumps(request).encode('utf-8'))
    response = json.loads(self.client_comm.receive())
    if response["status"] == "success":

```

```

        key_data = response["public_key"]
        return (key_data["e"], key_data["n"])
    else:
        raise Exception("Failed to retrieve server's public key")

    def create_login_register_screen(self):
        # יצירת מסגרת עיקרית
        self.login_register_frame = ctk.CTkFrame(self.root, corner_radius=15)
        self.login_register_frame.pack(pady=60, padx=60, fill="both", expand=True)

        # הגדרת רקע לא חובה)
        self.root.configure(bg="white")

        # ===== הצגת לוגו =====
        try:
            logo_image = Image.open("logo.jpeg")
            logo_ctk_image = ctk.CTkImage(light_image=logo_image, size=(150, 150))

            logo_label = ctk.CTkLabel(self.login_register_frame, text="", image=logo_ctk_image)
            # מניעת איסוף זיכרון
            logo_label.image = logo_ctk_image #
            logo_label.pack(pady=20)
        except Exception as e:
            # בעיה בטעינת הלוגו(e)
            print("

        # ===== כותרת =====
        title_label = ctk.CTkLabel(
            self.login_register_frame,
            text="Welcome to Easy Order",
            font=ctk.CTkFont(size=24, weight="bold")
        )
        title_label.pack(pady=10)

        # ===== כפתורים =====
        buttons_frame = ctk.CTkFrame(self.login_register_frame, fg_color="transparent")
        buttons_frame.pack(pady=30)

        login_button = ctk.CTkButton(
            buttons_frame,
            text="Log In",
            width=120,
            height=40,
            font=ctk.CTkFont(size=16),
            command=self.show_login_screen
        )
        login_button.pack(side="left", padx=20)

        register_button = ctk.CTkButton(
            buttons_frame,
            text="Register",
            width=120,

```

```

        height=40,
        font=ctk.CTkFont(size=16),
        fg_color="green",
        hover_color="#168f30",
        command=self.show_register_screen
    )
    register_button.pack(side="left", padx=20)

def show_login_screen(self):
    self.login_register_frame.pack_forget()
    self.login_frame = ctk.CTkFrame(self.root, corner_radius=20, fg_color="white")
    self.login_frame.pack(pady=40, padx=60, fill="both", expand=True)

לוגו    #
    try:
        logo_image = Image.open("logo.jpeg").resize((100, 100), Image.LANCZOS)
        self.logo_ctk_image = ctk.CTkImage(light_image=logo_image, dark_image=logo_image,
size=(100, 100))
        logo_label = ctk.CTkLabel(self.login_frame, text="", image=self.logo_ctk_image)
        logo_label.pack(pady=10)
    except Exception as e:
        שגיאה בטעינת הלוגו(e)    print("")

כותרת    #
        title_label = ctk.CTkLabel(self.login_frame, text="Welcome Back!",
                                font=ctk.CTkFont(size=24, weight="bold"),
                                text_color="#333")
        title_label.pack(pady=(10, 20))

שדות    #
        ctk.CTkLabel(self.login_frame, text="Username: ", anchor="w").pack(fill="x", padx=20)
        self.username_entry = ctk.CTkEntry(self.login_frame)
        self.username_entry.pack(pady=5, padx=20, fill="x")

        ctk.CTkLabel(self.login_frame, text="Password: ", anchor="w").pack(fill="x", padx=20)
        self.password_entry = ctk.CTkEntry(self.login_frame, show="*")
        self.password_entry.pack(pady=5, padx=20, fill="x")

כפתורים    #
        ctk.CTkButton(self.login_frame, text="Log In", width=150,
command=self.login).pack(pady=(20, 10))
        ctk.CTkButton(self.login_frame, text="Back", width=150, fg_color="red",
                                hover_color="#a80000", command=self.back_to_login_register_screen).pack()

def show_register_screen(self):
    self.login_register_frame.pack_forget()
    self.register_frame = ctk.CTkFrame(self.root, corner_radius=20, fg_color="white")
    self.register_frame.pack(pady=40, padx=40, fill="both", expand=True)

    try:

```

```

logo_image = Image.open("logo.jpeg").resize((100, 100), Image.LANCZOS)
self.logo_ckpt_image = ctk.CTkImage(light_image=logo_image, dark_image=logo_image,
size=(100, 100))
    ctk.CTkLabel(self.register_frame, text="", image=self.logo_ckpt_image).pack(pady=10)
except Exception as e:
: ", שגיאה בטעינת הלוגו) e) print("

    ctk.CTkLabel(self.register_frame, text="Create Your Account",
        font=ctk.CTkFont(size=24, weight="bold"), text_color="#333").pack(pady=(0, 20))

form_frame = ctk.CTkFrame(self.register_frame, fg_color="transparent")
form_frame.pack(padx=20, fill="both", expand=True)

left_frame = ctk.CTkFrame(form_frame, fg_color="transparent")
left_frame.pack(side="left", expand=True, fill="both", padx=10)
right_frame = ctk.CTkFrame(form_frame, fg_color="transparent")
right_frame.pack(side="right", expand=True, fill="both", padx=10)

# עמודה שמאלית
    ctk.CTkLabel(left_frame, text="First Name: ").pack(anchor="w")
    self.first_name_entry = ctk.CTkEntry(left_frame)
    self.first_name_entry.pack(pady=5, fill="x")

    ctk.CTkLabel(left_frame, text="Phone Number: ").pack(anchor="w")
    self.phone_entry = ctk.CTkEntry(left_frame)
    self.phone_entry.pack(pady=5, fill="x")

    ctk.CTkLabel(left_frame, text="Username: ").pack(anchor="w")
    self.reg_username_entry = ctk.CTkEntry(left_frame)
    self.reg_username_entry.pack(pady=5, fill="x")

# עמודה ימנית
    ctk.CTkLabel(right_frame, text="Last Name: ").pack(anchor="w")
    self.last_name_entry = ctk.CTkEntry(right_frame)
    self.last_name_entry.pack(pady=5, fill="x")

    ctk.CTkLabel(right_frame, text="Email: ").pack(anchor="w")
    self.email_entry = ctk.CTkEntry(right_frame)
    self.email_entry.pack(pady=5, fill="x")

    ctk.CTkLabel(right_frame, text="Password: ").pack(anchor="w")
    self.reg_password_entry = ctk.CTkEntry(right_frame, show="*")
    self.reg_password_entry.pack(pady=5, fill="x")

    self.password_strength_label = ctk.CTkLabel(right_frame, text="", text_color="gray")
    self.password_strength_label.pack(anchor="w", pady=(0, 5))
    self.reg_password_entry.bind("<KeyRelease>", self.update_password_strength)

    ctk.CTkLabel(right_frame, text="Confirm Password: ").pack(anchor="w")
    self.reg_confirm_password_entry = ctk.CTkEntry(right_frame, show="*")

```

```

self.reg_confirm_password_entry.pack(pady=5, fill="x")

# כפתורים
buttons_frame = ctk.CTkFrame(self.register_frame, fg_color="transparent")
buttons_frame.pack(pady=20)

ctk.CTkButton(buttons_frame, text="Register", width=150,
               fg_color="green", hover_color="#168f30",
               command=self.register).pack(side="left", padx=10)

ctk.CTkButton(buttons_frame, text="Back", width=150,
               fg_color="red", hover_color="#a80000",
               command=self.back_to_login_register_screen).pack(side="right", padx=10)

def login(self):
    username = self.username_entry.get().strip()
    password = self.password_entry.get().strip()

    if not username or not password:
        messagebox.showerror("Input Error", "Please fill in both username and password.")
        return

    if username == "admin" and password == "0000":
        self.open_admin_panel()
        return

    encrypted_password = self.rsa.encrypt_as_str(password, self.server_public_key)

    login_data = {
        'username': username,
        'encrypted_password': encrypted_password,
        'action': 'login'
    }
    self.client_comm.send(json.dumps(login_data).encode('utf-8'))

    response = self.client_comm.receive()
    data = json.loads(response.encode('utf-8'))
    check = data.get('status')

    if check == "success":
        self.username = username
        self.user_id = data.get("user_id")
        self.user_first_name = data.get("first_name")
        self.user_email = data.get("email_address") # ודא שהשרת מחזיר את האימייל

    # פתיחת חלון האימות מידית
    self.open_verification_window()

    # שליחת הקוד בת'רד נפרד
    threading.Thread(

```

```

        target=self.send_code_thread,
        daemon=True
    ).start()
else:
    messagebox.showerror("Login Error", data.get('message'))

def send_code_thread(self):
    try:
        self.mail_sender = MailSender()
        self.verification_code = self.mail_sender.send_verification_code(self.user_email)
        if not self.verification_code:
            self.root.after(0, lambda: messagebox.showerror("Error",
                                                            "Failed to send verification code. Please try again."))
    except Exception as e:
        self.root.after(0, lambda: messagebox.showerror("Error", f"Mail sending failed: \n{e}"))

def open_verification_window(self):
    self.verification_window = ctk.CTkToplevel(self.root)
    self.verification_window.title("Two-Factor Authentication")
    self.verification_window.geometry("400x400")
    self.verification_window.resizable(False, False)

    self.verification_window.lift()
    self.verification_window.focus_force()
    self.verification_window.grab_set()

# לוגו
    try:
        logo_image = Image.open("logo.jpeg").resize((80, 80), Image.LANCZOS)
        self.verification_logo = ctk.CTkImage(light_image=logo_image,
        dark_image=logo_image, size=(80, 80))
        ctk.CTkLabel(self.verification_window, image=self.verification_logo,
        text="").pack(pady=(20, 5))
    except Exception as e:
        print("שגיאה בטעינת הלוגו(e), ")

# כותרת
    ctk.CTkLabel(self.verification_window, text="Verification Required",
        font=ctk.CTkFont(size=20, weight="bold")).pack(pady=5)

# טקסט תיאור
    ctk.CTkLabel(self.verification_window, text=f"A verification code was sent to
    {self.user_email}",
        wraplength=350, justify="center").pack(pady=(5, 15))

# שדה קוד
    ctk.CTkLabel(self.verification_window, text="Enter the 8-digit code: ").pack()
    self.code_entry = ctk.CTkEntry(self.verification_window, justify="center", width=200)
    self.code_entry.pack(pady=10)

```

```

כפתור אימות      #
    ctk.CTkButton(self.verification_window, text="Verify", command=self.verify_code,
                    fg_color="green", hover_color="#168f30", width=180).pack(pady=5)

כפתור שליחה מחדש      #
    ctk.CTkButton(self.verification_window, text="Send Again",
                    command=self.resend_verification_code,
                    fg_color="gray", hover_color="#666", width=180).pack(pady=5)

טיימר      #
    self.timer_label = ctk.CTkLabel(self.verification_window, text="", text_color="gray")
    self.timer_label.pack(pady=10)

    self.start_verification_timer()

    def resend_verification_code(self):
        self.verification_code = self.mail_sender.send_verification_code(self.user_email)
איפוס טיימר      self.remaining_seconds = 120 #
        messagebox.showinfo("Code Resent", f"A new verification code was sent to
{self.user_email}")
        self.start_verification_timer()

    def start_verification_timer(self):
בדק אם כבר רץ      if getattr(self, "timer_running", False): #
        return
        self.timer_running = True
        self.remaining_seconds = 120

    def update_timer():
        if self.remaining_seconds > 0:
            mins, secs = divmod(self.remaining_seconds, 60)
            timer_text = f"⌚ Code expires in: {mins:02}:{secs:02}"

שינוי צבע לפי זמן      #
        if self.remaining_seconds > 60:
            color = "green"
        elif self.remaining_seconds > 30:
            color = "orange"
        else:
            color = "red"

עדכון תצוגה      #
        self.timer_label.configure(text=timer_text, text_color=color, font=("Arial", 18, "bold"))
        self.remaining_seconds -= 1
        self.timer_label.after(1000, update_timer)
    else:
        self.timer_label.configure(text="❌ Verification code expired", text_color="red",
                                    font=("Arial", 18, "bold"))
        self.verification_code = None
        self.timer_running = False

```

```

update_timer()

def verify_code(self):
    entered_code = self.code_entry.get().strip()
    if self.verification_code is None:
        messagebox.showerror("Code Expired", "The verification code has expired. Please click
'Send Again'.")
        return

    if entered_code == self.verification_code:
        self.verification_window.destroy()
        self.login_frame.pack_forget()
        self.create_widgets()
        messagebox.showinfo("Login Verified", f"Welcome back {self.user_first_name}!")
    else:
        messagebox.showerror("Verification Failed", "Incorrect verification code. Try again.")

def register(self):
    # שלב ההרשמה
    first_name = self.first_name_entry.get().strip()
    last_name = self.last_name_entry.get().strip()
    phone = self.phone_entry.get().strip()
    email = self.email_entry.get().strip()
    username = self.reg_username_entry.get().strip()
    password = self.reg_password_entry.get().strip()
    confirm_password = self.reg_confirm_password_entry.get().strip()

    if not (first_name and last_name and phone and email and username and password and
confirm_password):
        messagebox.showerror("Input Error", "Please fill in all fields.")
        return

    if password != confirm_password:
        messagebox.showerror("Input Error", "Passwords do not match.")
        return

    if not self.is_valid_email(email):
        messagebox.showerror("Input Error", "Invalid email format.")
        return

    # בדיקת חוזק סיסמה
    strength = self.evaluate_password_strength(password)
    if strength == "Weak":
        messagebox.showerror("Password Strength",
            "Your password is too weak. Use uppercase, lowercase, symbols and at least 8
characters.")
        return

    # הצפנת הסיסמה בעזרת המפתח הציבורי

```



```

encrypted_password = self.rsa.encrypt_as_str(password, self.server_public_key)

שליחת נתוני ההרשמה לשרת #
register_data = {
    'action': "register",
    'first_name': first_name,
    'last_name': last_name,
    'phone': phone,
    'email': email,
    'username': username,
    'encrypted_password': encrypted_password,
}

self.client_comm.send(json.dumps(register_data).encode('utf-8'))
response = self.client_comm.receive()
data = json.loads(response.encode('utf-8'))
check = data.get('status')

if check == "success":
    messagebox.showinfo("Registration Successful", "You have successfully registered!")

    if self.register_frame:
        self.register_frame.pack_forget()
        self.register_frame = None

    self.create_widgets()
else:
    messagebox.showerror("Registration Error", data.get('message'))

# פונקציית עזר פנימית במחלקה – אפשר להוציא אותה אם אתה מעדיף
def evaluate_password_strength(self, password):
    import re
    strength_points = 0
    if len(password) >= 8:
        strength_points += 1
    if re.search(r"[A-Z]", password):
        strength_points += 1
    if re.search(r"[a-z]", password):
        strength_points += 1
    if re.search(r"[^A-Za-z0-9]", password): # תווים מיוחדים
        strength_points += 1

    if strength_points == 4:
        return "Strong"
    elif strength_points == 3 or len(password) >= 8:
        return "Medium"
    else:
        return "Weak"

```

```

def update_password_strength(self, event=None):
    password = self.reg_password_entry.get()
    strength = self.evaluate_password_strength(password)

    if strength == "Strong":
        color = "green"
    elif strength == "Medium":
        color = "orange"
    else:
        color = "red"

    self.password_strength_label.configure(text=f"Password strength: {strength}",
text_color=color)

def back_to_login_register_screen(self):
    if self.login_frame:
        self.login_frame.pack_forget()
        self.login_frame = None

    if self.register_frame:
        self.register_frame.pack_forget()
        self.register_frame = None

    self.create_login_register_screen()

def is_valid_email(self, email):
    # בדיקה אם כתובת האימייל תקינה
    return re.match(r"^[^@]+@[^@]+\.[^@]+$", email)

def is_valid_date_format(self, date_str):
    return re.fullmatch(r"^\d{4}-\d{2}-\d{2}$", date_str) is not None

def is_valid_time_format(self, time_str):
    return re.fullmatch(r"^[01]\d2[0-3]:[0-5]\d$", time_str) is not None

def is_date_time_valid(self, date_str, time_str):
    """ בודק שהתאריך לא עבר, ואם היום – שהשעה מאוחרת יותר מהשעה הנוכחית """

    if not self.is_valid_date_format(date_str):
        return False, "Invalid date format. Please enter in YYYY-MM-DD format."

    if not self.is_valid_time_format(time_str):
        return False, "Invalid time format. Please enter in HH: MM format."

    # בדיקה שהתאריך באמת תקף
    try:
        user_date = datetime.strptime(date_str, "%Y-%m-%d").date()
    except ValueError:
        return False, "Invalid date. Please enter a real date."

```

```

user_time = datetime.strptime(time_str, "%H: %M").time()

now = datetime.now()
today = now.date()
current_time = now.time()

if user_date < today:
    return False, "The selected date has already passed. Please choose a future date."

if user_date == today and user_time <= current_time:
    return False, "The selected time has already passed today. Please choose a later time."

return True, "Date and time are valid."

def create_widgets(self):
    הגדרות עיצוב בתוך הפונקציה #
    default_font = ctk.CTkFont(family="Segoe UI", size=13)
    header_font = ctk.CTkFont(family="Segoe UI", size=15, weight="bold")

    רקע חלון כללי self.root.configure(bg="#f4f4f4") #

    מסגרת חיפוש #
    search_frame = ctk.CTkFrame(self.root, fg_color="#f4f4f4")
    search_frame.pack(pady=15, fill='x')

    search_button = ctk.CTkButton(
        search_frame, text="Search Restaurants", command=self.search_restaurant,
        font=header_font, fg_color="#22c55e", hover_color="#16a34a", text_color="white",
        ירוקcorner_radius=10 #
    )
    search_button.pack(pady=10)

    מסגרת ראשית לטופס #
    form_frame = ctk.CTkFrame(self.root, fg_color="#ffffff", border_color="#dddddd",
    border_width=1,
        corner_radius=12)
    form_frame.pack(pady=10)

    left_frame = ctk.CTkFrame(form_frame, fg_color="#ffffff")
    left_frame.grid(row=0, column=0, padx=20, pady=20)

    right_frame = ctk.CTkFrame(form_frame, fg_color="#ffffff")
    right_frame.grid(row=0, column=1, padx=20, pady=20)

    שדות טופס - עמודה שמאלית #
    self._add_form_entry(left_frame, "Number of People (Required)", "people_entry",
    font=default_font)
    self._add_form_entry(left_frame, "Time (HH: MM) (Required)", "time_entry",
    font=default_font)
    self._add_date_picker(left_frame, font=default_font)

```

```

שדה מחיר - עמודה ימנית    #
    self._add_form_entry(right_frame, "Average Price per Person (Optional)", "price_entry",
font=default_font)

סוג אוכל    #
    ctk.CTkLabel(left_frame, text="Food Type (Optional)", font=default_font).pack(pady=(10,
0))
    food_types = ["No Preference", "Italian", "Sushi", "Meat", "Cafa", "Shawarma",
"Hummus", "Burgers", "Mexican",
    "Fish", "Doner"]
    self.food_listbox = ctk.CTkOptionMenu(left_frame, values=food_types, font=default_font,
corner_radius=8)
    self.food_listbox.set("No Preference")
    self.food_listbox.pack(pady=5)

מחוז    #
    ctk.CTkLabel(left_frame, text="District (Optional)", font=default_font).pack(pady=(10, 0))
    districts = ["No Preference", "Gush Dan", "Sharon", "North", "Shfela", "Southeast", "Galil",
"South"]
    self.districts_listbox = ctk.CTkOptionMenu(left_frame, values=districts, font=default_font,
corner_radius=8)
    self.districts_listbox.set("No Preference")
    self.districts_listbox.pack(pady=5)

כשרות    #
    self.kosher_var = ctk.BooleanVar()
    kosher_check = ctk.CTkCheckBox(right_frame, text="Kosher", variable=self.kosher_var,
font=default_font)
    kosher_check.pack(pady=5)

דירוג    #
    ctk.CTkLabel(right_frame, text="Rating (Optional)", font=default_font).pack(pady=(10, 0))
    self.rating_var = ctk.StringVar()
    rating_options = ["", "1", "2", "3", "4", "5"]
    self.rating_menu = ctk.CTkOptionMenu(right_frame, variable=self.rating_var,
values=rating_options,
        font=default_font, corner_radius=8)
    self.rating_menu.pack(pady=5)

העדפת ישיבה    #
    self.seating_var = ctk.StringVar(value="Not specified")
    ctk.CTkLabel(right_frame, text="Seating Preference: ", font=default_font).pack(pady=(10, 0))
    for text in ["Indoor", "Outdoor", "Not specified"]:
        ctk.CTkRadioButton(right_frame, text=text, variable=self.seating_var, value=text,
font=default_font).pack(
            anchor='w', padx=10)

כפתור צפייה בהזמנות    #
    view_orders_button = ctk.CTkButton(

```

```

        self.root, text="View Your Orders", command=self.show_user_orders,
        font=default_font, fg_color="#4a4a4a", hover_color="#666666", text_color="white",
        corner_radius=10
    )
    view_orders_button.pack(side="left", padx=10, pady=15)

    # כפתור סגירת חיבור
    close_button = ctk.CTkButton(
        self.root, text="Close Connection", command=self.close_connection,
        fg_color="#bb2d2d", hover_color="#d43f3f", text_color="white", font=default_font,
        corner_radius=10
    )
    close_button.pack(side="right", padx=10, pady=15)

    # פונקציה להוספת שדה טופס עם תווית
    def _add_form_entry(self, parent, label_text, attr_name, font):
        ctk.CTkLabel(parent, text=label_text, font=font).pack(pady=(10, 0))
        entry = ctk.CTkEntry(parent, font=font, corner_radius=8)
        entry.pack(pady=5)
        setattr(self, attr_name, entry)

    # פונקציה לבחירת תאריך בעיצוב מודרני
    def _add_date_picker(self, parent, font):
        ctk.CTkLabel(parent, text="Select Date (Required)", font=font).pack(pady=(10, 0))

        self.date_var = ctk.StringVar()
        self.date_entry = ctk.CTkEntry(parent, textvariable=self.date_var, state="readonly",
        font=font, corner_radius=8)
        self.date_entry.pack(pady=5)

    def open_calendar():
        top = ctk.CTkToplevel(self.root)
        top.title("Choose a date")
        top.geometry("280x280")
        top.lift()
        top.grab_set()
        top.focus_force()

        cal = Calendar(top, selectmode='day', date_pattern='yyyy-mm-dd')
        cal.pack(pady=20)

    def select_date():
        selected_date = cal.get_date()
        self.date_var.set(selected_date)
        top.destroy()

    select_button = ctk.CTkButton(
        top, text="Select", command=select_date,
        font=font, fg_color="#3B82F6", hover_color="#2563EB", text_color="white" #
    )
    כחול

```

```

select_button.pack(pady=10)

calendar_button = ctk.CTkButton(
    parent, text="Open Calendar", command=open_calendar,
    כחול font=font, fg_color="#3B82F6", hover_color="#2563EB", text_color="white" #
)
calendar_button.pack(pady=(0, 10))

def show_user_orders(self):
    for widget in self.root.winfo_children():
        widget.pack_forget()

    request = {
        "action": "admin_view_orders_by_user",
        "user_name": self.username
    }
    self.client_comm.send(json.dumps(request).encode('utf-8'))
    response = self.client_comm.receive()
    data = json.loads(response.encode('utf-8'))

    if data.get("status") == "error":
        messagebox.showerror("Error", data.get("message"))
        return

    orders = data.get("orders", [])
    if not orders:
        messagebox.showinfo("No Orders", "You have no past reservations.")
        return

    def parse_datetime_safe(date_str, time_str):
        try:
            return datetime.strptime(f"{date_str} {time_str}", "%Y-%m-%d %H: %M: %S")
        except ValueError:
            try:
                return datetime.strptime(f"{date_str} {time_str}", "%Y-%m-%d %H: %M")
            except ValueError as e:
                שגיאה בפירוש תאריך (e), print("
                return datetime.min

    orders.sort(key=lambda x: parse_datetime_safe(x['reservation_date'], x['reservation_time']))

    self.orders_frame = ctk.CTkFrame(self.root, fg_color="#ecf0f1")
    self.orders_frame.pack(fill="both", expand=True, padx=20, pady=20)

    title_label = ctk.CTkLabel(
        self.orders_frame,
        text="Your Orders",
        font=ctk.CTkFont(size=20, weight="bold"),
        fg_color="#2980b9",
        text_color="white",

```

```

        corner_radius=8,
        height=40
    )
    title_label.pack(pady=(10, 20), fill="x", padx=10)

    scrollable_frame = ctk.CTkScrollableFrame(self.orders_frame, width=600, height=400,
fg_color="#d6eaf8",
        corner_radius=10)
    scrollable_frame.pack(padx=10, pady=10, fill="both", expand=True)

--- מעבר על ההזמנות --- # ---
    for index, order in enumerate(orders):
        res_datetime = parse_datetime_safe(order['reservation_date'], order['reservation_time'])
        is_past = res_datetime < datetime.now()

        card_frame = ctk.CTkFrame(scrollable_frame, fg_color="ffffff", corner_radius=12)
        card_frame.pack(fill="x", pady=8, padx=10)

אפקט הוור #
        def on_enter(event, frame=card_frame):
            frame.configure(fg_color="#f0f0f0")

        def on_leave(event, frame=card_frame):
            frame.configure(fg_color="ffffff")

        card_frame.bind("<Enter>", on_enter)
        card_frame.bind("<Leave>", on_leave)

תוכן הכרטיס #
        restaurant_label = ctk.CTkLabel(
            card_frame,
            text=f"Restaurant: {order['restaurant_name']}",
            font=ctk.CTkFont(size=16, weight="bold"),
            text_color="#2c3e50",
            anchor="w"
        )
        restaurant_label.pack(anchor="w", padx=10, pady=(8, 2))

        date_label = ctk.CTkLabel(
            card_frame,
            text=f"Date: {order['reservation_date']}",
            font=ctk.CTkFont(size=14),
            text_color="#34495e",
            anchor="w"
        )
        date_label.pack(anchor="w", padx=10, pady=2)

        time_label = ctk.CTkLabel(
            card_frame,
            text=f"Time: {order['reservation_time']}",

```

```

        font=ctk.CTkFont(size=14),
        text_color="#34495e",
        anchor="w"
    )
    time_label.pack(anchor="w", padx=10, pady=(2, 8))

    # כפתור ביטול אם צריך
    if not is_past:
        cancel_button = ctk.CTkButton(
            card_frame,
            text="Cancel Reservation",
            command=lambda o=order: self.confirm_cancel_order(o),
            fg_color="#e74c3c",
            hover_color="#c0392b",
            font=ctk.CTkFont(size=12, weight="bold"),
            corner_radius=8,
            height=32
        )
        cancel_button.pack(anchor="e", padx=10, pady=(0, 8))

    # --- אנימציה קטנה) שהכרטיסים יכנסו באיחור קטן אחד אחרי השני--- )
    self.root.update()
    time.sleep(0.05) # דיליי קטן בין כרטיסים

    # כפתור חזור
    def go_back():
        self.orders_frame.destroy()
    # חזרה למסך הראשי
    self.create_widgets()

    back_button = ctk.CTkButton(
        self.orders_frame,
        text="Back",
        command=go_back,
        fg_color="#2E8B57",
        hover_color="#276749",
        font=ctk.CTkFont(size=14, weight="bold"),
        corner_radius=10,
        height=40
    )
    back_button.pack(pady=20)

    def confirm_cancel_order(self, order):
        answer = messagebox.askyesno(
            "Cancel Order",
            f"Are you sure you want to cancel your reservation at {order['restaurant_name']} "
            f"on {order['reservation_date']} at {order['reservation_time']}?"
        )
        if not answer:
            return

```



```
cancel_request = {
    "action": "cancel_reservation",
    "reservation_id": order['reservation_id']
}
self.client_comm.send(json.dumps(cancel_request).encode('utf-8'))
response = self.client_comm.receive()
data = json.loads(response.encode('utf-8'))

if data.get("status") == "success":
    threading.Thread(target=self.send_cancelled_reservation_thread, args=(order,),
daemon=True).start()
    messagebox.showinfo("Cancelled", "Reservation cancelled successfully. A confirmation
email was sent.")
    self.orders_frame.destroy()
    self.show_user_orders()
else:
    messagebox.showerror("Error", data.get("message"))

def send_cancelled_reservation_thread(self, order):
    try:
        if self.user_email and self.mail_sender:
            self.mail_sender.send_cancel_confirmation(self.user_email, order)
    except Exception as e:
        print(f'Failed to send cancellation email: {e}')
        messagebox.showwarning("Email Error",
            "The reservation was cancelled, but we couldn't send the confirmation
email.")

def search_restaurant(self):
    if not self.client_comm.socket:
        messagebox.showerror("Connection Error", "No connection to the server.")
        return

    num_people = self.people_entry.get().strip()
    time = self.time_entry.get().strip()
    average_price_per_person = self.price_entry.get().strip()
    date = self.date_entry.get().strip()

    if not num_people or not time or not date:
        messagebox.showerror("Input Error", "You must enter the number of people, the time and
the date.")
        return

    valid_date_time, date_time_message = self.is_date_time_valid(date, time)
    if not valid_date_time:
        messagebox.showerror("Input Error", date_time_message)
        return
    if not num_people.isdigit():
        messagebox.showerror("Input Error", "The number of people must be a whole number.")
        return
```

```

food_type = self.food_listbox.get()
food_types = [] if food_type == "No Preference" else [food_type]

district = self.districts_listbox.get()
districts = [] if district == "No Preference" else [district]
kosher = self.kosher_var.get()
rating = self.rating_var.get()
seating = self.seating_var.get()

request_data = {
    'action': 'search',
    'table_size': num_people,
    'time': time,
    'date': date,
    'food_types': food_types,
    'districts': districts,
    'price': average_price_per_person,
    'kosher': kosher,
    'rating': rating,
    'category': seating
}
print("Sending request data: ", request_data)
self.client_comm.send(json.dumps(request_data).encode('utf-8'))

# Receiving the response from the server
response = self.client_comm.receive()
if response:
    try:
        restaurants = json.loads(response)
        print(restaurants)
        if restaurants:
            self.show_restaurants(restaurants)
        else:
            messagebox.showinfo("No Results", "No restaurants found matching your criteria.")
    except Exception as e:
        messagebox.showerror("JSON Error", f"Failed to decode the response: {e}")
else:
    messagebox.showerror("Server Error", "Failed to get a response from the server.")

def show_restaurants(self, restaurants):
    new_window = ctk.CTkToplevel(self.root)
    new_window.title("Search Results")
    new_window.geometry("500x600")
    רקע בהיר    new_window.configure(fg_color="#ffffff") #

    new_window.lift()
    new_window.grab_set()
    new_window.focus_force()

```

```

כותרת #
    ctk.CTkLabel(new_window, text="Restaurants found: ", font=("Segoe UI", 18, "bold"),
text_color="#2c3e50").pack(
    pady=20)

מסגרת גלילה #
    scroll_frame = ctk.CTkScrollableFrame(new_window, width=460, height=400,
fg_color="#ecf0f1")
    scroll_frame.pack(pady=10, padx=10, fill="both", expand=True)

    self.selected_restaurant_var = ctk.IntVar(value=-1)
    self.restaurants_mapping = { }

    for index, restaurant in enumerate(restaurants):
        name = restaurant.get('name')
        availability = restaurant.get('seating_availability')
        display_text = f'{ name } - { availability }'

        radio = ctk.CTkRadioButton(scroll_frame, text=display_text,
variable=self.selected_restaurant_var,
            value=index, font=("Segoe UI", 14), radiobutton_height=25,
            text_color="#34495e", hover_color="#bdc3c7")
        radio.pack(anchor="w", padx=20, pady=8)

        self.restaurants_mapping[index] = restaurant

כפתור המשך #
    confirm_button = ctk.CTkButton(new_window, text="Continue",
command=self.on_restaurant_select,
            font=("Segoe UI", 14), height=45, width=150, corner_radius=12,
            fg_color="#3498db", hover_color="#2980b9")
    confirm_button.pack(pady=20)

def on_restaurant_select(self):
    selected_index = self.selected_restaurant_var.get()
    if selected_index == -1:
        messagebox.showerror("Selection Error", "Please select a restaurant.")
        return

    selected_restaurant = self.restaurants_mapping[selected_index]
    self.restaurant = selected_restaurant.get('name')
    self.available_seating = selected_restaurant.get('seating_availability')

    get_info = { 'restaurant': self.restaurant, 'action': 'restaurant_info' }
    self.client_comm.send(json.dumps(get_info).encode('utf-8'))

    response = self.client_comm.receive()
    data = json.loads(response.encode('utf-8'))

    if data.get('status') == "success":

```

```

self.city = data.get("city")
self.food_type = data.get("food_type")
self.rating = data.get("rating")

=====חלון מידע=====
info_window = ctk.CTkToplevel(self.root)
info_window.title("Restaurant Info")
info_window.geometry("470x500")
info_window.configure(fg_color="#ecf0f1")

info_window.lift()
info_window.grab_set()
info_window.focus_force()

=====לוגו=====
try:
    logo_image = Image.open("logo.jpeg")
    logo_ctk_image = ctk.CTkImage(light_image=logo_image, size=(100, 100))
    logo_label = ctk.CTkLabel(info_window, text="", image=logo_ctk_image)
    logo_label.image = logo_ctk_image
    logo_label.pack(pady=(15, 10))
except Exception as e:
    print("בעיה בטעינת הלוגו(e) ")

=====כותרת=====
ctk.CTkLabel(info_window,
             text=f"Details for {self.restaurant}",
             font=ctk.CTkFont(size=20, weight="bold"),
             text_color="#2c3e50").pack(pady=(0, 20))

=====פרטי מסעדה=====
info_frame = ctk.CTkFrame(info_window, fg_color="#ffffff", corner_radius=10)
info_frame.pack(padx=30, pady=10, fill="x")

info_texts = [
    ("📍 City: ", self.city),
    ("🍽️ Food Type: ", self.food_type),
    ("★ Rating: ", str(self.rating))
]

for label, value in info_texts:
    row = ctk.CTkFrame(info_frame, fg_color="transparent")
    row.pack(anchor="w", pady=8, padx=10)
    ctk.CTkLabel(row, text=label, font=ctk.CTkFont(size=14, weight="bold"),
                 text_color="#34495e").pack(side="left")
    ctk.CTkLabel(row, text=value, font=ctk.CTkFont(size=14),
                 text_color="#7f8c8d").pack(side="left",
                                             padx=10)

```

```

=====כפתור המשך # =====
        ctk.CTkButton(info_window,
            text="Continue",
            command=self.seating_category,
            font=ctk.CTkFont(size=16, weight="bold"),
            height=48,
            width=160,
            corner_radius=12,
            fg_color="#3498db",
            hover_color="#2980b9",
            text_color="white").pack(pady=30)
    else:
        messagebox.showerror("Error", "Restaurant doesn't exist")

def seating_category(self):
    if self.seating_var.get() != "Not specified":
        self.show_order_table_button()
    else:
        if self.available_seating == "Indoor and Outdoor":
            self.show_seating_preference_window()
        else:
            if self.available_seating == "Indoor":
                self.seating_var.set("Indoor")
            elif self.available_seating == "Outdoor":
                self.seating_var.set("Outdoor")
            self.show_order_table_button()

def show_seating_preference_window(self):
    seating_window = ctk.CTkToplevel(self.root)
    seating_window.title("Select Seating Preference")
    seating_window.geometry("350x250")
    seating_window.configure(fg_color="#ecf0f1")

    seating_window.lift()
    seating_window.grab_set()
    seating_window.focus_force()

    def on_close():
        self.seating_var.set("Not specified")
        seating_window.destroy()

    seating_window.protocol("WM_DELETE_WINDOW", on_close)

    ctk.CTkLabel(seating_window, text=f"Select seating for {self.restaurant}:", font=("Segoe
UI", 15),
        text_color="#2c3e50").pack(pady=20)

    ctk.CTkRadioButton(seating_window, text="Indoor", variable=self.seating_var,
value="Indoor",
        font=("Segoe UI", 14), text_color="#34495e").pack(pady=5)

```

```

        ctk.CTkRadioButton(seating_window, text="Outdoor", variable=self.seating_var,
                           value="Outdoor",
                           font=("Segoe UI", 14), text_color="#34495e").pack(pady=5)

        ctk.CTkButton(seating_window, text="Confirm", command=self.show_order_table_button,
                      font=("Segoe UI", 14), height=45, width=140, corner_radius=12, fg_color="#3498db",
                      hover_color="#2980b9").pack(pady=20)

    def show_order_table_button(self):
        order_window = ctk.CTkToplevel(self.root)
        order_window.title("Order Your Table")
        order_window.geometry("500x520")
        order_window.configure(fg_color="#f5f5f5")

        order_window.lift()
        order_window.grab_set()
        order_window.focus_force()

===== לוגו      # =====
    try:
        logo_image = Image.open("logo.jpeg")
        logo_ckpt_image = ctk.CTkImage(light_image=logo_image, size=(100, 100))
        logo_label = ctk.CTkLabel(order_window, text="", image=logo_ckpt_image)
        logo_label.image = logo_ckpt_image
        logo_label.pack(pady=(15, 10))
    except Exception as e:
        print(":", e)

===== כותרת      # =====
        ctk.CTkLabel(order_window,
                      text=f"Reserve a table at {self.restaurant}",
                      font=ctk.CTkFont(size=20, weight="bold"),
                      text_color="#333333").pack(pady=(0, 20))

===== מידע על ההזמנה      # =====
        info_frame = ctk.CTkFrame(order_window, fg_color="#ffffff", corner_radius=10)
        info_frame.pack(padx=30, pady=10, fill="x")

        info_texts = [
            ("📅 Date: ", self.date_entry.get()),
            ("🕒 Time: ", self.time_entry.get()),
            ("👥 People: ", self.people_entry.get()),
            ("🪑 Seating: ", self.seating_var.get())
        ]

        for label, value in info_texts:
            row = ctk.CTkFrame(info_frame, fg_color="transparent")
            row.pack(anchor="w", pady=5, padx=10)
            ctk.CTkLabel(row, text=label, font=ctk.CTkFont(size=14, weight="bold"),

```

```

text_color="#444444").pack(
    side="left")
    ctk.CTkLabel(row, text=value, font=ctk.CTkFont(size=14),
text_color="#666666").pack(side="left", padx=10)

=====כפתור הזמנה=====
    ctk.CTkButton(order_window,
        text="Book Table",
        command=self.show_order_confirmation_window,
        font=ctk.CTkFont(size=16, weight="bold"),
        height=48,
        width=180,
        corner_radius=12,
        fg_color="#4CAF50",
        hover_color="#45a049",
        text_color="white").pack(pady=30)

def show_order_confirmation_window(self):
    request_order = {
        'action': 'order',
        'restaurant': self.restaurant,
        'category': self.seating_var.get(),
        'table_size': self.people_entry.get(),
        'reservation_time': self.time_entry.get(),
        'date': self.date_entry.get(),
        'user_id': self.user_id
    }

    self.client_comm.send(json.dumps(request_order).encode('utf-8'))
    response = self.client_comm.receive()

    if response:
        try:
            data = json.loads(response)
            status = data.get("status")

            if status == "success":
                raw_end_time = data.get("end_reservation")
                request_order["end_reservation"] = raw_end_time[: 5] if raw_end_time else None
                threading.Thread(
                    target=self.send_confirmed_reservation_thread,
                    args=(request_order,),
                    daemon=True
                ).start()

                confirmation_window = ctk.CTkToplevel(self.root)
                confirmation_window.title("Reservation Confirmed")
                confirmation_window.geometry("500x580")
                confirmation_window.configure(fg_color="#f5f5f5")

```

```

confirmation_window.lift()
confirmation_window.grab_set()
confirmation_window.focus_force()

לוגו      #
try:
    logo_image = Image.open("logo.jpeg").resize((100, 100))
    logo_ctk_image = ctk.CTkImage(light_image=logo_image, size=(100, 100))
    ctk.CTkLabel(confirmation_window, text="",
image=logo_ctk_image).pack(pady=(15, 10))
except Exception as e:
: ", e) בטעינת הלוגו) print("

כותרת      #
    ctk.CTkLabel(
        confirmation_window,
        text="Reservation Confirmed!",
        font=ctk.CTkFont(size=20, weight="bold"),
        text_color="#333333"
    ).pack(pady=(0, 15))

מסגרת מידע      #
    info_frame = ctk.CTkFrame(confirmation_window, fg_color="#ffffff",
corner_radius=10)
    info_frame.pack(padx=30, pady=10, fill="x")

    info_texts = [
        ("🍽️ Restaurant: ", self.restaurant),
        ("📅 Date: ", self.date_entry.get()),
        ("🕒 Time: ", f"{self.time_entry.get()} - {request_order.get('end_reservation')}"),
        ("👥 People: ", self.people_entry.get()),
        ("🪑 Seating: ", self.seating_var.get())
    ]

    for label, value in info_texts:
        row = ctk.CTkFrame(info_frame, fg_color="transparent")
        row.pack(anchor="w", pady=5, padx=10)
        ctk.CTkLabel(row, text=label, font=ctk.CTkFont(size=14, weight="bold"),
            text_color="#444444").pack(side="left")
        ctk.CTkLabel(row, text=value, font=ctk.CTkFont(size=14),
text_color="#666666").pack(side="left",
                                padx=10)

טקסט אישור      #
    ctk.CTkLabel(
        confirmation_window,
        text="A confirmation email has been sent to your inbox.",
        font=ctk.CTkFont(size=13),
        text_color="#666666"

```



```

        ).pack(pady=(15, 10))

# כפתור סגירה
    ctk.CTkButton(
        confirmation_window,
        text="Close",
        command=self.close_application,
        font=ctk.CTkFont(size=16, weight="bold"),
        height=48,
        width=180,
        corner_radius=12,
        fg_color="#3B82F6",
        hover_color="#2563EB",
        text_color="white"
    ).pack(pady=20)

    else:
        messagebox.showinfo("Error", "There was an issue with your reservation.")
    except json.JSONDecodeError:
        messagebox.showinfo("Error", "Failed to decode response from server.")
    else:
        messagebox.showinfo("Error", "No response received from server.")

def send_confirmed_reservation_thread(self, order):
    try:
        if self.user_email and self.mail_sender:
            self.mail_sender.send_reservation_confirmation(
                recipient_email=self.user_email,
                restaurant=order['restaurant'],
                city=order.get('city'), # אם אין עיר – ברירת מחדל
                date=order['date'],
                start_time=order['reservation_time'],
                end_time = order['end_reservation'],
                people=order['table_size'],
                seating_category=order['category']
            )
        except Exception as e:
            print(f"Failed to send confirmation email: {e}")

def open_admin_panel(self):
    admin_window = ctk.CTkToplevel(self.root)
    admin_window.title("Admin Panel")
    admin_window.geometry("500x650")
    admin_window.configure(fg_color="#f0f2f5")

    admin_window.lift()
    admin_window.grab_set()
    admin_window.focus_force()

=== לוגו בראש === # ===

```

```

try:
    logo_image = Image.open("logo.jpeg")
    logo_ckpt_image = ctk.CTkImage(light_image=logo_image, size=(100, 100))

    logo_label = ctk.CTkLabel(admin_window, text="", image=logo_ckpt_image)
    logo_label.image = logo_ckpt_image
    logo_label.pack(pady=(20, 10))
except Exception as e:
    print("בעיה בטעינת הלוגו(e) : ")

=== כותרת # ===
    ctk.CTkLabel(
        admin_window,
        text="Admin Panel",
        font=ctk.CTkFont(size=22, weight="bold"),
        text_color="#2c3e50"
    ).pack(pady=(0, 20))

=== מסגרת לכל הכפתורים # ===
    button_frame = ctk.CTkFrame(admin_window, fg_color="white", corner_radius=15)
    button_frame.pack(pady=10, padx=25, fill="both", expand=True)

=== רשימת כפתורים עם צבעים מעוצבים # ===
    buttons = [
        ("View Orders by Date", self.open_date_selection_window, "#3498db", "#2980b9"),
        ("View Orders by User", self.view_orders_by_user, "#2ecc71", "#27ae60"),
        ("View Most Popular Restaurants", self.view_popular_restaurants, "#f39c12", "#e67e22"),
        ("View Most Active Users", self.view_active_users, "#9b59b6", "#8e44ad"),
        ("Edit Restaurant", self.edit_restaurants, "#e74c3c", "#c0392b"),
        ("Edit Tables", self.edit_tables, "#95a5a6", "#7f8c8d"),
    ]

    for text, command, fg_color, hover_color in buttons:
        ctk.CTkButton(
            button_frame,
            text=text,
            command=command,
            font=ctk.CTkFont(size=15),
            height=45,
            corner_radius=12,
            fg_color=fg_color,
            hover_color=hover_color
        ).pack(pady=10, padx=20, fill="x")

=== כפתור סגירה קטן בתחתית # ===
    ctk.CTkButton(
        admin_window,
        text="Close",
        command=admin_window.destroy,
        font=ctk.CTkFont(size=13),

```

```

        width=100,
        fg_color="#bdc3c7",
        hover_color="#95a5a6",
        text_color="black"
    ).pack(pady=20)

def open_date_selection_window(self):
    date_window = ctk.CTkToplevel(self.root)
    date_window.title("Select Date")
    date_window.geometry("450x550")
    date_window.configure(fg_color="#eaf2f8")

    date_window.lift()
    date_window.grab_set()
    date_window.focus_force()

    # כפתור סגירה
    close_button = ctk.CTkButton(
        date_window, text="X", width=30, height=30,
        command=date_window.destroy, fg_color="transparent",
        text_color="#7f8c8d", hover_color="#dcdde1"
    )
    close_button.place(relx=0.96, rely=0.02, anchor="ne")

    # לוגו
    try:
        logo_image = Image.open("logo.jpeg")
        logo_ctk_image = ctk.CTkImage(light_image=logo_image, size=(90, 90))
        ctk.CTkLabel(date_window, text="", image=logo_ctk_image).pack(pady=(15, 5))
    except Exception as e:
        print("בעיה בטעינת הלוגו", e)

    # כותרת
    ctk.CTkLabel(
        date_window,
        text="View Orders by Date",
        font=ctk.CTkFont(size=22, weight="bold"),
        text_color="#2c3e50"
    ).pack()

    # תיאור
    ctk.CTkLabel(
        date_window,
        text="Select a date to display all reservations made on that day.",
        font=ctk.CTkFont(size=14),
        text_color="#34495e"
    ).pack(pady=(5, 15))

    # מסגרת עם לוח שנה
    content_frame = ctk.CTkFrame(date_window, fg_color="white", corner_radius=12)

```

```
content_frame.pack(padx=25, pady=10, fill="both", expand=True)

calendar = Calendar(content_frame, date_pattern="yyyy-mm-dd", selectmode="day")
calendar.pack(pady=25)

def on_date_confirm():
    selected_date = calendar.get_date()
    request_data = {
        "action": "admin_view_orders_by_date",
        "date": selected_date
    }
    self.client_comm.send(json.dumps(request_data).encode("utf-8"))
    response = self.client_comm.receive()

    if response:
        try:
            response_data = json.loads(response)
            if response_data.get('status') == 'success':
                orders = response_data.get('orders', [])
                self.show_orders_result_window(orders, selected_date)
            else:
                messagebox.showerror("Error", "No orders on this date.")
        except json.JSONDecodeError:
            messagebox.showerror("Error", "Failed to decode server response.")
    else:
        messagebox.showerror("Error", "No response from server.")

    ctk.CTkButton(
        date_window,
        text="Show Orders",
        command=on_date_confirm,
        font=ctk.CTkFont(size=15),
        fg_color="#2980b9", hover_color="#1f618d",
        text_color="white", width=150, height=40, corner_radius=12
    ).pack(pady=20)

def show_orders_result_window(self, orders, date):
    results_window = ctk.CTkToplevel(self.root)
    results_window.title(f"Orders on {date}")
    results_window.geometry("550x500")
    results_window.configure(bg="#ecf0f1")

    results_window.lift()
    results_window.grab_set()
    results_window.focus_force()

    if not orders:
        ctk.CTkLabel(results_window, text=f"No orders found on {date}.",
            font=ctk.CTkFont(size=16, weight="bold"), fg_color="#e74c3c",
            text_color="white",
```

```

        corner_radius=8, height=40).pack(pady=20, fill="x", padx=10)
    return

    # כותרת
    ctk.CTkLabel(results_window, text=f"Orders on {date}:", font=ctk.CTkFont(size=20,
weight="bold"),
        fg_color="#2980b9", text_color="white", corner_radius=8, height=40).pack(fill="x",
pady=(10, 5),
                                padx=10)

    # מסגרת עם גלילה
    scroll_frame = ctk.CTkScrollableFrame(results_window, width=500, height=400,
fg_color="#d6eaf8",
                                corner_radius=10)
    scroll_frame.pack(padx=10, pady=10, fill="both", expand=True)

    for order in orders:
        # כרטיס להזמנה
        card_frame = ctk.CTkFrame(scroll_frame, fg_color="ffffff", corner_radius=12)
        card_frame.pack(fill="x", pady=8, padx=10)

        user_label = ctk.CTkLabel(card_frame, text=f"User: {order['user_id']}",
                                font=ctk.CTkFont(size=16, weight="bold"),
                                text_color="#2c3e50", anchor="w")
        user_label.pack(anchor="w", padx=10, pady=(8, 2))

        restaurant_label = ctk.CTkLabel(card_frame, text=f"Restaurant:
{order['restaurant_name']}",
                                font=ctk.CTkFont(size=14),
                                text_color="#34495e", anchor="w")
        restaurant_label.pack(anchor="w", padx=10, pady=2)

        time_label = ctk.CTkLabel(card_frame, text=f"Time: {order['reservation_time']}",
                                font=ctk.CTkFont(size=14),
                                text_color="#34495e", anchor="w")
        time_label.pack(anchor="w", padx=10, pady=(2, 8))

    def view_orders_by_user(self):
        user_window = ctk.CTkToplevel(self.root)
        user_window.title("View Orders by User")
        user_window.geometry("450x420")
        user_window.configure(fg_color="#eaf2f8")

        user_window.lift()
        user_window.grab_set()
        user_window.focus_force()

    # כפתור סגירה בפינה
    close_button = ctk.CTkButton(
        user_window, text="X", width=30, height=30,

```

```

        command=user_window.destroy, fg_color="transparent",
        text_color="#7f8c8d", hover_color="#dcdde1"
    )
    close_button.place(relx=0.96, rely=0.02, anchor="ne")

לוגו    #
    try:
        logo_image = Image.open("logo.jpeg")
        logo_ctk_image = ctk.CTkImage(light_image=logo_image, size=(90, 90))
        ctk.CTkLabel(user_window, text="", image=logo_ctk_image).pack(pady=(15, 5))
    except Exception as e:
        שגיאה בטעינת הלוגו(e) : "    print("

כותרת    #
    ctk.CTkLabel(
        user_window,
        text="Search Orders by User",
        font=ctk.CTkFont(size=22, weight="bold"),
        text_color="#2c3e50"
    ).pack()

תיאור    #
    ctk.CTkLabel(
        user_window,
        text="Enter a username to see all their reservations.",
        font=ctk.CTkFont(size=14),
        text_color="#34495e"
    ).pack(pady=(5, 15))

מסגרת קלט    #
    input_frame = ctk.CTkFrame(user_window, fg_color="white", corner_radius=12)
    input_frame.pack(padx=25, pady=10, fill="both", expand=False)

    user_name_entry = ctk.CTkEntry(input_frame, placeholder_text="User Name",
font=ctk.CTkFont(size=14), width=250)
    user_name_entry.pack(pady=30)

    def on_user_name_confirm():
        user_name = user_name_entry.get()
        if not user_name:
            messagebox.showerror("Error", "Please enter a User Name.")
            return

        request_data = {
            "action": "admin_view_orders_by_user",
            "user_name": user_name
        }
        self.client_comm.send(json.dumps(request_data).encode("utf-8"))
        response = self.client_comm.receive()
        if response:

```

```

try:
    response_data = json.loads(response)
    if response_data.get('status') == 'success':
        orders = response_data.get('orders', [])
        self.show_user_orders_result_window(orders, user_name)
    else:
        messagebox.showerror("Error", response_data.get("message"))
except json.JSONDecodeError:
    messagebox.showerror("Error", "Failed to decode server response.")
else:
    messagebox.showerror("Error", "No response from server.")

    ctk.CTkButton(
        user_window,
        text="Show Orders",
        command=on_user_name_confirm,
        font=ctk.CTkFont(size=15),
        fg_color="#27ae60", hover_color="#229954",
        width=150, height=40, corner_radius=12
    ).pack(pady=20)

def show_user_orders_result_window(self, orders, user_name):
    results_window = ctk.CTkToplevel(self.root)
    results_window.title(f"Orders for User {user_name}")
    results_window.geometry("550x500")
    results_window.configure(bg="#ecf0f1")

    results_window.lift()
    results_window.grab_set()
    results_window.focus_force()

    if not orders:
        ctk.CTkLabel(results_window, text=f"No orders found for User {user_name}.",
            font=ctk.CTkFont(size=16, weight="bold"), fg_color="#e74c3c").pack(pady=20)
        return

    כותרת #
    ctk.CTkLabel(results_window, text=f"Orders for User {user_name}:",
        font=ctk.CTkFont(size=20, weight="bold"),
        fg_color="#2980b9", text_color="white", corner_radius=8, height=40).pack(fill="x",
        pady=(10, 5),
                                padx=10)

    מסגרת עם גלילה #
    scroll_frame = ctk.CTkScrollableFrame(results_window, width=500, height=400,
        fg_color="#d6eaf8",
                                corner_radius=10)
    scroll_frame.pack(padx=10, pady=10, fill="both", expand=True)

    for order in orders:

```

```

כרטיס להזמנה      #
    card_frame = ctk.CTkFrame(scroll_frame, fg_color="#ffffff", corner_radius=12)
    card_frame.pack(fill="x", pady=8, padx=10)

    restaurant_label = ctk.CTkLabel(card_frame, text=f'Restaurant:
{order["restaurant_name"]}',
                                   font=ctk.CTkFont(size=16, weight="bold"),
                                   text_color="#2c3e50", anchor="w")
    restaurant_label.pack(anchor="w", padx=10, pady=(8, 2))

    date_label = ctk.CTkLabel(card_frame, text=f'Date: {order["reservation_date"]}',
                              font=ctk.CTkFont(size=14),
                              text_color="#34495e", anchor="w")
    date_label.pack(anchor="w", padx=10, pady=2)

    time_label = ctk.CTkLabel(card_frame, text=f'Time: {order["reservation_time"]}',
                              font=ctk.CTkFont(size=14),
                              text_color="#34495e", anchor="w")
    time_label.pack(anchor="w", padx=10, pady=(2, 8))

def view_popular_restaurants(self):
שליחת בקשה לשרת      #
    request_data = {
        "action": "admin_view_popular_restaurants"
    }
    self.client_comm.send(json.dumps(request_data).encode("utf-8"))
    response = self.client_comm.receive()

    if response:
        try:
            response_data = json.loads(response)
            if response_data.get("status") == "success":
                restaurants = response_data.get("restaurants", [])
                self.show_popular_restaurants_window(restaurants)
            else:
                messagebox.showerror("Error",
                                     response_data.get("message", "Failed to retrieve popular restaurants."))
        except json.JSONDecodeError:
            messagebox.showerror("Error", "Failed to decode server response.")
        else:
            messagebox.showerror("Error", "No response from server.")

def show_popular_restaurants_window(self, restaurants):
    window = ctk.CTkToplevel(self.root)
    window.title("Top 5 Popular Restaurants")
    window.geometry("520x550")
    window.configure(fg_color="#eaf2f8")

    window.lift()
    window.grab_set()

```



```

window.focus_force()

# כפתור סגירה בפינה
close_button = ctk.CTkButton(
    window, text="X", width=30, height=30,
    command=window.destroy, fg_color="transparent",
    text_color="#7f8c8d", hover_color="#dcdde1"
)
close_button.place(relx=0.96, rely=0.02, anchor="ne")

# לוגו
try:
    logo_image = Image.open("logo.jpeg")
    logo_ctk_image = ctk.CTkImage(light_image=logo_image, size=(90, 90))
    ctk.CTkLabel(window, text="", image=logo_ctk_image).pack(pady=(15, 5))
except Exception as e:
    print("בעיה בטעינת הלוגו(e) :", e)

# כותרת
ctk.CTkLabel(
    window,
    text="Top 5 Popular Restaurants",
    font=ctk.CTkFont(size=22, weight="bold"),
    text_color="#2c3e50"
).pack()

# תיאור
ctk.CTkLabel(
    window,
    text="These restaurants received the most reservations",
    font=ctk.CTkFont(size=14),
    text_color="#34495e"
).pack(pady=(5, 15))

content_frame = ctk.CTkFrame(window, fg_color="white", corner_radius=12)
content_frame.pack(padx=25, pady=10, fill="both", expand=True)

if not restaurants:
    ctk.CTkLabel(
        content_frame,
        text="No restaurant data found.",
        font=ctk.CTkFont(size=14, weight="bold"),
        text_color="#e74c3c"
    ).pack(pady=30)
else:
    for i, res in enumerate(restaurants, 1):
        res_text = f"{i}. {res['name']} - {res['reservation_count']} reservations"
        row = ctk.CTkFrame(content_frame, fg_color="#f8f9f9", corner_radius=10)
        row.pack(fill="x", pady=5, padx=10)

```

```

        icon = ctk.CTkLabel(row, text="🔒", font=ctk.CTkFont(size=18))
        icon.pack(side="left", padx=10, pady=8)

        label = ctk.CTkLabel(row, text=res_text, font=ctk.CTkFont(size=15),
text_color="#2c3e50")
        label.pack(side="left", padx=5)

# כפתור חזרה
        ctk.CTkButton(
            window,
            text="Back to Admin Panel",
            command=window.destroy,
            font=ctk.CTkFont(size=14),
            fg_color="#95a5a6", hover_color="#7f8c8d",
            text_color="white", width=180, height=40, corner_radius=15
        ).pack(pady=15)

def view_active_users(self):
    request_data = {
        "action": "admin_view_active_users"
    }
    self.client_comm.send(json.dumps(request_data).encode("utf-8"))
    response = self.client_comm.receive()

    if response:
        try:
            response_data = json.loads(response)
            if response_data.get('status') == 'success':
                users = response_data.get('users', [])
                self.show_active_users_result_window(users)
            else:
                messagebox.showerror("Error", response_data.get("message", "Failed to retrieve
active users."))
        except json.JSONDecodeError:
            messagebox.showerror("Error", "Failed to decode server response.")
        else:
            messagebox.showerror("Error", "No response from server.")

def show_active_users_result_window(self, users):
    results_window = ctk.CTkToplevel(self.root)
    results_window.title("Most Active Users")
    results_window.geometry("520x550")
    רקע קצת פחות לבן results_window.configure(fg_color="#eaf2f8") #

    results_window.lift()
    results_window.grab_set()
    results_window.focus_force()

# כפתור סגירה בפינה
        close_button = ctk.CTkButton(

```

```

        results_window, text="X", width=30, height=30,
        command=results_window.destroy, fg_color="transparent",
        text_color="#7f8c8d", hover_color="#dcdde1"
    )
    close_button.place(relx=0.96, rely=0.02, anchor="ne")

    # לוגו
    try:
        logo_image = Image.open("logo.jpeg")
        logo_ckpt_image = ctk.CTkImage(light_image=logo_image, size=(90, 90))
        ctk.CTkLabel(results_window, text="", image=logo_ckpt_image).pack(pady=(15, 5))
    except Exception as e:
        print("בעיה בטעינת הלוגו(e) :", e)

    # כותרת
    ctk.CTkLabel(
        results_window,
        text="Top 5 Active Users",
        font=ctk.CTkFont(size=22, weight="bold"),
        text_color="#2c3e50"
    ).pack()

    # תיאור קצר
    ctk.CTkLabel(
        results_window,
        text="Below are the users with the most reservations",
        font=ctk.CTkFont(size=14),
        text_color="#34495e"
    ).pack(pady=(5, 15))

    content_frame = ctk.CTkFrame(results_window, fg_color="white", corner_radius=12)
    content_frame.pack(padx=25, pady=10, fill="both", expand=True)

    if not users:
        ctk.CTkLabel(
            content_frame,
            text="No active users found.",
            font=ctk.CTkFont(size=14, weight="bold"),
            text_color="#e74c3c"
        ).pack(pady=30)
    else:
        for i, user in enumerate(users, 1):
            user_name = user['user_name']
            reservation_count = user['reservation_count']
            user_text = f'{i}. {user_name} - {reservation_count} reservations'
            row = ctk.CTkFrame(content_frame, fg_color="#f8f9f9", corner_radius=10)
            row.pack(fill="x", pady=5, padx=10)

            icon = ctk.CTkLabel(row, text="👤", font=ctk.CTkFont(size=18))
            icon.pack(side="left", padx=10, pady=8)

```

```

        label = ctk.CTkLabel(row, text=user_text, font=ctk.CTkFont(size=15),
text_color="#2c3e50")
        label.pack(side="left", padx=5)

```

```

כפתור סיום למטה גם כן      #
        ctk.CTkButton(
            results_window,
            text="Back to Admin Panel",
            command=results_window.destroy,
            font=ctk.CTkFont(size=14),
            fg_color="#95a5a6", hover_color="#7f8c8d",
            text_color="white", width=180, height=40, corner_radius=15
        ).pack(pady=15)

```

```

def edit_restaurants(self):
    window = ctk.CTkToplevel(self.root)
    window.title("Edit Restaurant")
    window.geometry("400x300")
    window.configure(bg="#f9f9f9")

```

```

    window.lift()
    window.grab_set()
    window.focus_force()

```

```

        ctk.CTkLabel(window, text="Edit Restaurant", font=ctk.CTkFont(size=20,
weight="bold")).pack(pady=30)

```

```

        ctk.CTkButton(
            window,
            text="Add Restaurant",
            command=self.add_restaurant,
ירוק        fg_color="#2ecc71", #
            hover_color="#27ae60",
            font=ctk.CTkFont(size=14),
            width=200,
            height=40
        ).pack(pady=10)

```

```

        ctk.CTkButton(
            window,
            text="Remove Restaurant",
            command=self.delete_restaurant,
אדום        fg_color="#e74c3c", #
            hover_color="#c0392b",
            font=ctk.CTkFont(size=14),
            width=200,
            height=40
        ).pack(pady=10)

```

```

def add_restaurant(self):
    add_window = ctk.CTkToplevel(self.root)
    add_window.title("Add New Restaurant")
    add_window.geometry("600x700")
    add_window.configure(bg="#ecf0f1")

    add_window.lift()
    add_window.grab_set()
    add_window.focus_force()

    ctk.CTkLabel(add_window, text="Enter Restaurant Details", font=ctk.CTkFont(size=18,
weight="bold")).grid(row=0,
                                                                column=0,
                                                                colspan=2,
                                                                pady=20)

    entries = { }
    fields = [
        ("Name", "name"),
        ("Food Type", "food_type"),
        ("City", "city"),
        ("District", "district"),
        ("Average Price Per Person", "average_price_per_person"),
        ("Rating (1-5)", "rating")
    ]

# יצירת 2 טורים
    row = 1
    for label_text, key in fields:
        ctk.CTkLabel(add_window, text=label_text).grid(row=row, column=0, padx=10, pady=5,
sticky="w")
        entry = ctk.CTkEntry(add_window, placeholder_text=label_text)
        entry.grid(row=row, column=1, padx=10, pady=5, sticky="w")
        entries[key] = entry
        row += 1

# שדות זמן פתיחה וסגירה
    ctk.CTkLabel(add_window, text="Opening Time (HH: MM)").grid(row=row, column=0,
padx=10, pady=5, sticky="w")
    opening_time_entry = ctk.CTkEntry(add_window, placeholder_text="Opening Time")
    opening_time_entry.grid(row=row, column=1, padx=10, pady=5, sticky="w")
    row += 1

    ctk.CTkLabel(add_window, text="Closing Time (HH: MM)").grid(row=row, column=0,
padx=10, pady=5, sticky="w")
    closing_time_entry = ctk.CTkEntry(add_window, placeholder_text="Closing Time")
    closing_time_entry.grid(row=row, column=1, padx=10, pady=5, sticky="w")
    row += 1

# זמן שליחת ההזמנה

```

```

entries["opening_time"] = opening_time_entry
entries["closing_time"] = closing_time_entry

שדה הזמנת משך השולחן #
    ctk.CTkLabel(add_window, text="Table Reservation Duration (minutes)").grid(row=row,
column=0, padx=10, pady=5,
                                sticky="w")
    table_reservation_duration_entry = ctk.CTkEntry(add_window, placeholder_text="Duration
in minutes")
    table_reservation_duration_entry.grid(row=row, column=1, padx=10, pady=5, sticky="w")
    entries["table_reservation_duration"] = table_reservation_duration_entry
    row += 1

לבחירת כשר או לא # RadioButtons
    kosher_var_a = ctk.StringVar(value="0")
    kosher_label = ctk.CTkLabel(add_window, text="Kosher (1 = Yes, 0 = No)").grid(row=row,
column=0, columnspan=2,
                                pady=10)
    kosher_radio_yes = ctk.CTkRadioButton(add_window, text="Yes", value="1",
variable=kosher_var_a)
    kosher_radio_no = ctk.CTkRadioButton(add_window, text="No", value="0",
variable=kosher_var_a)
    kosher_radio_yes.grid(row=row + 1, column=0, pady=5)
    kosher_radio_no.grid(row=row + 1, column=1, pady=5)

def on_submit():
    קבלת הערכים מהטופס #
    try:
        restaurant_data = {
            key: entry.get() for key, entry in entries.items()
        }

המרות טיפוסים לשדות שדורשים זאת #
        restaurant_data["table_reservation_duration"] =
int(restaurant_data["table_reservation_duration"])
        restaurant_data["average_price_per_person"] =
float(restaurant_data["average_price_per_person"])
        restaurant_data["kosher"] = int(kosher_var_a.get())
        restaurant_data["rating"] = float(restaurant_data["rating"])

        food_types = ["No Preference", "Italian", "Sushi", "Meat", "Cafa", "Shawarma",
"Hummus", "Burgers",
                    "Mexican", "Fish", "Doner"]
        districts = ["No Preference", "Gush Dan", "Sharon", "North", "Shfela", "Southeast",
"Galil", "South"]
        בדיקת פורמט הזמן #
        if not self.is_valid_time_format(restaurant_data["opening_time"]):
            messagebox.showerror("Error", "Invalid opening time format. Use HH: MM (24-hour
format).")
        return

```

```

        if not self.is_valid_time_format(restaurant_data["closing_time"]):
            messagebox.showerror("Error", "Invalid closing time format. Use HH:MM (24-hour
format).")
        return
        if not restaurant_data["food_type"] in food_types:
            messagebox.showerror("Error", "food type does not exist")
            return
        if not restaurant_data["district"] in districts:
            messagebox.showerror("Error", "district does not exist")
            return

    except ValueError:
        messagebox.showerror("Error", "Invalid numeric input in one or more fields.")
        return

שליחת הבקשה לשרת #
    request_data = {
        "action": "admin_add_restaurant",
        "restaurant": restaurant_data
    }

    self.client_comm.send(json.dumps(request_data).encode("utf-8"))
    response = self.client_comm.receive()

    if response:
        try:
            response_data = json.loads(response)
            if response_data.get("status") == "success":
                messagebox.showinfo("Success", "Restaurant added successfully.")
                add_window.destroy()
            else:
                messagebox.showerror("Error", response_data.get("message", "Failed to add
restaurant."))
        except json.JSONDecodeError:
            messagebox.showerror("Error", "Failed to decode server response.")
        else:
            messagebox.showerror("Error", "No response from server.")

    ctk.CTkButton(add_window, text="Add Restaurant", command=on_submit,
font=ctk.CTkFont(size=14),
                fg_color="#2ecc71", hover_color="#27ae60").grid(row=row + 2, column=0,
columnspan=2, pady=20)

def edit_tables(self):
    window = ctk.CTkToplevel(self.root)
    window.title("Edit Tables")
    window.geometry("400x300")
    window.configure(bg="#f9f9f9")

```

```

window.lift()
window.grab_set()
window.focus_force()

ctk.CTkLabel(window, text="Edit Tables", font=ctk.CTkFont(size=20,
weight="bold")).pack(pady=30)

ctk.CTkButton(
    window,
    text="Add Tables",
    command=self.add_tables,
ירוק    fg_color="#2ecc71", #
    hover_color="#27ae60",
    font=ctk.CTkFont(size=14),
    width=200,
    height=40
).pack(pady=10)

ctk.CTkButton(
    window,
    text="Remove Tables",
    command=self.remove_tables,
אדום    fg_color="#e74c3c", #
    hover_color="#c0392b",
    font=ctk.CTkFont(size=14),
    width=200,
    height=40
).pack(pady=10)

def add_tables(self):
    table_window = ctk.CTkToplevel(self.root)
    table_window.title("Add Tables")
    table_window.geometry("400x500")
    table_window.configure(bg="#ecf0f1")

    table_window.lift()
    table_window.grab_set()
    table_window.focus_force()

    ctk.CTkLabel(table_window, text="Add Tables to Restaurant", font=ctk.CTkFont(size=18,
weight="bold")).pack(
        pady=20)

    שם המסעדה    #
    ctk.CTkLabel(table_window, text="Restaurant Name").pack()
    name_entry = ctk.CTkEntry(table_window, placeholder_text="Restaurant Name")
    name_entry.pack(pady=5)

    - Indoor / Outdoorקטגוריה    #
    add_category_var = ctk.StringVar(value="Indoor")

```



```

    ctk.CTkLabel(table_window, text="Table Category").pack(pady=5)
    ctk.CTkRadioButton(table_window, text="Indoor", variable=add_category_var,
value="Indoor").pack()
    ctk.CTkRadioButton(table_window, text="Outdoor", variable=add_category_var,
value="Outdoor").pack()

גודל שולחן      #
    ctk.CTkLabel(table_window, text="Table Size (Number of Seats)").pack()
    size_entry = ctk.CTkEntry(table_window, placeholder_text="e.g. 4")
    size_entry.pack(pady=5)

מספר שולחנות    #
    ctk.CTkLabel(table_window, text="Number of Tables").pack()
    count_entry = ctk.CTkEntry(table_window, placeholder_text="e.g. 3")
    count_entry.pack(pady=5)

כפתור שליחה    #
def on_submit():
    restaurant_name = name_entry.get()
    category = add_category_var.get()
    try:
        table_size = int(size_entry.get())
        number_of_tables = int(count_entry.get())
    except ValueError:
        messagebox.showerror("Error", "Table size and number of tables must be integers.")
        return

    if not restaurant_name:
        messagebox.showerror("Error", "Please enter a restaurant name.")
        return

שליחת הבקשה לשרת      #
    request_data = {
        "action": "admin_add_table",
        "table_info": {
            "restaurant_name": restaurant_name,
            "category": category,
            "table_size": table_size,
            "number_of_tables": number_of_tables
        }
    }

    self.client_comm.send(json.dumps(request_data).encode("utf-8"))
    response = self.client_comm.receive()

    if response:
        try:
            response_data = json.loads(response)
            if response_data.get("status") == "success":
                messagebox.showinfo("Success", "Tables added successfully.")

```

```

        table_window.destroy()
    else:
        messagebox.showerror("Error", response_data.get("message", "Failed to add
tables. "))
    except json.JSONDecodeError:
        messagebox.showerror("Error", "Invalid response from server.")
    else:
        messagebox.showerror("Error", "No response from server.")

    ctk.CTkButton(table_window, text="Add Tables", command=on_submit,
font=ctk.CTkFont(size=14),
        fg_color="#2ecc71", hover_color="#27ae60").pack(pady=20)

def remove_tables(self):
    table_window = ctk.CTkToplevel(self.root)
    table_window.title("Remove Tables")
    table_window.geometry("400x500")
    table_window.configure(bg="#ecf0f1")

    table_window.lift()
    table_window.grab_set()
    table_window.focus_force()

    ctk.CTkLabel(table_window, text="Remove Tables to Restaurant",
font=ctk.CTkFont(size=18, weight="bold")).pack(
        pady=20)

שם המסעדה      #
    ctk.CTkLabel(table_window, text="Restaurant Name").pack()
    name_entry = ctk.CTkEntry(table_window, placeholder_text="Restaurant Name")
    name_entry.pack(pady=5)

- Indoor / Outdoor קטגוריה      #
    add_category_var = ctk.StringVar(value="Indoor")
    ctk.CTkLabel(table_window, text="Table Category").pack(pady=5)
    ctk.CTkRadioButton(table_window, text="Indoor", variable=add_category_var,
value="Indoor").pack()
    ctk.CTkRadioButton(table_window, text="Outdoor", variable=add_category_var,
value="Outdoor").pack()

גודל שולחן      #
    ctk.CTkLabel(table_window, text="Table Size (Number of Seats)").pack()
    size_entry = ctk.CTkEntry(table_window, placeholder_text="e.g. 4")
    size_entry.pack(pady=5)

מספר שולחנות      #
    ctk.CTkLabel(table_window, text="Number of Tables").pack()
    count_entry = ctk.CTkEntry(table_window, placeholder_text="e.g. 3")
    count_entry.pack(pady=5)

```

```

# כפתור שליחה
def on_submit():
    restaurant_name = name_entry.get()
    category = add_category_var.get()
    try:
        table_size = int(size_entry.get())
        number_of_tables = int(count_entry.get())
    except ValueError:
        messagebox.showerror("Error", "Table size and number of tables must be integers.")
        return

    if not restaurant_name:
        messagebox.showerror("Error", "Please enter a restaurant name.")
        return

# שליחת הבקשה לשרת
request_data = {
    "action": "admin_remove_table",
    "table_info": {
        "restaurant_name": restaurant_name,
        "category": category,
        "table_size": table_size,
        "number_of_tables": number_of_tables
    }
}

self.client_comm.send(json.dumps(request_data).encode("utf-8"))
response = self.client_comm.receive()

if response:
    try:
        response_data = json.loads(response)
        if response_data.get("status") == "success":
            messagebox.showinfo("Success", "Tables removed successfully.")
            table_window.destroy()
        else:
            messagebox.showerror("Error", response_data.get("message", "Failed to remove tables."))
    except json.JSONDecodeError:
        messagebox.showerror("Error", "Invalid response from server.")
    else:
        messagebox.showerror("Error", "No response from server.")

    ctk.CTkButton(table_window, text="Remove Tables", command=on_submit,
font=ctk.CTkFont(size=14),
fg_color="#e74c3c", hover_color="#c0392b").pack(pady=20)

def delete_restaurant(self):
    delete_window = ctk.CTkToplevel(self.root)
    delete_window.title("Delete Restaurant")

```

```

delete_window.geometry("400x300")
delete_window.configure(bg="#ecf0f1")

delete_window.lift()
delete_window.grab_set()
delete_window.focus_force()

ctk.CTkLabel(delete_window, text="Enter Restaurant Name to Delete",
             font=ctk.CTkFont(size=18, weight="bold")).pack(pady=20)

שדה הקלדה של שם המסעדה      #
restaurant_name_entry = ctk.CTkEntry(delete_window, placeholder_text="Restaurant
Name")
restaurant_name_entry.pack(pady=10)

def on_submit():
    restaurant_name = restaurant_name_entry.get()

    if not restaurant_name:
        messagebox.showerror("Error", "Please enter the restaurant name.")
        return

שליחת בקשה לשרת למחיקת המסעדה      #
request_data = {
    "action": "admin_delete_restaurant",
    "restaurant_name": restaurant_name
}

self.client_comm.send(json.dumps(request_data).encode("utf-8"))
response = self.client_comm.receive()

if response:
    try:
        response_data = json.loads(response)
        if response_data.get("status") == "success":
            cancelled_orders = response_data.get("cancelled_orders", [])

            def send_cancellation_emails():
                self.mail_sender = MailSender()
                for order in cancelled_orders:
                    self.mail_sender.send_cancel_restaurant_closed(
                        recipient_email=order["email"],
                        order=order
                    )

            threading.Thread(target=send_cancellation_emails).start()

            messagebox.showinfo("Success", response_data["message"])
            delete_window.destroy()
        else:

```

```

        messagebox.showerror("Error", response_data.get("message", "Failed to delete
restaurant. "))
    except json.JSONDecodeError:
        messagebox.showerror("Error", "Failed to decode server response.")
    else:
        messagebox.showerror("Error", "No response from server.")

    ctk.CTkButton(delete_window, text="Delete Restaurant", command=on_submit,
                  font=ctk.CTkFont(size=14),
                  fg_color="#e74c3c", hover_color="#c0392b").pack(pady=20)

def close_connection(self):
    if self.client_comm.socket:
        try:
            self.client_comm.send("exit")
            response = self.client_comm.receive()
            if response:
                print(response)
        except Exception as e:
            print(f"Error sending exit message: {e}")
        finally:
            try:
                if self.root.winfo_exists():
                    messagebox.showinfo("Connection Closed", "The connection to the server has been
closed.")
            except:
                pass
            self.client_comm.close()
            self.close_application()

def close_application(self):
    סוגר את כל החלונות בצורה נקייה #
    if self.root.winfo_exists():
        self.root.quit()
        self.root.destroy()

def run(self):
    self.root.mainloop()

```